MCA — MAJOR PROJECT

MAJOR PROJECT REPORT

Submitted in partial fulfilment of the requirements
for the award of the degree of

MASTER'S DEGREE PROGRAMME

in

MASTER OF COMPUTER APPLICATIONS (MCA)

Submitted by

PATEL MAN VISHNUBHAI

Enrollment No.: 25IC03CA053

Project Title: OPENCV-BASED SYSTEM FOR CRACK IDENTIFICATION AND WIDTH ESTIMATION

Under the guidance of

Dr. Srikant Singh

Academic Coordinator (B.Sc. IT/B.Sc. CS-AI), School of Engineering

P P SAVANI UNIVERSITY

SCHOOL OF ENGINEERING

P P SAVANI UNIVERSITY, SURAT, GUJARAT

OCTOBER 2026

Student Declaration and Certificate of Originality

I hereby declare that the project entitled "OpenCV-Based System for Crack Identification and Width Estimation" is my original work, carried out under the guidance of Dr. Srikant Singh for the major project component of MCA, Semester III, at P P Savani University, Surat. The work has not been submitted, in whole or in part, for any other degree, diploma or qualification. All sources, datasets, libraries, frameworks, and external contributions used in this project (OpenCV, NumPy, scikit-image, pandas, Flask, PyMongo, ReportLab and Bootstrap) have been appropriately acknowledged. I understand that plagiarism, fabricated results, or misrepresentation of project work may lead to disciplinary action.

Student Name: PATEL MAN VISHNUBHAI Enrollment/Roll No.: 25IC03CA053

Signature: _____ Date: _____

Guide Certificate

This is to certify that the project entitled "OpenCV-Based System for Crack Identification and Width Estimation" has been carried out by PATEL MAN VISHNUBHAI, Enrollment/Roll No. 25IC03CA053, under my guidance. To the best of my knowledge, the report presents the work completed by the student and is submitted for academic evaluation in accordance with the applicable institutional requirements.

Guide Name and Designation: Dr. Srikant Singh, Academic Coordinator (B.Sc. IT/B.Sc. CS-AI), School of Engineering

Signature: _____ Date: _____

Department / School: Academic Coordinator (B.Sc. IT/B.Sc. CS-AI), School of Engineering, P P Savani University, Surat, Gujarat

Acknowledgement

I express my sincere gratitude to my project guide, Dr. Srikant Singh, for the guidance, review and encouragement received throughout this project. I thank the faculty and staff of the School of Engineering, P P Savani University, for providing the academic environment and facilities needed to complete this work.

I also acknowledge the open-source communities behind OpenCV, NumPy, scikit-image, pandas, Flask, PyMongo and ReportLab, whose libraries form the technical foundation of this system, and the researchers whose published work on crack detection informed the design. Finally, I thank my family and friends for their continuous support.

PATEL MAN VISHNUBHAI

Abstract

Cracks in concrete walls, pavements and road surfaces are an early indicator of structural deterioration, but manual inspection is slow, subjective and difficult to document consistently. This project develops a web-based system that identifies candidate cracks in photographs and estimates their length and width using classical computer vision, without requiring a labelled training dataset. The objective is to give an inspector a repeatable, explainable measurement with every intermediate image visible.

The tool is written in Python and served by a Flask web application; OpenCV, scikit-image, NumPy and pandas do the image work and reporting, and MongoDB keeps the history (with a session-only fallback when no database is running). An uploaded photograph is scaled down, turned to grayscale, blurred, locally contrast-equalised (CLAHE) and converted to a binary mask by adaptive Gaussian thresholding, with a BlackHat image kept as an additional view. Morphological opening and closing, connected-component analysis and shape and distance rules then keep long, connected crack networks and discard text, round blobs and plain surface texture. Every crack is thinned to a one-pixel skeleton with scikit-image: its length is traced along that skeleton and its width is taken as twice the Euclidean distance-transform value on the centreline, reported as the median (mean width) and the 95th percentile (maximum width). A pixels-per-millimetre factor (default 10) gives millimetre values, and configurable width and area limits label each crack Good, Medium or Heavy. CSV, PDF and ZIP exports are produced for each analysis.

The implementation was verified with the repository's 16 automated tests (all passed in the test environment described in Chapter 7), an end-to-end browser run, and controlled experiments on synthetic textured images with known crack widths. In the width sweep, the mean-width estimate was within 0.6 px of the true rasterised width for bars 3 to 13 px wide, skeleton length exceeded the geometric length by about 6 to 7 percent on oblique cracks, faint cracks with contrast below about 35 grey levels were missed, and a 1600 x 1200 image was processed in about 0.16 s. No quantitative evaluation on annotated real photographs has been carried out, and severity labels are software categories, not structural safety judgements.

Keywords: crack detection; OpenCV; image segmentation; skeletonisation; distance transform; crack width estimation; Flask

Table of Contents

Chapter 1 — Introduction 1
1.1 Background and context of the problem 1
1.2 Problem statement 1
1.3 Need and motivation 1
1.4 Aim and measurable objectives 2
1.5 Scope 2
1.6 Proposed solution and expected contribution 3
1.7 Brief overview of the methodology 3
1.8 Organisation of the report 3
Chapter 2 — Literature Review and Existing-System Study 4
2.1 Search strategy and sources consulted 4
2.2 Review of relevant research and systems 4
2.3 Comparison of existing approaches 5
2.4 Identified gap 5
2.5 How the proposed project addresses the gap 5
2.6 Summary 6
Chapter 3 — System Analysis and Requirements 7
3.1 Preliminary investigation and user needs 7
3.2 Existing system and its limitations 7
3.3 Proposed system and benefits 7
3.4 Functional requirements 7
3.5 Non-functional requirements 9
3.6 Hardware and software requirements 9
3.7 Feasibility study 10
3.8 Constraints, assumptions, dependencies and risks 10
3.9 Software Requirements Specification (SRS) summary 10
Chapter 4 — Project Planning and Methodology 11
4.1 Development methodology and reason for selection 11
4.2 Project phases and deliverables 11
4.3 Work breakdown structure (WBS) 11
4.4 Gantt chart with planned and actual dates 11
4.5 PERT chart / dependency diagram 13
4.6 Roles and responsibilities 13
4.7 Risk register and mitigation plan 13
4.8 Dataset preparation and ethical considerations 13
Chapter 5 — System Design 15
5.1 Overall system architecture 15
5.2 Module decomposition and responsibilities 15
5.3 Data flow diagrams 16
5.4 Use-case diagram and descriptions 17

5.5 Activity (process) diagram 18
5.6 Entity–relationship diagram 20
5.7 Database schema 20
5.8 Input and output design 21
5.9 Algorithms and process logic 21
5.10 Security, privacy, validation and access-control design 22
5.11 Design decisions and trade-offs 22
5.12 Worked numerical example of the width measurement 23
5.13 Pseudo-code of the main algorithms 23
5.14 Complexity and performance considerations 25
5.15 Function reference 25
5.16 Analysis life-cycle and status values 26
5.17 User-interface design notes 26
Chapter 6 — Implementation 28
6.1 Development environment and tool versions 28
6.2 Project folder structure and configuration 28
6.3 Module-wise implementation 29
6.4 Important algorithms and code excerpts 29
6.5 Database implementation 30
6.6 API and external-service integration 30
6.7 Validation and error handling 30
6.8 Screenshots of working features 31
6.9 Build, run and deployment instructions 33
6.10 Project file structure 33
6.11 Source code screenshots with explanation 35
6.12 Additional user-interface screenshots 55
6.13 Generated output files 56
6.14 JSON API response 60
6.15 Step-by-step walkthrough of one analysis 61
6.16 Implementation challenges and how they were handled 68
Chapter 7 — Testing and Validation 69
7.1 Testing strategy and test environment 69
7.2 Unit testing 69
7.3 Integration testing 69
7.4 System testing 70
7.5 Performance and validation tests 70
7.6 Test results, defects found and fixes 70
7.7 Traceability matrix 71
7.8 Test code and results for each automated test 72
7.9 Test coverage assessment 79
Chapter 8 — Results, Discussion and Limitations 80
8.1 Implemented features and evidence of completion 80
8.2 Results against each project objective 80

8.3 Quantitative metrics and how they were measured 81
8.4 Comparison with baseline / existing method 82
8.5 Interpretation of results and unexpected outcomes 82
8.6 Known limitations and unresolved issues 83
8.7 User feedback or evaluation 84
8.8 Additional experiments: robustness and sensitivity 84
8.9 Discussion 88
8.10 Threats to validity 89
Chapter 9 — Conclusion and Future Scope 90
9.1 Summary of the problem and solution 90
9.2 Objective-wise completion status 90
9.3 Main contributions and conclusions 90
9.4 Limitations 90
9.5 Future enhancements 90
9.6 Learning outcomes and skills demonstrated 91
References 92
Appendices / Annexures 94
Appendix A — Approved synopsis / proposal and approval record 94
Appendix B — Detailed Software Requirements Specification 94
Appendix C — Database dictionary and sample report 94
Appendix D — Detailed test cases and test evidence 95
Appendix E — User manual / installation and execution guide 95
Appendix F — Repository details and selected source 96
Appendix G — Progress reports / mentor review records 113
Appendix H — Questionnaire, consent form or data-collection instrument 114
Appendix I — Team-member contribution statement 114

List of Figures

Figure 4.1: Gantt chart: progress-report schedule and actual implementation activity 12
Figure 5.1: Layered architecture of the system 15
Figure 5.2: Context-level (Level 0) data flow diagram 16
Figure 5.3: Level 1 data flow diagram 17
Figure 5.4: Use-case diagram 17
Figure 5.5: Processing workflow for an uploaded batch 19
Figure 5.6: Logical data model (MongoDB collections and relationships) 20
Figure 6.1: Processing stages for a synthetic crack (drawn width 6 px) 31
Figure 6.2: Upload page with two selected images 31
Figure 6.3: Results page: summary, remark and downloads 32
Figure 6.4: Per-image results: original, annotated, mask and every processing stage 32
Figure 6.5: First page of the generated PDF report 33
Figure 6.6: File and folder structure of the project 34
Figure 6.7: app.py, lines 1 to 34 35
Figure 6.8: app.py, lines 35 to 54 36
Figure 6.9: config.py, lines 1 to 34 36
Figure 6.10: config.py, lines 35 to 48 36
Figure 6.11: services/image_service.py, lines 1 to 20 37
Figure 6.12: services/preprocessing_service.py, lines 1 to 33 37
Figure 6.13: services/detection_service.py, lines 1 to 34 38
Figure 6.14: services/detection_service.py, lines 35 to 68 38
Figure 6.15: services/detection_service.py, lines 69 to 102 39
Figure 6.16: services/detection_service.py, lines 103 to 130 39
Figure 6.17: services/measurement_service.py, lines 1 to 34 40
Figure 6.18: services/measurement_service.py, lines 35 to 68 40
Figure 6.19: services/measurement_service.py, lines 69 to 102 41
Figure 6.20: services/measurement_service.py, lines 103 to 136 41
Figure 6.21: services/measurement_service.py, lines 137 to 164 41
Figure 6.22: services/classification_service.py, lines 1 to 16 42
Figure 6.23: services/calibration_service.py, lines 1 to 10 42
Figure 6.24: services/storage_service.py, lines 1 to 15 42
Figure 6.25: services/csv_service.py, lines 1 to 34 43
Figure 6.26: services/csv_service.py, lines 35 to 57 43
Figure 6.27: services/zip_service.py, lines 1 to 34 44
Figure 6.28: services/zip_service.py, lines 35 to 50 44
Figure 6.29: models/database.py, lines 1 to 31 45
Figure 6.30: models/analysis_model.py, lines 1 to 34 45
Figure 6.31: models/analysis_model.py, lines 35 to 56 46
Figure 6.32: models/image_model.py, lines 1 to 28 46
Figure 6.33: models/measurement_model.py, lines 1 to 34 47
Figure 6.34: models/measurement_model.py, lines 35 to 43 47

Figure 6.35: routes/upload_routes.py, lines 1 to 34 48
Figure 6.36: routes/upload_routes.py, lines 35 to 68 48
Figure 6.37: routes/upload_routes.py, lines 69 to 102 48
Figure 6.38: routes/upload_routes.py, lines 103 to 136 49
Figure 6.39: routes/upload_routes.py, lines 137 to 169 49
Figure 6.40: routes/analysis_routes.py, lines 1 to 34 50
Figure 6.41: routes/analysis_routes.py, lines 35 to 68 50
Figure 6.42: routes/analysis_routes.py, lines 69 to 98 50
Figure 6.43: routes/history_routes.py, lines 1 to 29 51
Figure 6.44: routes/report_routes.py, lines 1 to 34 51
Figure 6.45: routes/report_routes.py, lines 35 to 48 51
Figure 6.46: utils/validators.py, lines 1 to 22 52
Figure 6.47: utils/file_utils.py, lines 1 to 15 52
Figure 6.48: static/app.js, lines 1 to 34 53
Figure 6.49: static/app.js, lines 35 to 68 53
Figure 6.50: static/app.js, lines 69 to 102 54
Figure 6.51: static/app.js, lines 103 to 113 54
Figure 6.52: requirements.txt, lines 1 to 10 54
Figure 6.53: .env.example, lines 1 to 5 54
Figure 6.54: Upload page before selecting files 55
Figure 6.55: History page 55
Figure 6.56: Download-report card 56
Figure 6.57: Severity thresholds card, shown in pixels and millimetres 56
Figure 6.58: Per-image result card for the first synthetic image 56
Figure 6.59: Crack measurement table in pixels and millimetres 56
Figure 6.60: CSV report opened as a table (selected columns) 57
Figure 6.61: PDF report, page 2 57
Figure 6.62: PDF report, page 3 58
Figure 6.63: PDF report, page 4 59
Figure 6.64: Input image 61
Figure 6.65: Grayscale 62
Figure 6.66: Gaussian blur (5 x 5) 62
Figure 6.67: CLAHE 63
Figure 6.68: BlackHat transform 63
Figure 6.69: Adaptive threshold 64
Figure 6.70: Morphological opening and closing 64
Figure 6.71: Component mask 65
Figure 6.72: Skeleton 65
Figure 6.73: Distance transform 66
Figure 6.74: Width estimation 66
Figure 6.75: Annotated result 67
Figure 6.76: Second test image: three separate synthetic cracks 67
Figure 6.77: Annotated result: cracks C1, C2 and C3 measured independently 68
Figure 7.1: tests/test_app.py, lines 1 to 34 75

Figure 7.2: tests/test_app.py, lines 35 to 67 75
Figure 7.3: tests/test_processing.py, lines 1 to 34 76
Figure 7.4: tests/test_processing.py, lines 35 to 68 76
Figure 7.5: tests/test_processing.py, lines 69 to 102 77
Figure 7.6: tests/test_processing.py, lines 103 to 136 77
Figure 7.7: tests/test_processing.py, lines 137 to 170 78
Figure 7.8: tests/test_processing.py, lines 171 to 193 78
Figure 8.1: Input, annotated result, skeleton and width-estimation image 80
Figure 8.2: Measured versus true raster crack width 81
Figure 8.3: Detection rate versus noise level 85
Figure 8.4: Measured length versus contrast (dashed line: geometric length 578.9 px) 85
Figure 8.5: Length error versus line angle 86
Figure 8.6: Measured width versus image scale 88
Figure 9.1: Progress report schedule 114

List of Tables

Table 1.1: Objectives, planned outputs and verification evidence 2
Table 2.1: Comparison of related approaches 5
Table 3.1: Functional requirements 7
Table 3.2: Non-functional requirements 9
Table 3.3: Hardware and software requirements 9
Table 4.1: Project phases and deliverables 11
Table 4.2: Planned schedule of progress reports 12
Table 4.3: Activities and actual evidence 12
Table 4.4: Risk register 13
Table 5.1: Modules and responsibilities 15
Table 5.2: Use-case descriptions 18
Table 5.3: Design decisions and trade-offs 22
Table 5.4: Distance-transform values for a 5 px wide, 20 px long bar (every second column shown) 23
Table 5.5: Measurement output for the worked example 23
Table 5.6: Function reference 25
Table 6.1: Main configuration parameters (config.py) 28
Table 6.2: Module-wise inputs, processing and outputs 29
Table 6.3: HTTP interface 30
Table 6.4: Size of each part of the repository (lines, from `wc -l`) 34
Table 6.5: Implementation challenges 68
Table 7.1: Test cases and results 70
Table 7.2: Requirements-to-tests traceability 71
Table 7.3: Test coverage assessment 79
Table 8.1: Objective-wise results 80
Table 8.2: Measured width against true raster width on synthetic images (geometric polyline length 578.9 px) 81
Table 8.3: Effect of background noise on detection and width 84
Table 8.4: Effect of crack contrast on detection 85
Table 8.5: Length and width of a straight crack at different angles 86
Table 8.6: Sensitivity to the threshold offset (texture SD 10) 87
Table 8.7: Sensitivity to the threshold neighbourhood size 87
Table 8.8: Same scene at five image scales 87
Table 8.9: Mean stage time over 20 images of 1024 x 768 px with one crack 88
Table 9.1: External interface specification 94
Table 9.2: Logical database dictionary 94
Table 9.3: Sample CSV measurement rows 95
Table 9.4: Commit history summary 96
Table 9.5: Larger source files (line counts); the repository has about 1,990 lines in total 96

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
BGR	Blue-Green-Red colour channel order used by OpenCV
CLAHE	Contrast Limited Adaptive Histogram Equalisation
CSV	Comma-Separated Values
CV	Computer Vision
DFD	Data Flow Diagram
DT	Distance Transform
ER	Entity-Relationship
FR / NFR	Functional / Non-Functional Requirement
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
MCA	Master of Computer Applications
NDT	Non-Destructive Testing
PCA	Principal Component Analysis
PDF	Portable Document Format
PERT	Program Evaluation and Review Technique
PNG / JPEG	Portable Network Graphics / Joint Photographic Experts Group
ppm	Pixels per millimetre (calibration factor)
P95	95th percentile
SRS	Software Requirements Specification
SVD	Singular Value Decomposition
UI	User Interface
UUID	Universally Unique Identifier
WBS	Work Breakdown Structure

Chapter 1 — Introduction

1.1 Background and context of the problem

Surfaces of concrete, masonry and asphalt develop cracks for many reasons: drying shrinkage, temperature swings, heavy loads, foundation movement and plain ageing. Some cracks are only a cosmetic nuisance. Others let moisture and chlorides reach the steel reinforcement, shorten the life of the structure or point to a deeper structural fault. For this reason inspectors note the position, length and width of each crack and come back later to see whether it has grown.

On site, width is normally read with a crack gauge or comparator card and length with a tape, and the findings are written down by hand. The method is time-consuming over large areas, varies from one inspector to another, cannot be repeated exactly, and leaves almost no visual record. Since photographs are inexpensive to take and their geometry can be extracted by software, an automated and documented measurement workflow is a natural improvement.

1.2 Problem statement

Given one or more photographs of a concrete, wall or pavement surface, the system must (i) decide whether the image contains crack-like structures, (ii) segment those structures, (iii) measure the length and width of each crack, in pixels and, when a scale is known, in millimetres, (iv) assign an easily understood severity category, and (v) present and export the results together with the intermediate images so that a human can check them. The affected users are inspectors, civil-engineering students and maintenance staff who currently perform these steps by hand.

1.3 Need and motivation

- Manual gauge-based measurement is slow and subjective; an automated pipeline gives the same output for the same image every time.
- Many published crack detectors are deep-learning models that need large labelled datasets and offer limited insight into why a result was produced. A classical pipeline can be inspected stage by stage, runs on an ordinary CPU and needs no training data.
- Detection alone is not enough for condition assessment; width and length are the quantities inspectors actually record, so the project concentrates on measurement as well as detection.
- A web interface with exportable CSV and PDF reports makes the tool usable by people who do not write code.

1.4 Aim and measurable objectives

The aim of the project is to design, implement and test an OpenCV-based web application that identifies cracks in images and estimates their length and width. The measurable objectives and the evidence used to verify each are summarized in Table 1.1.

No.	Objective	Planned feature / output	Evidence used to verify completion
O1	Accept and validate crack photographs	Multi-file upload (JPG, JPEG, JFIF, PNG; at most 20 files; 16 MB request limit)	Upload tests TC-02, TC-03; end-to-end run TC-17
O2	Segment crack-like regions	Resize, grayscale, Gaussian blur, CLAHE, adaptive threshold, morphology, component filtering	Processing tests TC-04, TC-09 to TC-14; stage images (Fig. 6.1)
O3	Reject non-crack images	Shape, coherence, coverage and alignment validation	TC-07, TC-08, TC-16, TC-21
O4	Measure crack length and width	Skeleton length; distance-transform width; median and P95 statistics	TC-05, TC-15, TC-22; Table 8.2
O5	Convert to physical units and classify	px/mm calibration; Good / Medium / Heavy labels	TC-06
O6	Present and export results	Results page, annotated images, CSV, PDF and ZIP packages	TC-17 to TC-20; Figs. 6.2 to 6.5
O7	Keep a history of analyses	MongoDB-backed history with in-memory fallback; delete and search	Code review (Chapter 6); TC-17

Table 1.1: Objectives, planned outputs and verification evidence

1.5 Scope

Included: single or batch image upload; automatic pre-processing, segmentation, validation and measurement; calibration with a configurable default of 10 pixels per millimetre; severity labelling by configurable width and area thresholds; annotated images and all processing stages; CSV, PDF and ZIP exports; stored analysis history; automated tests.

Boundaries: the system analyses still images only (no video or drone streams); it targets visible surface cracks on concrete, wall and pavement images; it does not estimate depth, does not correct perspective or lens distortion, and does not read a scale automatically from the image. Severity categories are software labels defined by the configured thresholds and are not a professional structural safety assessment. There is no user authentication, because the application is intended for local, single-user use.

1.6 Proposed solution and expected contribution

The proposed solution is a Flask web application whose back end applies a deterministic OpenCV pipeline to each uploaded image. The contribution of the project is an end-to-end, explainable crack-measurement tool: it combines contrast enhancement,

adaptive thresholding, shape-based filtering that suppresses texture, per-crack skeletonisation, a robust width statistic that is not dominated by junctions or blobs, and a complete reporting workflow, all in one open-source repository that can be run on a standard computer.

1.7 Brief overview of the methodology

An iterative, incremental approach was used. A first working pipeline and web application were committed on 5 October 2026; subsequent iterations on 12 October 2026 reduced pavement-texture false positives, restricted annotations to coherent crack paths, supported several crack networks per image and stabilised width measurement; the repository was finalised on 18 October 2026. Each iteration was checked with synthetic test images added to the automated test suite. Chapter 4 describes the method in more detail.

1.8 Organisation of the report

Chapter 2 reviews related work and existing approaches. Chapter 3 analyses the requirements. Chapter 4 describes planning and methodology. Chapter 5 presents the system design. Chapter 6 describes the implementation. Chapter 7 covers testing and validation. Chapter 8 reports results, discussion and limitations. Chapter 9 concludes and lists future scope. References and appendices follow.

Chapter 2 — Literature Review and Existing-System Study

2.1 Search strategy and sources consulted

Sources were identified by searching Google Scholar and the publisher sites of journals in civil-infrastructure informatics and computer vision using the terms "crack detection", "image processing", "concrete", "pavement", "crack width", "skeleton" and "convolutional neural network". Surveys were used to find primary papers. Official documentation for OpenCV, scikit-image, Flask and MongoDB was consulted for implementation details.

2.2 Review of relevant research and systems

Surveys. Koch et al. (2015) surveyed vision-based defect detection and condition assessment for concrete and asphalt structures, describing the chain from image capture to detection and then assessment. Mohan and Poobal (2018) examined image-processing crack detectors critically and pointed out how strongly they depend on lighting, surface texture and noise.

Classical image processing. Yamaguchi and Hashimoto (2010) described a percolation-based approach aimed at quickly finding cracks in large concrete images. Typical classical pipelines filter the image, threshold it or detect edges, clean the result morphologically and analyse connected components. Otsu (1979) chooses one global threshold from the grey-level histogram, while adaptive thresholding computes it locally and tolerates uneven illumination. CLAHE (Zuiderveld, 1994) raises local contrast while limiting noise amplification.

Deep learning. Zhang et al. (2016) classified road-image patches with a deep convolutional network, and Cha et al. (2017) detected concrete cracks with a convolutional network that needs no hand-designed features. Özgenel and Gönenç Sorguç (2018) compared pretrained networks on building crack images and published a commonly used concrete crack dataset (Özgenel, 2019). Dorafshan et al. (2018) set deep networks against classical edge detectors on concrete images; the networks did better on their data, whereas edge detectors need no training set.

Skeletons and distance transforms. Thinning methods such as Zhang and Suen (1984) shrink a binary shape to a one-pixel centreline, and a distance transform (Borgefors, 1986) assigns each foreground pixel its distance to the nearest background pixel.

Combined, they measure the local width of a long thin shape: the distance value on the centreline equals half the width there.

2.3 Comparison of existing approaches

Author / year	Problem	Method / technology	Findings (as reported)	Limitations	Relevance to this project
Koch et al., 2015	Survey of CV defect detection for concrete and asphalt	Literature review	Organises detection and assessment methods and open problems	Review only; no implementation	Frames the detection-to-assessment pipeline
Mohan & Poobal, 2018	Critical review of image-processing crack detection	Literature review	Summarises classical methods and their issues with noise and lighting	Review only	Motivates CLAHE, adaptive thresholding and filtering
Yamaguchi & Hashimoto, 2010	Fast crack detection in large concrete images	Percolation-based image processing	Designed for efficient processing of large images	Detection focus; width not the main output	Classical, training-free precedent
Zhang et al., 2016	Road crack detection	Deep CNN on image patches	CNN learns crack features from data	Needs labelled training data; output is detection	Contrast with training-free approach
Cha et al., 2017	Concrete crack detection	CNN without hand-crafted features	Detects cracks under varied conditions	Training data and GPU resources; limited geometry output	Shows the deep-learning alternative
Dorafshan et al., 2018	CNN vs edge detectors for concrete cracks	DCNN and classical edge detection	CNNs better on their data; edge detectors need no training	Dataset-specific	Supports a hybrid role for classical methods
This project	Detection plus length and width measurement	OpenCV adaptive pipeline, skeleton, distance transform, Flask	See Chapter 8	No real-photo benchmark yet	Delivers measurement and reporting

Table 2.1: Comparison of related approaches

2.4 Identified gap

Most published work concentrates on classifying or segmenting cracks. Fewer open implementations give an inspector a complete workflow that (a) runs without training data or a GPU, (b) shows every intermediate image, (c) reports width and length in physical units with a robust width statistic, and (d) exports an audit-ready report. Deep-learning approaches also depend on the similarity between the training set and the surface being inspected.

2.5 How the proposed project addresses the gap

The project implements a transparent classical pipeline and couples it to measurement (skeleton length, median and P95 width), calibration, severity classification, history and export. Because no learning is involved, behaviour is controlled by documented parameters in `config.py`, and each stage image can be inspected in the results page.

2.6 Summary

Classical pipelines are fast, interpretable and training-free but sensitive to texture and lighting; deep-learning models are more robust on matching data but need labelled datasets and give less direct geometric output. This project adopts the classical route, adds texture-suppression filters and a validation step, and focuses on reproducible measurement and reporting.

Chapter 3 — System Analysis and Requirements

3.1 Preliminary investigation and user needs

The intended users are site inspectors, maintenance engineers and civil-engineering students who photograph a surface and want to know whether and how badly it is cracked. Their needs, as identified from the problem context in Chapter 1, are: upload several photographs at once; obtain results without tuning parameters; see where the software found cracks; obtain numbers in millimetres; keep a record; and export results for a report or a spreadsheet. The needs were captured from the project guide's brief and from the literature, not from a formal user survey.

3.2 Existing system and its limitations

- Manual inspection with gauges and tape: accurate at a point, but slow, subjective and poorly documented.
- General image-editing or measurement tools: require the user to trace cracks by hand and calibrate manually.
- Research detectors (often deep-learning): need training data and expertise to run, and typically output masks or boxes rather than width and length reports.

3.3 Proposed system and benefits

The proposed system automates detection and measurement behind a browser interface. Benefits are repeatability (same image, same result), speed (about 0.16 s for a 1600 x 1200 image in the test environment), transparency (all stages saved), physical units, and ready-to-share CSV, PDF and ZIP reports.

3.4 Functional requirements

ID	Requirement	Priority	Verification / acceptance criteria
FR-01	Upload one or more images (JPG, JPEG, JFIF, PNG), up to 20 per analysis	High	Valid upload accepted; empty batch rejected with HTTP 400 (TC-02)
FR-02	Validate each file: extension, non-empty, decodable, dimensions between 2 px and 8000 px	High	Invalid extension rejected (TC-03)
FR-03	Resize to at most 1600 px on the longest side, grayscale, Gaussian blur, CLAHE, BlackHat and adaptive threshold	High	Stage images saved and shown (TC-04, TC-17)
FR-04	Clean the binary mask with morphology; extract connected components; drop regions below 30 px area	High	TC-04, TC-09
FR-05	Keep elongated crack-like regions and coherent groups; reject round blobs, text and dispersed regions	High	TC-07, TC-08, TC-11, TC-16, TC-21
FR-06	Reject an upload batch in which an image contains no detectable crack	Medium	Rejection message with HTTP 422 (code review; TC-21 for logic)
FR-07	Skeletonise each crack; compute length, area, mean and maximum width	High	TC-05, TC-15, TC-22
FR-08	Convert measurements to millimetres using a positive pixels-per-millimetre value (default 10)	High	TC-06
FR-09	Classify each crack as Good, Medium or Heavy using configurable thresholds	Medium	TC-06
FR-10	Produce annotated image, skeleton, distance-transform and width-estimation images	Medium	TC-17; Figs. 6.1, 6.4
FR-11	Show a results page with summary, remark, thresholds, stage images and measurement table	High	TC-17; Figs. 6.3, 6.4
FR-12	Export CSV, PDF and ZIP reports	High	TC-18 to TC-20
FR-13	Store analyses, images and measurements; list, search and delete history	Medium	Code review; TC-17
FR-14	Continue to work, with session-only storage, if MongoDB is unavailable	Medium	All tests and runs in this report used the fallback

Table 3.1: Functional requirements

3.5 Non-functional requirements

ID	Requirement	Priority	Measurable threshold / review method
NFR-01	Performance: process a 1600 x 1200 image in under 1 s on a standard CPU	High	Measured about 0.16 s (Section 8.3)
NFR-02	Security: sanitise file names, restrict served files to the analysis folder, enforce upload size limit	High	`safe_filename`, `is_within`, `MAX_CONTENT_LENGTH` (Section 5.10)
NFR-03	Usability: no parameter entry needed for a normal run; image previews and zoomable result images	Medium	Manual walkthrough (Figs. 6.2 to 6.4)
NFR-04	Reliability: invalid or undecodable images never crash the request	High	TC-02, TC-03; error handling in `_process_analysis`
NFR-05	Portability: run on Windows, Linux or macOS with Python 3.10+	Medium	Pure-Python dependencies; README
NFR-06	Maintainability: modular layers; thresholds in one configuration file	Medium	Folder structure (Section 6.2)
NFR-07	Transparency: every pipeline stage is visible to the user	Medium	Stage grid (Fig. 6.4)

Table 3.2: Non-functional requirements

3.6 Hardware and software requirements

Item	Minimum / used
Processor / memory	Any modern 64-bit CPU; 4 GB RAM recommended (no GPU required)
Storage	Space for uploaded images and generated stage images (a two-image analysis produced a 4.3 MB ZIP)
Operating system	Windows 10/11, Linux or macOS
Language / runtime	Python 3.10 or later
Libraries (requirements.txt)	Flask 3, python-dotenv, pymongo 4.6+, opencv-python-headless 4.9+, numpy 1.26+, scikit-image 0.22+, pandas 2.1+, reportlab 4+, pytest 8+, mongomock 4.1+
Database	MongoDB Community Server or Atlas (optional; in-memory fallback otherwise)
Browser	Any current browser; Bootstrap 5.3.3 is loaded from a CDN

Table 3.3: Hardware and software requirements

3.7 Feasibility study

Technical: all required algorithms (CLAHE, adaptive thresholding, morphology, connected components, distance transform, skeletonisation) exist in OpenCV and scikit-image; the project ran on a CPU-only machine.

Operational: the browser interface needs no training; defaults work without configuration.

Economic: every component is open-source; no licence cost. MongoDB is optional.

Schedule: a first working version was committed on 5 October 2026 and refined on 12 and 18 October 2026 (Chapter 4).

3.8 Constraints, assumptions, dependencies and risks

Constraints: image-only input; no depth or perspective correction; fixed default scale of 10 px/mm.
Assumptions: cracks appear darker than the surrounding surface; the image is reasonably in focus; the supplied scale is correct for the resized image.
Dependencies: OpenCV, scikit-image, Flask, pandas, ReportLab; Bootstrap from a CDN for styling.
Risks: false positives on textured or stained surfaces, shadows, joints and markings; false negatives on faint or very thin cracks; scale error if the real pixel size differs from the default. Mitigation is described in Section 4.7.

3.9 Software Requirements Specification (SRS) summary

The system shall provide the functions FR-01 to FR-14 and satisfy NFR-01 to NFR-07. Inputs are image files and, internally, a pixels-per-millimetre value; outputs are annotated images, a measurement table (area, length, mean and maximum width in px and mm, severity), and CSV, PDF and ZIP files. External interfaces are an HTTP interface served by Flask on 127.0.0.1:5000 and an optional MongoDB connection (`MONGO_URI`). A fuller SRS is given in Appendix B.

Chapter 4 — Project Planning and Methodology

4.1 Development methodology and reason for selection

An iterative, incremental model was chosen because image-processing parameters cannot be fixed in advance: each change to a threshold or filter affects several outputs, so each increment was tested on synthetic images before the next was added. The repository history reflects this: an initial full application, followed by several focused refinements (texture suppression, coherent-path filtering, multi-crack support, normal-direction width sampling and stable width statistics).

4.2 Project phases and deliverables

Phase	Deliverable
Requirement analysis and literature study	Objectives (Table 1.1), requirements (Chapter 3), reference list
Design	Architecture, data-flow, ER and process diagrams (Chapter 5)
Implementation	Flask application, processing services, MongoDB models, report generators
Refinement	Texture suppression, validation rules, multi-crack and width-stability improvements
Testing and validation	Automated tests, end-to-end run, synthetic accuracy experiment (Chapter 7)
Documentation	README and this report

Table 4.1: Project phases and deliverables

4.3 Work breakdown structure (WBS)

1 Analysis: 1.1 problem definition; 1.2 literature study; 1.3 requirements.

2 Design: 2.1 architecture; 2.2 pipeline design; 2.3 data model; 2.4 user interface.

3 Implementation: 3.1 upload and validation; 3.2 pre-processing; 3.3 detection and filtering; 3.4 measurement and classification; 3.5 persistence; 3.6 reports; 3.7 web pages.

4 Verification: 4.1 unit tests; 4.2 end-to-end test; 4.3 accuracy experiment.

5 Documentation: 5.1 README; 5.2 project report.

4.4 Gantt chart with planned and actual dates

Progress was reviewed with the guide through ten weekly progress reports between 16 July and 24 September 2026 (Table 4.2). The repository holds the actual implementation dates from its Git history (5, 12 and 18 October 2026). Figure 4.1 shows

both: the diamonds are the progress-report dates and the bars are the commit-based implementation activity, followed by testing and report preparation on 18 October 2026.

Reporting	Date
Progress Report 1	16 July 2026
Progress Report 2	23 July 2026
Progress Report 3	06 August 2026
Progress Report 4	13 August 2026
Progress Report 5	20 August 2026
Progress Report 6	27 August 2026
Progress Report 7	03 September 2026
Progress Report 8	10 September 2026
Progress Report 9	17 September 2026
Progress Report 10	24 September 2026

Table 4.2: Planned schedule of progress reports

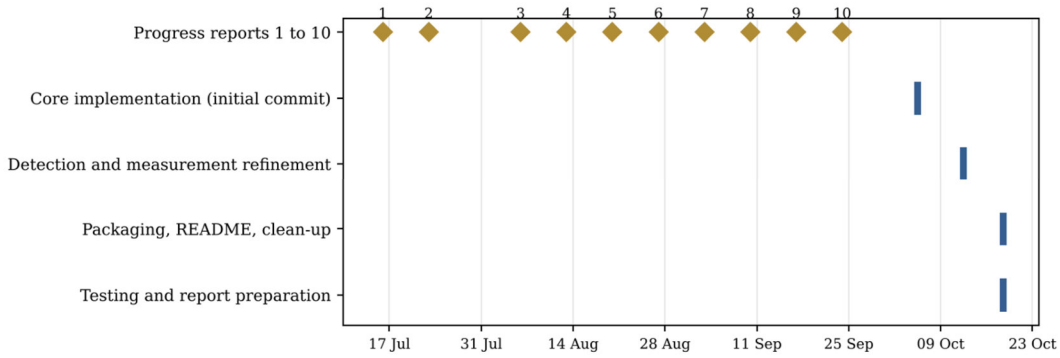


Figure 4.1: Gantt chart: progress-report schedule and actual implementation activity

Task / phase	Actual dates / deliverable evidence	Status
Requirement analysis and literature study	Weekly progress reports 1 to 10 (16 Jul to 24 Sep 2026); Chapters 2 and 3	Complete
Design	Progress-report period; Chapter 5	Complete
Core implementation	05 Oct 2026 initial commit	Complete
Detection / measurement refinement	12 Oct 2026 (nine commits)	Complete
Packaging and clean-up	18 Oct 2026 (README, launcher removal)	Complete
Testing and report	18 Oct 2026; Chapter 7	Complete

Table 4.3: Activities and actual evidence

4.5 PERT chart / dependency diagram

The critical path follows the data flow of the system: Requirements, then Design, then the processing services (pre-processing, detection, measurement), then web routes and reports, then Testing, then Documentation. Persistence (MongoDB models) and report generation could proceed in parallel once the measurement output format was

fixed; the user interface depended on the routes. Figure 5.5 shows the processing dependency at run time.

4.6 Roles and responsibilities

This is an individual project. PATEL MAN VISHNUBHAI carried out analysis, design, implementation, testing and documentation under the guidance of Dr. Srikant Singh.

4.7 Risk register and mitigation plan

Risk	Impact	Mitigation in this project
Textured pavement or stains produce false cracks	High	Adaptive threshold with offset 16; minimum area; aspect/PCA ratio filter; coherent-group filter; coverage limit (commits of 12 Oct 2026)
Non-crack images (text, faces, objects) accepted	High	`validate_crack_image` with component count, coverage and alignment tests; rejection of the whole batch with a message
Width distorted by junctions and blobs	Medium	Median of junction-excluded centreline samples for mean width; 95th percentile for maximum width
Wrong scale gives wrong millimetres	High	Positive-value validation; scale shown on the results page; limitation documented
MongoDB unavailable	Medium	Start-up ping with 1.5 s timeout; in-memory fallback store
Large or malicious uploads	Medium	16 MB limit, 20 files, 8000 px limit, extension and decode checks, filename sanitising

Table 4.4: Risk register

4.8 Dataset preparation and ethical considerations

The system itself needs no training dataset. For testing, synthetic images were generated in code (Gaussian-noise background with polylines of known width, and negative images such as plain surface, round blob, text and texture only). No personal data, faces or private property images were collected for this report. Real photographs used in future evaluation should be taken with the owner's permission and stored without personal metadata.

Chapter 5 — System Design

5.1 Overall system architecture

The application follows a layered architecture (Figure 5.1). The presentation layer is a set of Jinja2 templates styled with Bootstrap 5 and a small JavaScript file (`static/app.js`) that handles file previews, upload submission and the image lightbox. Flask blueprints in `routes/` form the application layer and contain only request handling. The image-processing logic lives in `services/` and has no dependency on Flask, which allows it to be unit-tested directly. The data layer consists of model modules that talk to MongoDB (or an in-memory dictionary) and a `storage/` folder that holds image files and reports; images are deliberately not stored in the database.

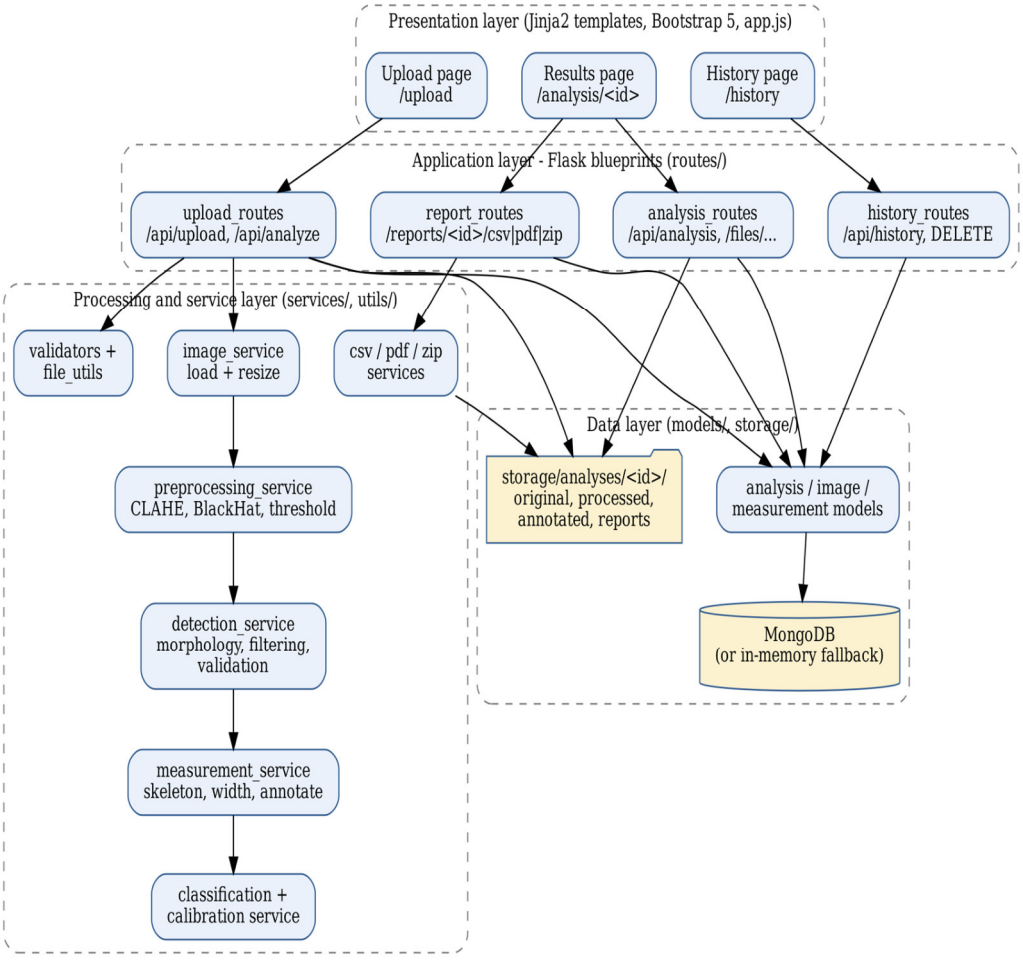


Figure 5.1: Layered architecture of the system

5.2 Module decomposition and responsibilities

Module	Responsibility
app.py, config.py	Application factory, blueprint registration, error handlers; configuration, limits and all processing parameters
routes/upload_routes.py	Upload page, /api/upload and /api/analyze; orchestrates validation, storage and processing per image
routes/analysis_routes.py	Results page, JSON API and protected file serving
routes/history_routes.py	History page, search API, delete API
routes/report_routes.py	On-demand CSV, PDF and ZIP generation and download
services/image_service.py	Image decoding, aspect-preserving resize, image writing
services/preprocessing_service.py	Grayscale, Gaussian blur, CLAHE, BlackHat, thresholding
services/detection_service.py	Morphology, connected components, crack filtering, image validation
services/measurement_service.py	Skeletonisation, length, width, diagnostic images, annotation
services/classification_service.py, calibration_service.py	Good / Medium / Heavy rule; pixel-to-millimetre conversion
services/csv_service.py, pdf_service.py, zip_service.py	Report generation (pandas, ReportLab, zipfile)
models/*.py	MongoDB access and in-memory fallback for analyses, images and measurements
utils/validators.py, file_utils.py	Upload validation, safe file names, path-containment check

Table 5.1: Modules and responsibilities

5.3 Data flow diagrams

Figure 5.2 is the context (level 0) diagram: the user supplies images and receives annotated results and reports, while the system reads and writes a persistent store. Figure 5.3 decomposes the system into six processes.

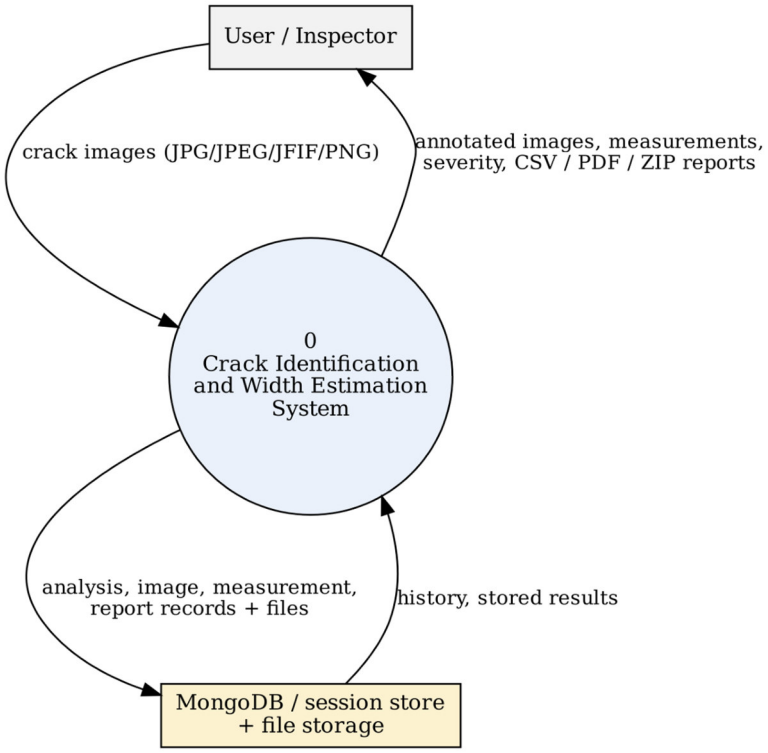


Figure 5.2: Context-level (Level 0) data flow diagram

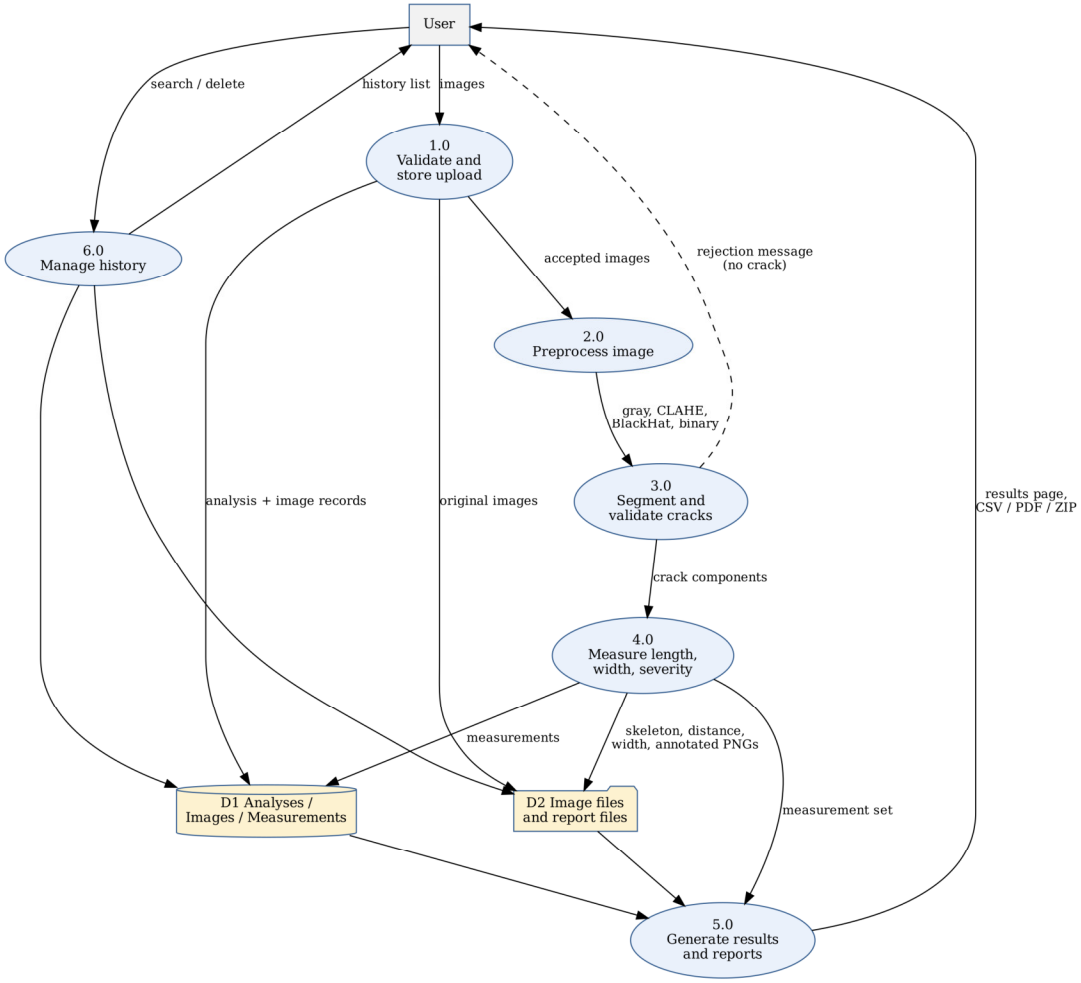


Figure 5.3: Level 1 data flow diagram

5.4 Use-case diagram and descriptions

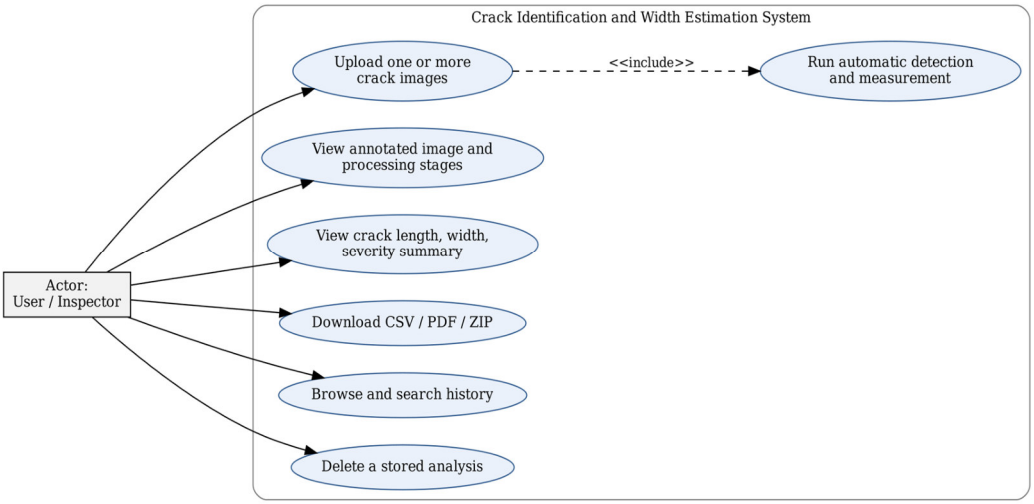


Figure 5.4: Use-case diagram

Table 5.2: Use-case descriptions

Use case	Description
Upload images	Precondition: browser open at /upload. The user selects 1 to 20 images; previews appear; submitting starts the analysis. Postcondition: the user is taken to the results page, or sees an error message.
View results	The user sees the summary (count, total length, average and maximum width, severity counts), remark, severity thresholds, per-image annotated results, stage images and the measurement table.
Download reports	The user chooses CSV, PDF or ZIP; the file is generated on demand.
Browse and delete history	The user lists earlier analyses, searches by name and may delete one, which removes database records and files.

5.5 Activity (process) diagram

Figure 5.5 shows the processing flow executed for each upload. If any image in a batch is rejected as containing no crack, or cannot be processed, the whole analysis (records and files) is removed and an error is returned; otherwise measurements are stored and the analysis is marked Completed.

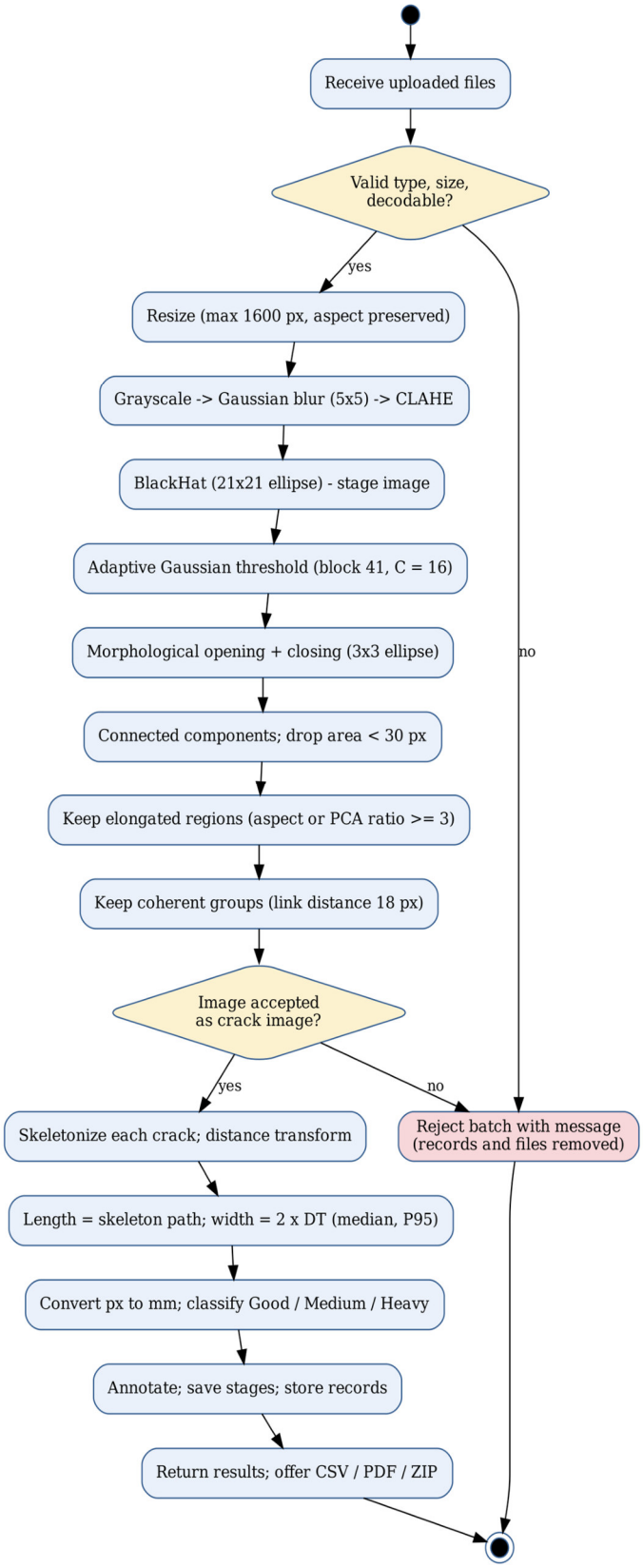


Figure 5.5: Processing workflow for an uploaded batch

5.6 Entity–relationship diagram

Three collections are used: `analyses`, `images` and `crack_measurements`. One analysis has many images and many measurements; each image has many measurements. A fourth index, `report_history`, is created at start-up but no code in the repository currently writes to it.

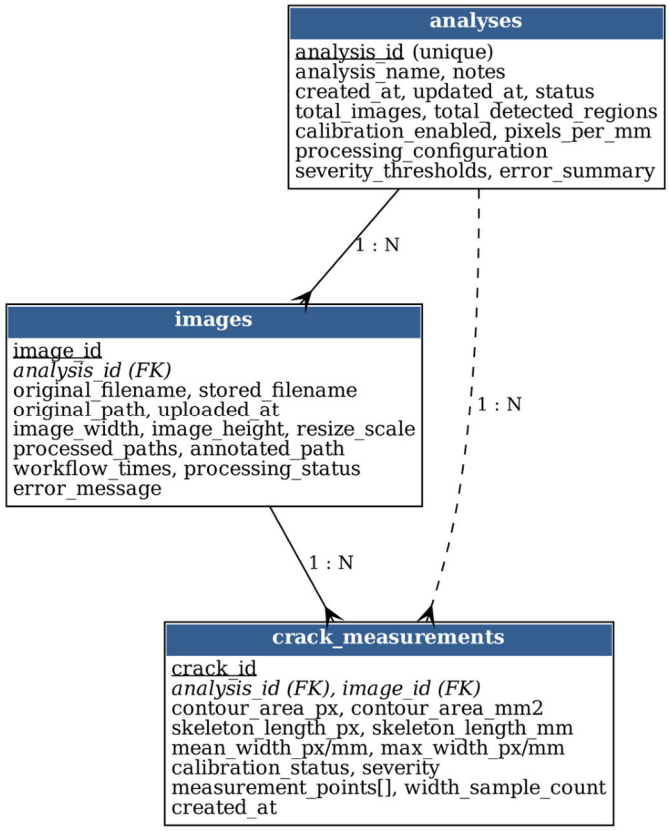


Figure 5.6: Logical data model (MongoDB collections and relationships)

5.7 Database schema

MongoDB is schema-less; the logical schema in Appendix C lists the fields written by the models. Indexes created at start-up are: unique index on `analyses.analysis_id`; compound index on `images (analysis_id, uploaded_at desc)`; compound index on `crack_measurements (analysis_id, image_id)`; and on `report_history (analysis_id, generated_at desc)`. Identifiers are UUID hexadecimal strings; relationships are by `analysis_id` and `image_id`, not by database references.

5.8 Input and output design

Input is a drag-and-drop or file-picker area that accepts `.jpg`, `.jpeg`, `.jfif` and `.png` files, shows thumbnails with a Remove button and submits all files in one request. Output is the results page (Section 6.8) plus downloadable CSV (one row per crack with analysis-level summary columns), PDF and a ZIP package containing all original, processed and annotated images, the reports and a text manifest.

5.9 Algorithms and process logic

Pre-processing. The picture is first turned into grayscale and blurred with a 5 x 5 Gaussian filter. CLAHE (clip limit 2.0, 8 x 8 tiles) evens out local contrast. A BlackHat operation using a 21 x 21 elliptical element brings out thin dark lines and is stored as a viewing stage only. The binary mask itself comes from an inverted adaptive Gaussian threshold (neighbourhood 41 px, constant 16) on the grayscale image; if no adaptive block size is configured, the code instead thresholds the BlackHat image by percentile or Otsu.

Detection. The mask is opened and then closed with a 3 x 3 ellipse to remove specks and close single-pixel gaps, and components smaller than 30 px are discarded. A component passes as crack-like when its bounding rotated rectangle has an aspect ratio of at least 3, or when the ratio of the two largest singular values of its pixel coordinates (a PCA-style elongation measure) reaches 3. Passing components lying within 18 px of one another are merged into groups, and a group survives if its area is large enough compared with the largest group. Finally, the image is judged to contain a crack if the component count, the covered fraction (up to 8 percent) and the alignment of component centres stay within the configured limits, or if one dominant elongated component exists (a second rule that tolerates up to 15 percent coverage).

Measurement. Each component is thinned to a skeleton with scikit-image. Its length is obtained by adding 1 for every horizontal or vertical link between neighbouring skeleton pixels and the square root of 2 for every diagonal link. An OpenCV Euclidean distance transform (5 x 5 mask) gives the inscribed-circle radius at every pixel, so the local width on the centreline is twice that radius. Skeleton pixels with three or more skeleton neighbours (junctions) are left out; the median of the remaining samples is the mean width and their 95th percentile is the maximum width. About twenty evenly spaced

samples per crack are also drawn on the annotated image, aligned with the local normal obtained from an SVD of neighbouring skeleton points.

Classification. A crack is labelled Heavy when its area reaches 5000 px² or its mean width is above 8 px, Medium when its mean width is above 3 px, and Good otherwise. The limits are in pixels and are saved with each analysis.

5.10 Security, privacy, validation and access-control design

Upload validation: allowed extensions, non-empty content, successful decode with OpenCV, and dimension limits (2 to 8000 px).

`MAX_CONTENT_LENGTH` (16 MB by default) and a 413 JSON error handler; at most 20 files per analysis.

File names are sanitised with `safe_filename` and prefixed with a UUID; served files must resolve inside the analysis folder (`is_within`).

Secrets (`FLASK_SECRET_KEY`, `MONGO_URI`) come from environment variables or `.env`, which is excluded from the repository by `.gitignore`.

The application binds to 127.0.0.1 and has no login; it is not designed for public deployment (see limitations).

5.11 Design decisions and trade-offs

Decision	Alternative	Reason / trade-off
Classical OpenCV pipeline	Deep-learning segmentation	No labelled data or GPU needed; every stage inspectable. Less robust on unseen textures.
Adaptive Gaussian threshold	Global Otsu on BlackHat	Handles uneven lighting and keeps thin lines; the offset must be tuned, although it had no visible effect on a high-contrast crack in the sensitivity tests (Section 8.8).
Median and P95 of centreline widths	Mean and maximum	Robust against junctions and end blobs; hides the single widest point.
Files on disk, metadata in MongoDB	GridFS or Base64	Simpler and faster to serve images; backup must cover both.
In-memory fallback store	Fail if MongoDB missing	Easier to try and test; data are lost when the process ends.
Reject the whole batch if one image has no crack	Skip that image	Keeps results consistent with the "crack images only" rule; one bad file discards a batch.

Table 5.3: Design decisions and trade-offs

5.12 Worked numerical example of the width measurement

To make the measurement stage concrete, a perfect rectangular bar 20 pixels long and 5 pixels wide was passed through the same OpenCV distance transform and scikit-image skeletonisation that the application uses. Table 5.4 shows the distance-transform values across the bar (rows 4 to 8 of a 13 x 24 mask; every other row is zero). The values rise from 1 at the border to 3 at the centre line, and the centre row is the skeleton.

Row	c2	c4	c6	c8	c10	c12	c14	c16	c18	c20
row 4	1	1	1	1	1	1	1	1	1	1
row 5	1	2	2	2	2	2	2	2	2	2
row 6	1	3	3	3	3	3	3	3	3	2
row 7	1	2	2	2	2	2	2	2	2	2
row 8	1	1	1	1	1	1	1	1	1	1

Table 5.4: Distance-transform values for a 5 px wide, 20 px long bar (every second column shown)

At the centre row the distance value is 3.0, so the radius of the largest inscribed circle is 3 and the width reported is 2 x 3.0 = 6.0 px, one pixel more than the true width of 5 px. This one-pixel offset is a property of the distance-transform convention (the distance is measured to the nearest background pixel, not to the pixel edge). The skeleton of this bar has length 16.414 px (the bar is 20 px long, and thinning shortens each end by a few pixels); with the default scale of 10 px/mm the mean width is 0.6 mm, and the crack is classified Medium. The same bar gives a contour area of 100 px² and 15 centre-line width samples.

Quantity	Value for the 5 x 20 px bar
Area (px²)	100
Skeleton length (px)	16.414
Mean width, median of samples (px)	6.0
Maximum width, 95th percentile (px)	6.0
Mean width (mm at 10 px/mm)	0.6
Severity	Medium
Width samples	15

Table 5.5: Measurement output for the worked example

5.13 Pseudo-code of the main algorithms

The algorithms are given in language-neutral pseudo-code. They correspond to the functions described in Section 5.15 and to the source code shown in Chapter 6.

Algorithm 1 — Pre-processing

```
ALGORITHM 1  Pre-processing (image, config)
  gray      <- convert image to grayscale
  blurred   <- GaussianBlur(gray, k x k)
  enhanced  <- CLAHE(blurred, clipLimit, tileGrid)
  blackhat  <- Morphology(enhanced, BLACKHAT, ellipse(21))   # visual stage
  IF adaptive block size is configured THEN
      binary <- AdaptiveThreshold(gray, GAUSSIAN, INVERTED, block, C)
  ELSE IF percentile configured THEN binary <- blackhat >= percentile
  ELSE binary <- Otsu(blackhat)
  RETURN gray, blurred, enhanced, blackhat, binary
```

Algorithm 2 — Component extraction

```
ALGORITHM 2  Component extraction (binary, config)
  cleaned <- CLOSE(OPEN(binary, ellipse(3)), ellipse(3))
  labels, stats <- ConnectedComponents(cleaned, 8-connectivity)
  FOR each label with area >= minimum_region_area
      mask    <- pixels of that label
      contour <- largest external contour of mask
      store {label, area, mask, contour}
  RETURN cleaned, union of kept masks, component list
```

Algorithm 3 — Crack-likeness filter and grouping

```
ALGORITHM 3  Crack-likeness filter (components, config)
  FOR each component
      aspect <- longer / shorter side of minimum-area rectangle
      ratio  <- first / second singular value of the pixel coordinates
      IF aspect >= 3 OR ratio >= 3 THEN keep component
  groups <- cluster kept components whose bounding boxes are
            within 18 px of each other (transitive closure)
  keep groups whose area >= 80 px AND
       (area >= 8% of the largest group OR (>= 2 components AND area >= 160))
  IF none kept THEN keep the largest group
  RETURN components of kept groups
```

Algorithm 4 — Crack-image validation

```
ALGORITHM 4  Image validation (components, image_shape, config)
  IF components is empty THEN RETURN false
  coverage  <- total component area / image area
  alignment <- first / second singular value of component centres
  IF count <= 60 AND coverage <= 0.08 AND
     (count <= 8 OR alignment >= 3) THEN RETURN true
  dominant <- component with the largest area
  RETURN dominant.area/image_area >= 0.003 AND dominant.aspect >= 2.4
         AND coverage <= 0.15
```

Algorithm 5 — Crack measurement

```
ALGORITHM 5  Measurement (component, pixels per mm, thresholds)
  skeleton <- Skeletonize(component.mask)
  dist     <- DistanceTransform(component.mask, L2, 5x5)
  samples  <- skeleton pixels with fewer than 3 skeleton neighbours
  widths   <- 2 * dist[p] for p in samples
  mean_w   <- median(widths);  max_w <- percentile(widths, 95)
  length   <- sum over neighbour links: 1 (axis) or sqrt(2) (diagonal)
  convert to mm if pixels_per_mm is available
  severity <- Classify(area, mean_w, thresholds)
  pick about 20 evenly spaced samples; angle <- normal to local tangent (SVD)
  RETURN measurements and diagnostic images
```

Algorithm 6 — Severity classification

```
ALGORITHM 6  Classification (area, mean_width, thresholds)
  IF area >= heavy_min_area OR mean_width > medium_max_width THEN "Heavy"
  ELSE IF mean_width > good_max_width THEN "Medium"
  ELSE "Good"
```

5.14 Complexity and performance considerations

Pre-processing is linear in the number of pixels (a few image-wide passes). Connected-component labelling is linear in pixels. Filtering is linear in the number of components, except the grouping step, which compares components pairwise and is quadratic in the number of retained components (limited in practice by the area filter). Measurement is the most expensive step: for every skeleton pixel the code searches the skeleton-point array for neighbours, so the cost grows roughly with the square of the skeleton length. This agrees with the timing results in Section 8.8, where measurement took about 0.057 s of a 0.095 s total on a 1024 x 768 image. If very long skeletons ever become a problem, replacing the array search with a neighbour lookup in a set would reduce the cost to linear.

5.15 Function reference

Function	File	Purpose
create_app(config_class)	app.py	Builds the Flask app, storage folders, database and blueprints
init_db(app)	models/database.py	Connects to MongoDB (1.5 s timeout), creates indexes, or enables fallback
get_db(), get_fallback_store()	models/database.py	Return the live handle or the in-memory dictionaries
create_analysis, get_analysis, update_analysis, list_analyses, delete_analysis_records	models/analysis_model.py	CRUD for analyses; delete cascades to images, measurements and reports
create_image, list_images, update_image	models/image_model.py	Per-image records and status updates
create_measurements, list_measurements, update_measurement_calibration	models/measurement_model.py	Per-crack records; recalibration helper (not called by any route)
load_and_resize(path, max_dimension)	services/image_service.py	Decode and downscale preserving aspect ratio; returns image and scale
write_image(path, image)	services/image_service.py	Create folder and write PNG; raise on failure
preprocess(image, config)	services/preprocessing_service.py	Algorithm 1
detect_components(binary, config)	services/detection_service.py	Algorithm 2
filter_crack_components(components, config)	services/detection_service.py	Algorithm 3 (with `_select_coherent_path`, `_component_gap`)

Function	File	Purpose
validate_crack_image(components, shape, config)	services/detection_service.py	Algorithm 4 (with `_component_alignment_ratio`)
measure_component(component, ppm, thresholds)	services/measurement_service.py	Algorithm 5 (with `_skeleton_length`, `_robust_widths`)
annotate(image, components, measurements)	services/measurement_service.py	Draw contours, labels C1.. and width sample lines
classify, threshold_description	services/classification_service.py	Algorithm 6 and a text description of the limits
validate_pixels_per_mm, convert_pixels	services/calibration_service.py	Positive-number check and px to mm conversion
analysis_dir, remove_analysis	services/storage_service.py	Create or delete `storage/analyses/<id>`
create_csv	services/csv_service.py	One row per crack plus analysis summary columns (pandas)
create_pdf	services/pdf_service.py	ReportLab A4 report with tables, images and footer
create_manifest, create_zip	services/zip_service.py	Text manifest and ZIP of all analysis files and reports
validate_upload(file, config)	utils/validators.py	Extension, empty, decodable and size checks
safe_filename, is_within	utils/file_utils.py	Sanitise names; ensure a path stays inside a base folder
upload, analyze	routes/upload_routes.py	HTTP entry points for upload and processing
analysis_page, analysis_api, file_route, processed_file_route	routes/analysis_routes.py	Results page, JSON API and image serving
history, history_api, delete	routes/history_routes.py	History page, search and delete
reports, build_report, download_report	routes/report_routes.py	Report page and on-demand report generation

Table 5.6: Function reference

5.16 Analysis life-cycle and status values

An analysis moves through the states Uploaded (records created and originals saved), Processing (set at the start of `_process_analysis`) and Completed (set after measurements are stored). Each image has its own status: Uploaded, Completed or Failed. When any image is rejected or fails, the analysis is not marked Failed; instead all of its records and files are deleted and an error is returned to the user, so no partial analysis ever appears in the history. This keeps the history clean but means that a failed batch cannot be inspected afterwards, which is noted as a limitation.

5.17 User-interface design notes

- A fixed top navigation bar carries the application name; the upload page is the landing page.

- The upload form previews every selected image with a Remove button; the submit button triggers an asynchronous request, and the status area reports success or the server's error message.
- The results page orders information from summary to detail: summary card, remark, download buttons, severity thresholds, then one card per image with original, annotated and mask images, a grid of every processing stage, and finally the measurement table.
- Every image can be opened in a lightbox with zoom in, zoom out and reset controls and keyboard and wheel support (`static/app.js`).
- Colour is used for severity badges (green, amber, red) together with text labels, so the meaning does not depend on colour alone.

Chapter 6 — Implementation

6.1 Development environment and tool versions

The code targets Python 3.10 or later and the libraries in `requirements.txt`. The tests, experiments and screenshots in this report were produced in a Linux container with Python 3.13.16, OpenCV 5.0.0.93, NumPy 2.5.3, scikit-image 0.26.0, pandas 3.0.5, ReportLab 5.0.1 and Flask 3.1.3. The repository is hosted at https://github.com/ManPatel448/OpenCV-Based-System-for-Crack-Identification-and-Width-Estimation.git

6.2 Project folder structure and configuration

```
app.py                    Flask application factory and entry point
config.py                 Limits and PROCESSING parameters
requirements.txt          Python dependencies
.env.example              MONGO_URI, MONGO_DB_NAME, FLASK_SECRET_KEY,
...
models/                   database.py, analysis_model.py,
image_model.py, measurement_model.py
routes/                   upload_routes.py, analysis_routes.py,
history_routes.py, report_routes.py
services/                 image, preprocessing, detection, measurement,
classification,
                          calibration, storage, csv, pdf, zip services
utils/                    validators.py, file_utils.py
templates/                base, upload, results, history, reports,
analysis_not_found
static/                   app.css, app.js
storage/                  runtime output (analyses/<id>/original_images,
processed_images,
                          annotated_images, reports)
tests/                    test_app.py, test_processing.py
```

Repository layout

All tunable values are in `config.py`. The main processing parameters are listed in Table 6.1.

Parameter	Value	Purpose
resize_max_dimension	1600	Longest image side after resize
gaussian_kernel	5	Blur kernel size
clahe_clip_limit / clahe_tile_grid	2.0 / 8	Local contrast enhancement
blackhat_kernel	21	Elliptical structuring element for BlackHat
adaptive_threshold_block_size / offset	41 / 16	Adaptive Gaussian threshold
morphology_kernel / iterations	3 / 1	Opening and closing
minimum_region_area	30 px	Smallest kept component
minimum_crack_aspect_ratio	3.0	Elongation test
crack_link_distance_px	18	Grouping distance for components
maximum_crack_components / coverage	60 / 8 %	Validation limits
maximum_dispersed_components	8	Components allowed without alignment
minimum_crack_path_alignment	3.0	Alignment ratio of component centres

Parameter	Value	Purpose
dominant crack: area ratio / aspect / coverage	0.003 / 2.4 / 15 %	Dominant-crack acceptance rule
severity thresholds	Good ≤ 3 px; Medium ≤ 8 px; Heavy > 8 px or ≥ 5000 px²	Classification
DEFAULT_PIXELS_PER_MM	10.0	Calibration

6.3 Module-wise implementation

Module	Inputs	Processing	Outputs
Upload route	Multipart files	Validate; save originals; create analysis and image records; loop over images	JSON with analysis id, or error 400 / 422
Image service	Image path	Decode as colour; scale down if larger than 1600 px	BGR array, scale factor
Pre-processing	BGR image	Grayscale, blur, CLAHE, BlackHat, adaptive threshold	Dictionary of five stage images
Detection	Binary mask	Morphology, components, shape and group filters, validation	Cleaned mask, component list, accept flag
Measurement	Component	Skeleton, distance transform, robust width, severity	Measurement dict and three diagnostic images
Reports	Analysis, images, measurements	pandas to CSV; ReportLab PDF; zipfile	CSV, PDF, ZIP files

Table 6.2: Module-wise inputs, processing and outputs

6.4 Important algorithms and code excerpts

Two short excerpts from the repository illustrate the core of the method; the remaining logic is described in Section 5.9.

Excerpt 1 shows the elongation test that rejects round blobs and generic texture.

Excerpt 1 — services/detection_service.py, filter_crack_components (abridged)

```
_, (width, height), _ = cv2.minAreaRect(component["contour"])
aspect_ratio = max(width, height) / max(min(width, height), 1.0)
points = np.argwhere(component["mask"] > 0)
_, sv, _ = np.linalg.svd(points - points.mean(axis=0), full_matrices=False)
principal ratio = float(sv[0] / max(sv[1], 1e-6))
if aspect_ratio >= minimum_aspect or principal_ratio >= minimum_aspect:
    crack_components.append(component)
```

Excerpt 2 shows how width is derived from the skeleton and the distance transform, and how the robust statistics are taken.

Excerpt 2 — services/measurement_service.py, measure_component (abridged)

```
skeleton = skeletonize(mask)
distance = cv2.distanceTransform(component["mask"], cv2.DIST_L2, 5)
widths = _robust_widths(distance, np.argwhere(skeleton))   # 2 * distance,
junctions skipped
mean_width = float(np.median(widths))
max_width  = float(np.percentile(widths, 95))
length     = _skeleton_length(skeleton)                    # 1 or sqrt(2) per
neighbour link
```

6.5 Database implementation

`models/database.py` creates a PyMongo client with a 1.5 s server-selection timeout and pings the server. On success it stores the database handle and creates the indexes listed in Section 5.7; on any exception it sets `mongo_available` to false and leaves the handle empty. Every model function checks the handle and, if it is empty, reads or writes a dictionary kept in the Flask application extensions. Appendix C lists the fields of each collection and a sample CSV row.

6.6 API and external-service integration

Method and path	Purpose
GET /upload	Upload page
POST /api/upload, POST /api/analyze	Upload and analyse; JSON result (`ok`, `analysis_id`, ...) or error
GET /analysis/<id>	Results page (404 page if unknown)
GET /api/analysis/<id>	Analysis, images and measurements as JSON
GET /files/<id>/<image>/original | annotated	Original or annotated image
GET /files/<id>/<image>/processed/<stage>	A processing-stage image
GET /history, GET /api/history?q=	History page; JSON search by analysis name
DELETE /api/analysis/<id>	Delete records and files
GET /reports/<id>/csv | pdf | zip	Generate and download a report

Table 6.3: HTTP interface

The only external services are the optional MongoDB server and the Bootstrap stylesheet, which is loaded from a CDN.

6.7 Validation and error handling

`validate_upload` checks the extension, reads the file to confirm it is non-empty and decodable, and checks dimensions. `_validate_files` rejects an empty selection or more than 20 files. Errors are returned as JSON with status 400 (invalid input), 413 (too large) or 422 (no crack found or processing error). During processing, an exception for

one image marks it Failed, and the batch is then removed and reported; an unknown analysis id gives a 404 page.

6.8 Screenshots of working features

Figure 6.1 shows the main processing stages for the synthetic image used in the accuracy experiment; the other screenshots below were captured from the running application in the test environment, using synthetic crack images. Because the sandbox could not reach the Bootstrap CDN, the page was rendered with a small local stand-in stylesheet for Bootstrap's grid, card and button classes; page content, data and layout logic are the application's own.

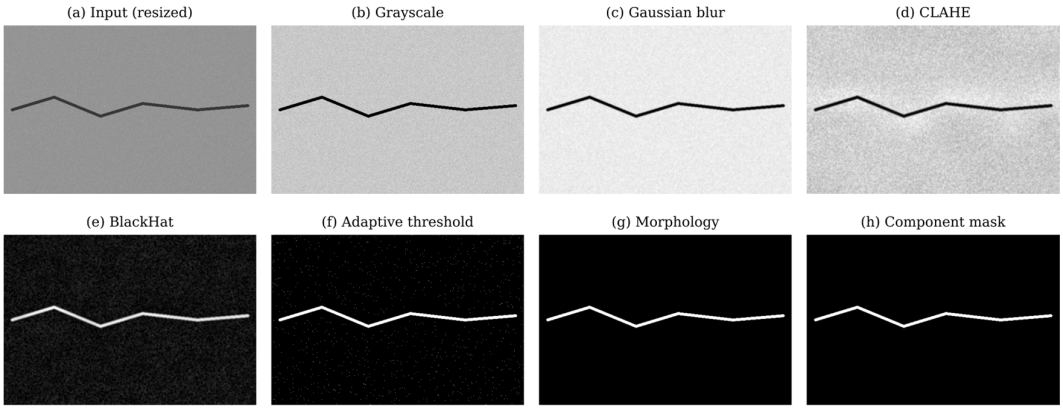


Figure 6.1: Processing stages for a synthetic crack (drawn width 6 px)

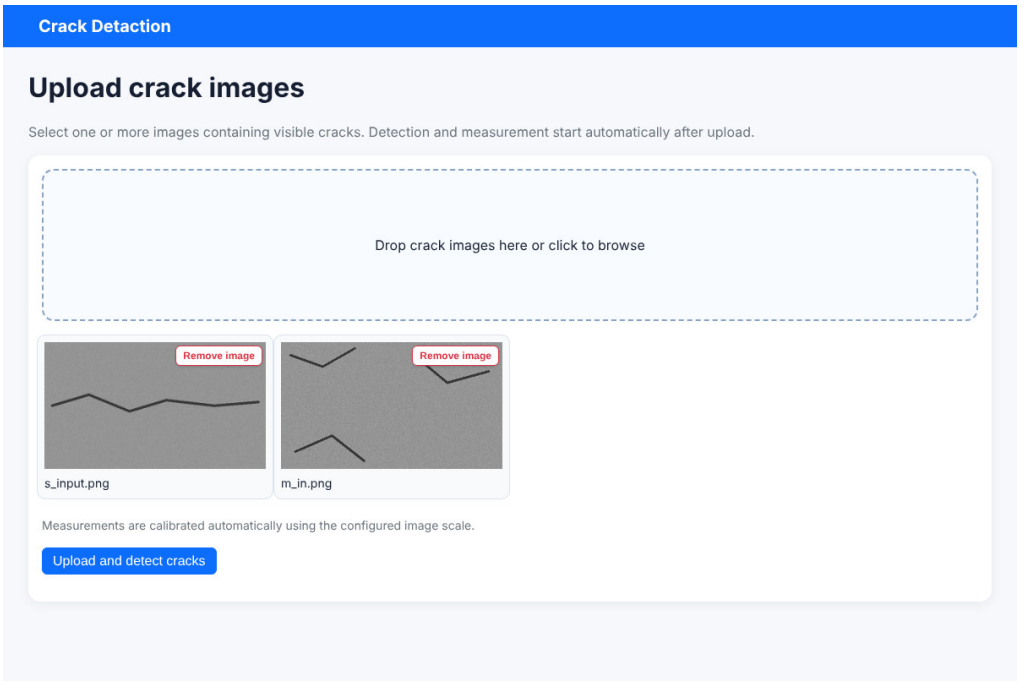


Figure 6.2: Upload page with two selected images

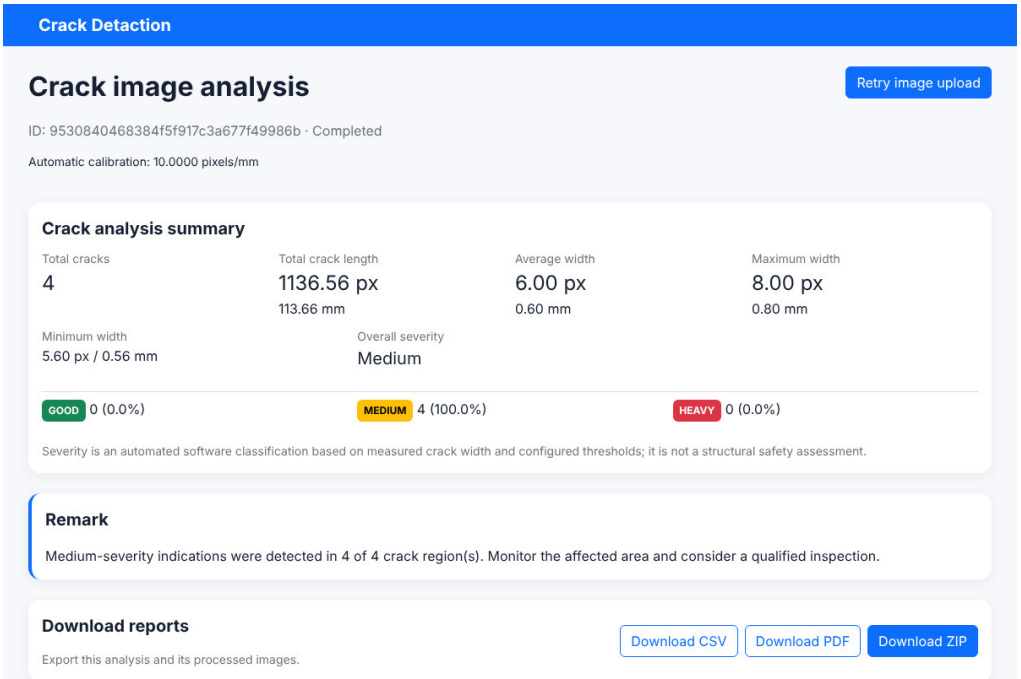


Figure 6.3: Results page: summary, remark and downloads

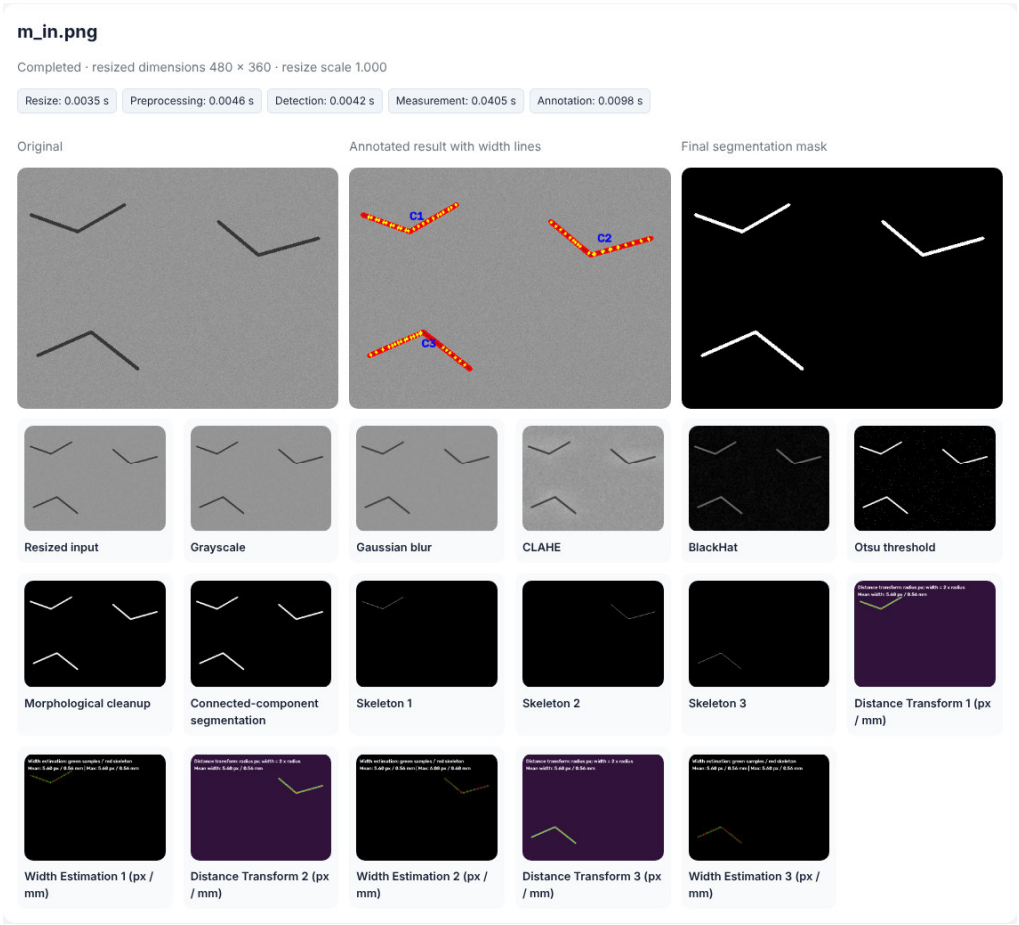
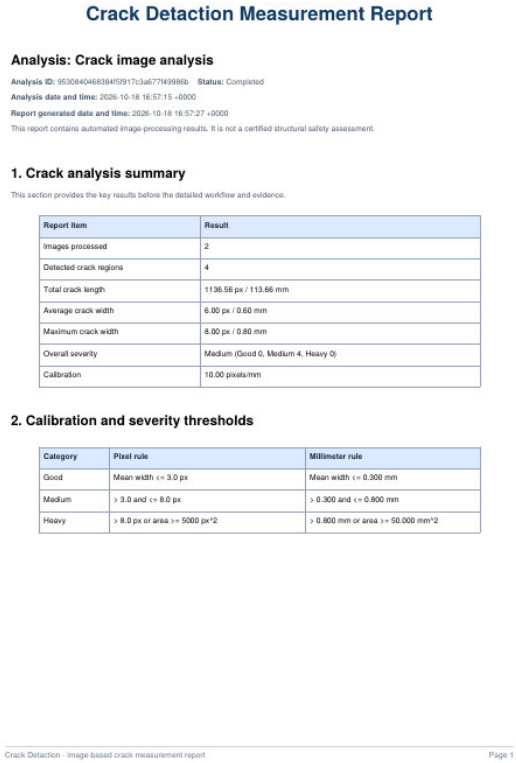


Figure 6.4: Per-image results: original, annotated, mask and every processing stage



Crack Detaction Measurement Report

Analysis: Crack image analysis

Analysis ID: 9530840468384f5f917c3a67714998b Status: Completed

Analysis date and time: 2026-10-18 16:57:15 +0000

Report generated date and time: 2026-10-18 16:57:27 +0000

This report contains automated image-processing results. It is not a certified structural safety assessment.

1. Crack analysis summary

This section provides the key results before the detailed workflow and evidence.

Report item	Result
Images processed	2
Detected crack regions	4
Total crack length	1136.56 px / 113.66 mm
Average crack width	6.00 px / 0.60 mm
Maximum crack width	8.00 px / 0.80 mm
Overall severity	Medium (Good 0, Medium 4, Heavy 0)
Calibration	10.00 pixels/mm

2. Calibration and severity thresholds

Category	Pixel rule	Millimeter rule
Good	Mean width <= 3.0 px	Mean width <= 0.300 mm
Medium	> 3.0 and <= 8.0 px	> 0.300 and <= 0.800 mm
Heavy	> 8.0 px or area >= 5000 px^2	> 0.800 mm or area >= 50.000 mm^2

Crack Detaction - image-based crack measurement report Page 1

Figure 6.5: First page of the generated PDF report

6.9 Build, run and deployment instructions

1. Install Python 3.10 or later; optionally install MongoDB Community Server or create an Atlas cluster.
2. Clone the repository and create a virtual environment: `python -m venv .venv` and activate it.
3. Install dependencies: `pip install -r requirements.txt`.
4. Copy `.env.example` to `.env` and set `MONGO_URI`, `MONGO_DB_NAME` and `FLASK_SECRET_KEY`.
5. Start the server: `python app.py`; open http://127.0.0.1:5000. Set `FLASK_AUTO_RELOAD=0` to disable auto-restart.
6. Run the tests with `pytest -q`.

6.10 Project file structure

Figure 6.6 is a rendering of the repository's folder tree, as listed from the cloned repository (generated folders such as `__pycache__` and the runtime contents of

`storage/` are omitted). The layout separates routes, services, models, utilities, templates and tests, matching the layered architecture in Chapter 5.

```
OpenCV-Based-System-for-Crack-Identification-and-Width-Estimation/
├── models/
│   ├── __init__.py
│   ├── analysis_model.py
│   ├── database.py
│   ├── image_model.py
│   └── measurement_model.py
├── routes/
│   ├── __init__.py
│   ├── analysis_routes.py
│   ├── history_routes.py
│   ├── report_routes.py
│   └── upload_routes.py
├── services/
│   ├── __init__.py
│   ├── calibration_service.py
│   ├── classification_service.py
│   ├── csv_service.py
│   ├── detection_service.py
│   ├── image_service.py
│   ├── measurement_service.py
│   ├── pdf_service.py
│   ├── preprocessing_service.py
│   ├── storage_service.py
│   └── zip_service.py
├── static/
│   ├── app.css
│   └── app.js
├── storage/
│   └── .gitkeep
├── templates/
│   ├── analysis_not_found.html
│   ├── base.html
│   ├── history.html
│   ├── reports.html
│   ├── results.html
│   └── upload.html
├── tests/
│   ├── test_app.py
│   └── test_processing.py
├── utils/
│   ├── __init__.py
│   ├── file_utils.py
│   └── validators.py
├── .env.example
├── .gitignore
├── app.py
├── config.py
├── pytest.ini
├── README.md
└── requirements.txt
```

Figure 6.6: File and folder structure of the project

Folder / file	Contents	Lines of code
models/	database.py (31), analysis_model.py (56), image_model.py (28), measurement_model.py (43)	158
routes/	upload_routes.py (169), analysis_routes.py (98), history_routes.py (29), report_routes.py (48)	344
services/	preprocessing (33), detection (130), measurement (164), classification (16), calibration (10), image (20), storage (15), csv (57), pdf (291), zip (50)	786
utils/	validators.py (22), file_utils.py (15)	37
templates/	base, upload, results, history, reports, analysis_not_found (HTML)	59
static/	app.js (113), app.css (32)	145
tests/	test_app.py (67), test_processing.py (193)	260
top level	app.py (54), config.py (48), requirements.txt (10), pytest.ini (3), .env.example (5), README.md (60)	180

Table 6.4: Size of each part of the repository (lines, from `wc -l`)

6.11 Source code screenshots with explanation

This section presents the important source files as screenshots, in dependency order, each followed by a short explanation. Screenshots were produced from the repository files with syntax highlighting and line numbers; the numbers shown are the original line numbers. Files not shown here (the PDF service, templates, style sheet and the two test files) are reproduced in Appendix F or discussed in Chapter 7.

app.py — Application factory and entry point

`create_app` loads the configuration, creates the `storage` folders, initialises the database and registers the four blueprints. It also defines the home redirect to `/upload`, a JSON handler for HTTP 413 (upload too large), an empty favicon response and a 404 handler that returns JSON when the client asks for it. The module creates a global `app` object, and `python app.py` runs the development server on 127.0.0.1:5000 with the reloader controlled by `FLASK_AUTO_RELOAD`.

```
1  from pathlib import Path
2
3  from flask import Flask, jsonify, redirect, request
4
5  from config import Config
6  from models.database import init_db
7  from routes.analysis_routes import analysis_bp
8  from routes.history_routes import history_bp
9  from routes.report_routes import report_bp
10 from routes.upload_routes import upload_bp
11
12
13 def create_app(config_class=Config):
14     app = Flask(__name__)
15     app.config.from_object(config_class)
16     Path(app.config["STORAGE_DIR"]).mkdir(parents=True, exist_ok=True)
17     for name in ("original", "processed", "annotated", "reports"):
18         (Path(app.config["STORAGE_DIR"]) / name).mkdir(exist_ok=True)
19     init_db(app)
20     app.register_blueprint(upload_bp)
21     app.register_blueprint(analysis_bp)
22     app.register_blueprint(history_bp)
23     app.register_blueprint(report_bp)
24
25     @app.get("/")
26     def upload_home():
27         return redirect("/upload")
28
29     @app.errorhandler(413)
30     def too_large(_error):
31         return jsonify({"ok": False, "error": "The upload exceeds the configured size limit."}), 413
32
33     @app.get("/favicon.ico")
34     def favicon():
```

Figure 6.7: app.py, lines 1 to 34

```
35         return "", 204
36
37     @app.errorhandler(404)
38     def not_found(_error):
39         if request.accept_mimetypes.best == "application/json":
40             return jsonify({"ok": False, "error": "Resource not found."}), 404
41         return "Page not found", 404
42
43     return app
44
45
46 app = create_app()
47
48 if __name__ == "__main__":
49     app.run(
50         debug=False,
51         host="127.0.0.1",
52         port=5000,
53         use_reloader=app.config["AUTO_RELOAD"],
54     )
```

Figure 6.8: app.py, lines 35 to 54

config.py — Configuration and processing parameters

All limits (file size, number of files, image dimension, allowed extensions) and every image-processing parameter are collected in the `Config` class and can be overridden through environment variables where it matters (`MONGO_URI`, `MONGO_DB_NAME`, `MAX_CONTENT_LENGTH`, `STORAGE_DIR`, `FLASK_AUTO_RELOAD`). Keeping the thresholds in one dictionary means they can be tuned without touching the algorithms, and the dictionary is stored with each analysis for traceability.

```
1  import os
2  from pathlib import Path
3
4  from dotenv import load_dotenv
5
6  load_dotenv()
7
8  BASE_DIR = Path(__file__).resolve().parent
9
10
11 class Config:
12     SECRET_KEY = os.getenv("FLASK_SECRET_KEY", "development-only-key")
13     MONGO_URI = os.getenv("MONGO_URI", "mongodb://localhost:27017")
14     MONGO_DB_NAME = os.getenv("MONGO_DB_NAME", "crack_detection_db")
15     MAX_CONTENT_LENGTH = int(os.getenv("MAX_CONTENT_LENGTH", str(16 * 1024 * 1024)))
16     STORAGE_DIR = Path(os.getenv("STORAGE_DIR", str(BASE_DIR / "storage")))
17     ALLOWED_EXTENSIONS = {"jpg", "jpeg", "jfif", "png"}
18     MAX_FILES_PER_ANALYSIS = 20
19     MAX_IMAGE_DIMENSION = 8000
20     AUTO_RELOAD = os.getenv("FLASK_AUTO_RELOAD", "1").strip().lower() in {"1", "true", "yes", "on"}
21     DEFAULT_PIXELS_PER_MM = 10.0
22     PROCESSING = {
23         "resize_max_dimension": 1600,
24         "gaussian_kernel": 5,
25         "clahe_clip_limit": 2.0,
26         "clahe_tile_grid": 8,
27         "blackhat_kernel": 21,
28         "adaptive_threshold_block_size": 41,
29         "adaptive_threshold_offset": 16,
30         "morphology_kernel": 3,
31         "morphology_iterations": 1,
32         "minimum_region_area": 30,
33         "minimum_crack_aspect_ratio": 3.0,
34         "crack_link_distance_px": 18.0,
```

Figure 6.9: config.py, lines 1 to 34

```
35         "maximum_crack_components": 60,
36         "maximum_crack_coverage": 0.08,
37         "maximum_dispersed_components": 8,
38         "minimum_crack_path_alignment": 3.0,
39         "minimum_dominant_crack_area_ratio": 0.003,
40         "minimum_dominant_crack_aspect_ratio": 2.4,
41         "minimum_fragmented_crack_alignment": 3.0,
42         "maximum_fragmented_crack_coverage": 0.15,
43         "severity_thresholds": {
44             "good_max_width_px": 3.0,
45             "medium_max_width_px": 8.0,
46             "heavy_min_area_px": 5000,
47         },
48     }
```

Figure 6.10: config.py, lines 35 to 48

services/image_service.py — Image loading and writing

`load_and_resize` reads the image as colour, raises a clear error if it cannot be decoded, and shrinks it with area interpolation when its longest side exceeds 1600 px. `write_image` creates the target folder if needed and raises an error if OpenCV cannot write the file.

```python
from pathlib import Path

import cv2


def load_and_resize(path, max_dimension=1600):
    image = cv2.imread(str(path), cv2.IMREAD_COLOR)
    if image is None:
        raise ValueError("Unable to decode the image.")
    height, width = image.shape[:2]
    scale = min(1.0, max_dimension / max(height, width))
    if scale < 1:
        image = cv2.resize(image, (round(width * scale), round(height * scale)), interpolation=cv2.INTER_AREA)
    return image, scale


def write_image(path, image):
    Path(path).parent.mkdir(parents=True, exist_ok=True)
    if not cv2.imwrite(str(path), image):
        raise OSError(f"Unable to write image: {path}")
```

Figure 6.11: services/image_service.py, lines 1 to 20

services/preprocessing_service.py — Pre-processing stage (Algorithm 1)

The function converts to grayscale, blurs, applies CLAHE, computes a BlackHat image and produces the binary mask by adaptive Gaussian thresholding on the grayscale image. Two alternative thresholding branches (percentile and Otsu on the BlackHat image) remain available through configuration. All five images are returned in a dictionary so that the caller can save each one.

```python
import cv2
import numpy as np


def preprocess(image, config):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    kernel_size = int(config["gaussian_kernel"])
    blurred = cv2.GaussianBlur(gray, (kernel_size, kernel_size), 0)
    clahe = cv2.createCLAHE(
        clipLimit=float(config["clahe_clip_limit"]),
        tileGridSize=(int(config["clahe_tile_grid"]), int(config["clahe_tile_grid"])),
    )
    enhanced = clahe.apply(blurred)
    k = int(config["blackhat_kernel"])
    blackhat = cv2.morphologyEx(enhanced, cv2.MORPH_BLACKHAT, cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (k, k)))
    adaptive_block = config.get("adaptive_threshold_block_size")
    if adaptive_block:
        binary = cv2.adaptiveThreshold(
            gray,
            255,
            cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
            cv2.THRESH_BINARY_INV,
            int(adaptive_block),
            float(config.get("adaptive_threshold_offset", 8)),
        )
    else:
        percentile = config.get("blackhat_threshold_percentile")
        if percentile is None:
            _, binary = cv2.threshold(blackhat, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
        else:
            threshold = float(np.percentile(blackhat, float(percentile)))
            binary = cv2.inRange(blackhat, threshold, 255)
    return {"gray": gray, "blurred": blurred, "clahe": enhanced, "blackhat": blackhat, "binary": binary}
```

Figure 6.12: services/preprocessing_service.py, lines 1 to 33

services/detection_service.py — Detection, filtering and validation (Algorithms 2 to 4)

`detect_components` cleans the mask and extracts connected components. `filter_crack_components` keeps elongated components and then `_select_coherent_path` groups nearby components and discards isolated texture blobs while preserving several separate crack networks. `validate_crack_image` decides whether the remaining components look like a crack image at all, using component count, coverage, alignment and a second rule for a single dominant elongated component.

```python
import cv2
import numpy as np


def detect_components(binary, config):
    k = int(config["morphology_kernel"])
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (k, k))
    iterations = int(config["morphology_iterations"])
    cleaned = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel, iterations=iterations)
    cleaned = cv2.morphologyEx(cleaned, cv2.MORPH_CLOSE, kernel, iterations=iterations)
    count, labels, stats, _ = cv2.connectedComponentsWithStats(cleaned, 8)
    minimum = int(config["minimum_region_area"])
    mask = np.zeros_like(cleaned)
    components = []
    for label in range(1, count):
        area = int(stats[label, cv2.CC_STAT_AREA])
        if area < minimum:
            continue
        component_mask = np.where(labels == label, 255, 0).astype(np.uint8)
        contours, _ = cv2.findContours(component_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        components.append({"label": label, "area": area, "mask": component_mask, "contour": max(contours, key=cv2.contourArea)})
        mask[labels == label] = 255
    return cleaned, mask, components


def filter_crack_components(components, config):
    """Keep thin, elongated regions and reject generic blobs from ordinary photos."""
    minimum_aspect = float(config.get("minimum_crack_aspect_ratio", 3.0))
    crack_components = []
    for component in components:
        _, (width, height), _ = cv2.minAreaRect(component["contour"])
        short_side = min(width, height)
        aspect_ratio = max(width, height) / max(short_side, 1.0)
        points = np.argwhere(component["mask"] > 0)
```

Figure 6.13: services/detection_service.py, lines 1 to 34

```python
        if len(points) < 2:
            continue
        _, singular_values, _ = np.linalg.svd(points - points.mean(axis=0), full_matrices=False)
        principal_ratio = float(singular_values[0] / max(singular_values[1], 1e-6))
        if aspect_ratio >= minimum_aspect or principal_ratio >= minimum_aspect:
            crack_components.append(component)
    return _select_coherent_path(crack_components, config)


def _select_coherent_path(components, config):
    """Discard isolated texture blobs while preserving separate crack networks."""
    if len(components) <= 1:
        return components
    link_distance = float(config.get("crack_link_distance_px", 18.0))
    groups = []
    remaining = set(range(len(components)))
    while remaining:
        group = {remaining.pop()}
        changed = True
        while changed:
            changed = False
            for index in tuple(remaining):
                if any(_component_gap(components[index], components[member]) <= link_distance for member in group):
                    group.add(index)
                    remaining.remove(index)
                    changed = True
        groups.append(group)

    group_areas = [
        sum(components[index]["area"] for index in group)
        for group in groups
    ]
    largest_area = max(group_areas)
    minimum_group_area = float(config.get("minimum_crack_group_area", 80.0))
```

Figure 6.14: services/detection_service.py, lines 35 to 68

```
    retained_groups = [
        group for group, area in zip(groups, group_areas)
        if area >= minimum_group_area and (
            area >= largest_area * float(config.get("minimum_crack_group_ratio", 0.08))
            or (len(group) >= 2 and area >= minimum_group_area * 2)
        )
    ]
    if not retained_groups:
        retained_groups = [groups[group_areas.index(largest_area)]]
    return [
        components[index]
        for group in retained_groups
        for index in sorted(group)
    ]


def _component_gap(first, second):
    first_x, first_y, first_width, first_height = cv2.boundingRect(first["contour"])
    second_x, second_y, second_width, second_height = cv2.boundingRect(second["contour"])
    horizontal_gap = max(first_x - (second_x + second_width), second_x - (first_x + first_width), 0)
    vertical_gap = max(first_y - (second_y + second_height), second_y - (first_y + first_height), 0)
    return float(np.hypot(horizontal_gap, vertical_gap))


def validate_crack_image(components, image_shape, config):
    """Reject unrelated images while allowing fragmented, dominant crack paths."""
    if not components:
        return False
    max_components = int(config.get("maximum_crack_components", 60))
    max_coverage = float(config.get("maximum_crack_coverage", 0.08))
    image_area = max(1, image_shape[0] * image_shape[1])
    coverage = sum(component["area"] for component in components) / image_area
    alignment_ratio = _component_alignment_ratio(components)
    dispersed_limit = int(config.get("maximum_dispersed_components", 8))
```

Figure 6.15: services/detection_service.py, lines 69 to 102

```
    if len(components) <= max_components and coverage <= max_coverage and (
        len(components) <= dispersed_limit
        or alignment_ratio >= float(config.get("minimum_crack_path_alignment", 3.0))
    ):
        return True

    dominant = max(components, key=lambda component: component["area"])
    _, (width, height), _ = cv2.minAreaRect(dominant["contour"])
    short_side = min(width, height)
    dominant_aspect = max(width, height) / max(short_side, 1.0)
    dominant_area_ratio = dominant["area"] / image_area
    return (
        dominant_area_ratio >= float(config.get("minimum_dominant_crack_area_ratio", 0.002))
        and dominant_aspect >= float(config.get("minimum_dominant_crack_aspect_ratio", 4.5))
        and coverage <= float(config.get("maximum_fragmented_crack_coverage", 0.15))
    )


def _component_alignment_ratio(components):
    centers = []
    for component in components:
        ys, xs = np.where(component["mask"] > 0)
        if len(xs):
            centers.append((float(xs.mean()), float(ys.mean())))
    if len(centers) <= 3:
        return float("inf")
    _, values, _ = np.linalg.svd(np.asarray(centers) - np.mean(centers, axis=0), full_matrices=False)
    return float(values[0] / max(values[1], 1e-6))
```

Figure 6.16: services/detection_service.py, lines 103 to 130

services/measurement_service.py — Measurement and annotation (Algorithm 5)

`measure_component` skeletonises the component, computes the distance transform, derives robust width statistics and the skeleton length, builds the diagnostic images (distance transform with colour map, and width samples over the skeleton) and returns a measurement dictionary. `annotate` draws the contours, the labels C1, C2, ... and the width sample lines on a copy of the input image.

```python
import cv2
import numpy as np
from skimage.morphology import skeletonize

from .calibration_service import convert_pixels
from .classification_service import classify


def _skeleton_length(skeleton):
    points = np.argwhere(skeleton)
    point_set = set(map(tuple, points))
    length = 0.0
    for y, x in points:
        for dy, dx, distance in ((0, 1, 1.0), (1, 0, 1.0), (1, 1, 2 ** 0.5), (1, -1, 2 ** 0.5)):
            if (y + dy, x + dx) in point_set:
                length += distance
    return length


def _robust_widths(distance, skeleton_points):
    """Return centerline widths while ignoring junction and mask-edge outliers."""
    if not len(skeleton_points):
        return np.array([], dtype=float)

    points = []
    for y, x in skeleton_points:
        neighbors = skeleton_points[
            np.sum((skeleton_points - np.array([y, x])) ** 2, axis=1) <= 2
        ]
        if len(neighbors) <= 3:
            points.append((y, x))

    sample_points = points or [tuple(point) for point in skeleton_points]
    return np.array([distance[y, x] * 2 for y, x in sample_points], dtype=float)
```

Figure 6.17: services/measurement_service.py, lines 1 to 34

```python
def measure_component(component, pixels_per_mm=None, thresholds=None):
    mask = component["mask"] > 0
    skeleton = skeletonize(mask)
    distance = cv2.distanceTransform(component["mask"], cv2.DIST_L2, 5)
    skeleton_points = np.argwhere(skeleton)
    sample_points = [
        tuple(point)
        for point in skeleton_points
        if skeleton[max(0, point[0] - 1):point[0] + 2, max(0, point[1] - 1):point[1] + 2].sum() - 1 == 2
    ] or [tuple(point) for point in skeleton_points]
    widths = _robust_widths(distance, skeleton_points)
    if widths.size:
        # Median and P95 are stable when a crack joins another region or ends
        # in a widened segmentation blob.
        mean_width = float(np.median(widths))
        max_width = float(np.percentile(widths, 95))
    else:
        mean_width = max_width = 0.0
    length = _skeleton_length(skeleton)
    step = max(1, len(sample_points) // 20)
    points = []
    for y, x in sample_points[::step]:
        nearby = skeleton_points[np.sum((skeleton_points - np.array([y, x])) ** 2, axis=1).argsort()[:9]]
        if len(nearby) > 2:
            _, _, vectors = np.linalg.svd(nearby - nearby.mean(axis=0), full_matrices=False)
            tangent_y, tangent_x = vectors[0]
            angle = float(np.arctan2(tangent_y, tangent_x) + np.pi / 2)
        else:
            angle = 0.0
        points.append({
            "x": int(x), "y": int(y),
            "width_px": round(float(distance[y, x] * 2), 3),
```

Figure 6.18: services/measurement_service.py, lines 35 to 68

```
            "width_mm": convert_pixels(float(distance[y, x] * 2), pixels_per_mm),
            "angle": angle,
        })
    distance_visual = cv2.normalize(distance, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
    distance_visual = cv2.applyColorMap(distance_visual, cv2.COLORMAP_TURBO)
    cv2.putText(
        distance_visual,
        "Distance transform: radius px; width = 2 x radius",
        (12, 28),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.55,
        (255, 255, 255),
        2,
        cv2.LINE_AA,
    )
    cv2.putText(
        distance_visual,
        "Mean width: %.2f px / %s mm" % (
            mean_width,
            "%.2f" % (mean_width / pixels_per_mm) if pixels_per_mm else "Not calibrated",
        ),
        (12, 52),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.55,
        (255, 255, 255),
        2,
        cv2.LINE_AA,
    )
    width_visual = cv2.cvtColor((mask > 0).astype(np.uint8) * 35, cv2.COLOR_GRAY2BGR)
    width_visual[skeleton] = (0, 0, 255)
    for point in points:
        px, py = point["x"], point["y"]
        radius = max(2, int(round(point["width_px"] / 2)))
        dx = int(round(np.cos(point["angle"]) * radius))
```

Figure 6.19: services/measurement_service.py, lines 69 to 102

```
        dy = int(round(np.sin(point["angle"]) * radius))
        cv2.line(width_visual, (px - dx, py - dy), (px + dx, py + dy), (0, 255, 0), 1)
    cv2.putText(
        width_visual,
        "Width estimation: green samples / red skeleton",
        (12, 28),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.55,
        (255, 255, 255),
        2,
        cv2.LINE_AA,
    )
    cv2.putText(
        width_visual,
        "Mean: %.2f px / %s mm | Max: %.2f px / %s mm" % (
            mean_width,
            "%.2f" % (mean_width / pixels_per_mm) if pixels_per_mm else "Not calibrated",
            max_width,
            "%.2f" % (max_width / pixels_per_mm) if pixels_per_mm else "Not calibrated",
        ),
        (12, 52),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.55,
        (255, 255, 255),
        2,
        cv2.LINE_AA,
    )
    return {
        "contour_area_px": float(component["area"]),
        "contour_area_mm2": convert_pixels(float(component["area"]), pixels_per_mm ** 2 if pixels_per_mm else None),
        "skeleton_length_px": round(length, 3),
        "skeleton_length_mm": convert_pixels(length, pixels_per_mm),
        "mean_width_px": round(mean_width, 3),
        "max_width_px": round(max_width, 3),
```

Figure 6.20: services/measurement_service.py, lines 103 to 136

```
        "mean_width_mm": convert_pixels(mean_width, pixels_per_mm),
        "max_width_mm": convert_pixels(max_width, pixels_per_mm),
        "calibration_status": "calibrated" if pixels_per_mm else "pixel-only",
        "severity": classify(component["area"], mean_width, thresholds),
        "measurement_points": points,
        "width_sample_count": len(sample_points),
        "skeleton": (skeleton * 255).astype(np.uint8),
        "distance_transform": distance_visual,
        "width_estimation": width_visual,
    }


def annotate(image, components, measurements):
    output = image.copy()
    for index, component in enumerate(components, 1):
        cv2.drawContours(output, [component["contour"]], -1, (0, 0, 255), 2)
        ys, xs = np.where(component["mask"] > 0)
        if len(xs):
            x, y = int(xs.mean()), int(ys.mean())
            cv2.putText(output, f"C{index}", (x, y), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 0, 0), 2)
        for point in measurements[index - 1].get("measurement_points", []):
            px, py = point["x"], point["y"]
            radius = max(2, int(round(point["width_px"] / 2)))
            dx = int(round(np.cos(point.get("angle", 0)) * radius))
            dy = int(round(np.sin(point.get("angle", 0)) * radius))
            cv2.line(output, (px - dx, py - dy), (px + dx, py + dy), (0, 255, 255), 1)
            cv2.circle(output, (px, py), 2, (0, 255, 255), -1)
    return output
```

Figure 6.21: services/measurement_service.py, lines 137 to 164

services/classification_service.py — Severity classification (Algorithm 6)

The rule uses mean width and area thresholds. `threshold_description` returns the same limits as readable text for use in reports.

```python
def classify(area_px, mean_width_px, thresholds=None):
    thresholds = thresholds or {"good_max_width_px": 3.0, "medium_max_width_px": 8.0, "heavy_min_area_px": 5000}
    if area_px >= thresholds["heavy_min_area_px"] or mean_width_px > thresholds["medium_max_width_px"]:
        return "Heavy"
    if mean_width_px > thresholds["good_max_width_px"]:
        return "Medium"
    return "Good"


def threshold_description(thresholds=None):
    thresholds = thresholds or {"good_max_width_px": 3.0, "medium_max_width_px": 8.0, "heavy_min_area_px": 5000}
    return {
        "Good": f"mean width <= {thresholds['good_max_width_px']:.2f} px",
        "Medium": f"mean width > {thresholds['good_max_width_px']:.2f} px and <= {thresholds['medium_max_width_px']:.2f} px",
        "Heavy": f"mean width > {thresholds['medium_max_width_px']:.2f} px or area >= {thresholds['heavy_min_area_px']:.0f} px²",
    }
```

Figure 6.22: services/classification_service.py, lines 1 to 16

services/calibration_service.py — Calibration helpers

`validate_pixels_per_mm` accepts only a positive number and returns None otherwise; `convert_pixels` divides a pixel value by the scale, or returns None when no scale is available.

```python
def validate_pixels_per_mm(value):
    try:
        result = float(value)
    except (TypeError, ValueError):
        return None
    return result if result > 0 else None


def convert_pixels(value, pixels_per_mm):
    return value / pixels_per_mm if pixels_per_mm else None
```

Figure 6.23: services/calibration_service.py, lines 1 to 10

services/storage_service.py — Storage helpers

Every analysis has its own folder `storage/analyses/<analysis_id>`. These helpers create it on demand and delete it with all its contents when an analysis is removed.

```python
import shutil
from pathlib import Path


def analysis_dir(storage_dir, analysis_id):
    root = Path(storage_dir)
    path = root / "analyses" / analysis_id
    path.mkdir(parents=True, exist_ok=True)
    return path


def remove_analysis(storage_dir, analysis_id):
    path = Path(storage_dir) / "analyses" / analysis_id
    if path.exists():
        shutil.rmtree(path)
```

Figure 6.24: services/storage_service.py, lines 1 to 15

services/csv_service.py — CSV report

The function builds an analysis-level summary (counts, total and average length and width, minimum and maximum width, severity counts, overall severity and a remark) and repeats it on every row of a table that has one row per crack. Images without any

crack still get a placeholder row. The result is written with pandas using three decimal places.

```python
from pathlib import Path

import pandas as pd
from collections import Counter


def create_csv(analysis, images, measurements, output_path):
    rows = []
    counts = Counter(item.get("severity", "Unknown") for item in measurements)
    total = len(measurements)
    summary = {
        "Total_Detected_Cracks": total,
        "Total_Length_Pixels": sum(item.get("skeleton_length_px", 0) for item in measurements),
        "Total_Length_MM": sum(item.get("skeleton_length_mm", 0) for item in measurements),
        "Average_Width_Pixels": sum(item.get("mean_width_px", 0) for item in measurements) / total if total else 0,
        "Average_Width_MM": sum(item.get("mean_width_mm", 0) for item in measurements) / total if total else 0,
        "Maximum_Width_Pixels": max((item.get("max_width_px", 0) for item in measurements), default=0),
        "Maximum_Width_MM": max((item.get("max_width_mm", 0) for item in measurements), default=0),
        "Minimum_Width_Pixels": min((item.get("mean_width_px", 0) for item in measurements), default=0),
        "Minimum_Width_MM": min((item.get("mean_width_mm", 0) for item in measurements), default=0),
        "GOOD_Count": counts.get("Good", 0), "MEDIUM_Count": counts.get("Medium", 0), "HEAVY_Count": counts.get("Heavy", 0),
        "Overall_Severity": "Heavy" if counts.get("Heavy", 0) else "Medium" if counts.get("Medium", 0) else "Good" if total else "No data",
        "Remark": (
            f"Heavy-severity indications were detected in {counts.get('Heavy', 0)} of {total} crack region(s). "
            "Review the annotated image and arrange a qualified structural inspection."
            if counts.get("Heavy", 0) else
            f"Medium-severity indications were detected in {counts.get('Medium', 0)} of {total} crack region(s). "
            "Monitor the affected area and consider a qualified inspection."
            if counts.get("Medium", 0) else
            f"All {total} detected crack region(s) are in the Good software category. Continue routine monitoring."
            if total else "No crack measurements were produced. Upload a clearer crack image for another analysis."
        ),
    }
    by_image = {}
```

Figure 6.25: services/csv_service.py, lines 1 to 34

```python
    for item in measurements:
        by_image.setdefault(item["image_id"], []).append(item)
    for image in images:
        records = by_image.get(image["image_id"], [])
        if not records:
            records = [{}]
        for index, measurement in enumerate(records, 1):
            rows.append({
                "Analysis_ID": analysis["analysis_id"], "Analysis_Name": analysis.get("analysis_name", ""),
                "Analysis_Date": analysis.get("created_at", ""), "Image_ID": image["image_id"],
                "Image_Name": image["original_filename"], "Crack_ID": measurement.get("crack_id", f"none-{index}"),
                "Image_Width": image.get("image_width"), "Image_Height": image.get("image_height"),
                "Crack_Area_Pixels": measurement.get("contour_area_px"), "Crack_Area_MM2": measurement.get("contour_area_mm2"), "Crack_Length_Pixels": measurement.get("skeleton_length_px"),
                "Crack_Length_MM": measurement.get("skeleton_length_mm"), "Mean_Width_Pixels": measurement.get("mean_width_px"),
                "Max_Width_Pixels": measurement.get("max_width_px"), "Mean_Width_MM": measurement.get("mean_width_mm"),
                "Max_Width_MM": measurement.get("max_width_mm"), "Calibration_Status": measurement.get("calibration_status"),
                "Pixels_Per_MM": analysis.get("pixels_per_mm"), "Severity": measurement.get("severity"),
                "Processing_Status": image.get("processing_status"),
                **summary,
            })
    Path(output_path).parent.mkdir(parents=True, exist_ok=True)
    pd.DataFrame(rows).to_csv(output_path, index=False, float_format="%.3f")
    return output_path
```

Figure 6.26: services/csv_service.py, lines 35 to 57

services/zip_service.py — ZIP package and manifest

`create_manifest` writes a short text description of the package. `create_zip` adds every file under the analysis folder (except the archive itself) under a top-level folder `Crack_Analysis_Results`, and adds the CSV, PDF and manifest under `reports/` without duplicates.

```python
from pathlib import Path
from zipfile import ZIP_DEFLATED, ZipFile
from datetime import datetime, timezone


def create_manifest(analysis, images, measurements, output_path):
    output = Path(output_path)
    output.parent.mkdir(parents=True, exist_ok=True)
    lines = [
        "Crack Detaction complete analysis package",
        f"Analysis ID: {analysis.get('analysis_id', '')}",
        f"Analysis name: {analysis.get('analysis_name', '')}",
        f"Analysis date: {analysis.get('created_at', '')}",
        f"Package generated: {datetime.now(timezone.utc).isoformat()}",
        f"Status: {analysis.get('status', '')}",
        f"Images: {len(images)}",
        f"Detected crack regions: {len(measurements)}",
        "",
        "Package contents:",
        "- Original uploaded images",
        "- All processed image stages and annotated results",
        "- CSV measurement report",
        "- PDF measurement report",
        "- This manifest",
    ]
    output.write_text("\n".join(lines) + "\n", encoding="utf-8")
    return str(output)


def create_zip(analysis_id, analysis_dir, report_paths, output_path):
    root = Path(analysis_dir)
    output = Path(output_path)
    output.parent.mkdir(parents=True, exist_ok=True)
    files = []
```

Figure 6.27: services/zip_service.py, lines 1 to 34

```python
    seen = set()
    for path in root.rglob("*"):
        if path.is_file() and path.resolve() != output.resolve():
            archive_name = Path("Crack_Analysis_Results") / path.relative_to(root)
            files.append((path, archive_name))
            seen.add(archive_name.as_posix())
    for path in report_paths:
        report_path = Path(path)
        archive_name = Path("Crack_Analysis_Results") / "reports" / report_path.name
        if report_path.exists() and archive_name.as_posix() not in seen:
            files.append((report_path, archive_name))
            seen.add(archive_name.as_posix())
    with ZipFile(output, "w", ZIP_DEFLATED) as archive:
        for path, archive_name in files:
            archive.write(path, archive_name)
    return str(output)
```

Figure 6.28: services/zip_service.py, lines 35 to 50

models/database.py — Database initialisation and fallback

`init_db` prepares the in-memory fallback store first, then tries to connect to MongoDB with a short timeout, pings it and creates the indexes. Any failure sets `mongo_available` to false. `get_db` and `get_fallback_store` give the model modules access to the current handle.

```python
from pymongo import MongoClient


def init_db(app):
    app.extensions["fallback_store"] = {
        "analyses": {}, "images": {}, "measurements": [], "reports": []
    }
    try:
        client = MongoClient(app.config["MONGO_URI"], serverSelectionTimeoutMS=1500)
        db = client[app.config["MONGO_DB_NAME"]]
        db.command("ping")
        app.extensions["mongo_client"] = client
        app.extensions["mongo_db"] = db
        app.extensions["mongo_available"] = True
        db.analyses.create_index("analysis_id", unique=True)
        db.images.create_index([("analysis_id", 1), ("uploaded_at", -1)])
        db.crack_measurements.create_index([("analysis_id", 1), ("image_id", 1)])
        db.report_history.create_index([("analysis_id", 1), ("generated_at", -1)])
    except Exception:
        app.extensions["mongo_available"] = False
        app.extensions["mongo_db"] = None


def get_db():
    from flask import current_app
    return current_app.extensions.get("mongo_db")


def get_fallback_store():
    from flask import current_app
    return current_app.extensions["fallback_store"]
```

Figure 6.29: models/database.py, lines 1 to 31

models/analysis_model.py — Analysis model

These functions insert, read, update, list and delete analysis documents. Every function has two branches: the MongoDB call, and the equivalent operation on the fallback dictionary. Deleting an analysis also deletes its images, measurements and report-history records.

```python
from datetime import datetime, timezone

from .database import get_db, get_fallback_store


def now():
    return datetime.now(timezone.utc)


def create_analysis(document):
    db = get_db()
    if db is None:
        get_fallback_store()["analyses"][document["analysis_id"]] = document.copy()
        return document
    db.analyses.insert_one(document)
    return document


def get_analysis(analysis_id):
    db = get_db()
    if db is not None:
        return db.analyses.find_one({"analysis_id": analysis_id})
    return get_fallback_store()["analyses"].get(analysis_id)


def update_analysis(analysis_id, values):
    db = get_db()
    if db is not None:
        db.analyses.update_one({"analysis_id": analysis_id}, {"$set": values})
    elif analysis_id in get_fallback_store()["analyses"]:
        get_fallback_store()["analyses"][analysis_id].update(values)


def list_analyses(query=None, skip=0, limit=20):
```

Figure 6.30: models/analysis_model.py, lines 1 to 34

```
    db = get_db()
    if db is not None:
        return list(db.analyses.find(query or {}).sort("created_at", -1).skip(skip).limit(limit))
    values = list(get_fallback_store()["analyses"].values())
    if query and "analysis_name" in query:
        expression = query["analysis_name"].get("$regex", "").lower()
        values = [item for item in values if expression in item.get("analysis_name", "").lower()]
    return sorted(values, key=lambda item: item.get("created_at", now()), reverse=True)[skip:skip + limit]


def delete_analysis_records(analysis_id):
    db = get_db()
    if db is not None:
        db.analyses.delete_one({"analysis_id": analysis_id})
        db.images.delete_many({"analysis_id": analysis_id})
        db.crack_measurements.delete_many({"analysis_id": analysis_id})
        db.report_history.delete_many({"analysis_id": analysis_id})
    else:
        store = get_fallback_store()
        store["analyses"].pop(analysis_id, None)
        store["images"].pop(analysis_id, None)
        store["measurements"] = [item for item in store["measurements"] if item["analysis_id"] != analysis_id]
```

Figure 6.31: models/analysis_model.py, lines 35 to 56

models/image_model.py — Image model

Image documents hold the stored file name and path, status, dimensions, resize scale, the paths of all processed images and the per-stage timings.

```
from .database import get_db, get_fallback_store


def create_image(document):
    db = get_db()
    if db is not None:
        db.images.insert_one(document)
    else:
        get_fallback_store()["images"].setdefault(document["analysis_id"], []).append(document.copy())
    return document


def list_images(analysis_id):
    db = get_db()
    if db is not None:
        return list(db.images.find({"analysis_id": analysis_id}).sort("uploaded_at", 1))
    return sorted(get_fallback_store()["images"].get(analysis_id, []), key=lambda item: item.get("uploaded_at"))


def update_image(image_id, values):
    db = get_db()
    if db is not None:
        db.images.update_one({"image_id": image_id}, {"$set": values})
    else:
        for images in get_fallback_store()["images"].values():
            for image in images:
                if image["image_id"] == image_id:
                    image.update(values)
```

Figure 6.32: models/image_model.py, lines 1 to 28

models/measurement_model.py — Measurement model

Crack measurement documents are inserted in bulk and listed sorted by crack id. `update_measurement_calibration` can recompute all millimetre values for a new scale, but no route currently exposes it.

```python
from .database import get_db, get_fallback_store


def create_measurements(documents):
    db = get_db()
    if db is not None and documents:
        db.crack_measurements.insert_many(documents)
    elif documents:
        get_fallback_store()["measurements"].extend(item.copy() for item in documents)


def list_measurements(analysis_id, image_id=None):
    db = get_db()
    if db is None:
        values = [item for item in get_fallback_store()["measurements"] if item["analysis_id"] == analysis_id]
        if image_id:
            values = [item for item in values if item["image_id"] == image_id]
        return sorted(values, key=lambda item: item.get("crack_id", ""))
    query = {"analysis_id": analysis_id}
    if image_id:
        query["image_id"] = image_id
    return list(db.crack_measurements.find(query).sort("crack_id", 1))


def update_measurement_calibration(analysis_id, pixels_per_mm):
    db = get_db()
    measurements = list_measurements(analysis_id)
    for measurement in measurements:
        values = {
            "contour_area_mm2": measurement["contour_area_px"] / (pixels_per_mm * pixels_per_mm),
            "skeleton_length_mm": measurement["skeleton_length_px"] / pixels_per_mm,
            "mean_width_mm": measurement["mean_width_px"] / pixels_per_mm,
            "max_width_mm": measurement["max_width_px"] / pixels_per_mm,
            "calibration_status": "calibrated",
```

Figure 6.33: models/measurement_model.py, lines 1 to 34

```python
            "measurement_points": [
                {**point, "width_mm": point["width_px"] / pixels_per_mm}
                for point in measurement.get("measurement_points", [])
            ],
        }
        if db is not None:
            db.crack_measurements.update_one({"_id": measurement["_id"]}, {"$set": values})
        else:
            measurement.update(values)
```

Figure 6.34: models/measurement_model.py, lines 35 to 43

routes/upload_routes.py — Upload and processing route

`_validate_files` and `_create_uploaded_analysis` check and store the uploads. `_process_analysis` is the heart of the application: for each image it loads, pre-processes, detects, filters, validates, measures, annotates and saves everything, recording the time taken by each stage. If any image has no detectable crack or fails, the whole analysis is deleted and an error message is returned. `/api/upload` and `/api/analyze` currently perform the same work.

```python
import time
import uuid
from pathlib import Path

from flask import Blueprint, current_app, jsonify, render_template, request

from models.analysis_model import create_analysis, delete_analysis_records, get_analysis, now, update_analysis
from models.image_model import create_image, list_images, update_image
from models.measurement_model import create_measurements
from services.detection_service import detect_components, filter_crack_components, validate_crack_image
from services.image_service import load_and_resize, write_image
from services.measurement_service import annotate, measure_component
from services.preprocessing_service import preprocess
from services.storage_service import analysis_dir, remove_analysis
from utils.file_utils import safe_filename
from utils.validators import validate_upload

upload_bp = Blueprint("upload", __name__)


@upload_bp.get("/upload")
def upload_page():
    return render_template("upload.html")


def _validate_files(files):
    if not files:
        return "Select at least one image."
    if len(files) > current_app.config["MAX_FILES_PER_ANALYSIS"]:
        return "Too many images in one analysis."
    for file in files:
        error = validate_upload(file, current_app.config)
        if error:
            return f"{file.filename}: {error}"
```

Figure 6.35: routes/upload_routes.py, lines 1 to 34

```python
    return None


def _create_uploaded_analysis(files):
    analysis_id = uuid.uuid4().hex
    pixels_per_mm = current_app.config.get("DEFAULT_PIXELS_PER_MM", 10.0)
    analysis = {
        "analysis_id": analysis_id,
        "analysis_name": "Crack image analysis",
        "notes": "",
        "created_at": now(), "updated_at": now(), "status": "Uploaded",
        "total_images": len(files), "total_detected_regions": 0,
        "calibration_enabled": bool(pixels_per_mm), "pixels_per_mm": pixels_per_mm,
        "processing_configuration": current_app.config["PROCESSING"],
        "severity_thresholds": current_app.config["PROCESSING"]["severity_thresholds"],
        "error_summary": [],
    }
    create_analysis(analysis)
    root = analysis_dir(current_app.config["STORAGE_DIR"], analysis_id)
    for file in files:
        image_id = uuid.uuid4().hex
        original_path = root / "original_images" / f"{image_id}_{safe_filename(file.filename)}"
        original_path.parent.mkdir(parents=True, exist_ok=True)
        file.save(original_path)
        create_image({
            "image_id": image_id, "analysis_id": analysis_id,
            "original_filename": file.filename, "stored_filename": original_path.name,
            "original_path": str(original_path), "uploaded_at": now(),
            "processing_status": "Uploaded", "workflow_times": {},
        })
    return analysis_id


def _process_analysis(analysis_id):
```

Figure 6.36: routes/upload_routes.py, lines 35 to 68

```python
    analysis = get_analysis(analysis_id)
    if not analysis:
        return None, "Analysis not found."
    update_analysis(analysis_id, {"status": "Processing", "updated_at": now()})
    all_measurements = []
    images_without_cracks = []
    processing_errors = []
    for record in list_images(analysis_id):
        timings = {}
        try:
            started = time.perf_counter()
            image, scale = load_and_resize(record["original_path"], current_app.config["PROCESSING"]["resize_max_dimension"])
            timings["resize"] = round(time.perf_counter() - started, 4)
            started = time.perf_counter()
            stages = preprocess(image, current_app.config["PROCESSING"])
            timings.update(stages.pop("_timings", {}))
            timings["preprocessing"] = round(time.perf_counter() - started, 4)
            started = time.perf_counter()
            cleaned, mask, components = detect_components(stages["binary"], current_app.config["PROCESSING"])
            components = filter_crack_components(components, current_app.config["PROCESSING"])
            timings["detection"] = round(time.perf_counter() - started, 4)
            if not validate_crack_image(components, image.shape, current_app.config["PROCESSING"]):
                images_without_cracks.append(record["original_filename"])
            processed_dir = analysis_dir(current_app.config["STORAGE_DIR"], analysis_id) / "processed_images"
            paths = {}
            for name, stage in {"resized": image, **stages, "cleaned_mask": cleaned, "segmentation_mask": mask}.items():
                path = processed_dir / f"{record['image_id']}_{name}.png"
                write_image(path, stage)
                paths[name] = str(path)
            started = time.perf_counter()
            measurements = []
            for index, component in enumerate(components, 1):
                measurement = measure_component(component, analysis.get("pixels_per_mm"), analysis.get("severity_thresholds"))
                measurement.update({"crack_id": f"{record['image_id'][:8]}-{index}", "analysis_id": analysis_id, "image_id": record["image_id"], "created_at": now()})
```

Figure 6.37: routes/upload_routes.py, lines 69 to 102

```
                all_measurements.append(measurement)
                measurements.append(measurement)
                skeleton_path = processed_dir / f"{record['image_id']}_skeleton_{index}.png"
                write_image(skeleton_path, measurement.pop("skeleton"))
                paths[f"skeleton_{index}"] = str(skeleton_path)
                distance_path = processed_dir / f"{record['image_id']}_distance_transform_{index}.png"
                distance_image = measurement.pop("distance_transform")
                write_image(distance_path, distance_image)
                paths[f"distance_transform_{index}"] = str(distance_path)
                width_path = processed_dir / f"{record['image_id']}_width_estimation_{index}.png"
                width_image = measurement.pop("width_estimation")
                write_image(width_path, width_image)
                paths[f"width_estimation_{index}"] = str(width_path)
            timings["measurement"] = round(time.perf_counter() - started, 4)
            started = time.perf_counter()
            annotated_path = analysis_dir(current_app.config["STORAGE_DIR"], analysis_id) / "annotated_images" / f"{record['image_id']}_annotated.png"
            write_image(annotated_path, annotate(image, components, measurements))
            timings["annotation"] = round(time.perf_counter() - started, 4)
            update_image(record["image_id"], {
                "image_width": int(image.shape[1]), "image_height": int(image.shape[0]),
                "resize_scale": scale, "processed_paths": paths,
                "annotated_path": str(annotated_path), "workflow_times": timings,
                "processing_status": "Completed",
            })
        except Exception as exc:
            processing_errors.append(f"{record['original_filename']}: {exc}")
            update_image(record["image_id"], {"processing_status": "Failed", "error_message": str(exc), "workflow_times": timings})
    if images_without_cracks or processing_errors:
        rejected_files = images_without_cracks + processing_errors
        delete_analysis_records(analysis_id)
        remove_analysis(current_app.config["STORAGE_DIR"], analysis_id)
        if images_without_cracks:
            return None, (
                "Only images containing detectable cracks are allowed. "
```

Figure 6.38: routes/upload_routes.py, lines 103 to 136

```
                f"No crack was detected in: {', '.join(images_without_cracks)}."
            )
        return None, f"Unable to process the uploaded image(s): {'; '.join(rejected_files)}"
    create_measurements(all_measurements)
    update_analysis(analysis_id, {"status": "Completed", "total_detected_regions": len(all_measurements), "updated_at": now()})
    return analysis_id, None


@upload_bp.post("/api/upload")
def upload():
    files = request.files.getlist("images")
    error = _validate_files(files)
    if error:
        return jsonify({"ok": False, "error": error}), 400
    analysis_id = _create_uploaded_analysis(files)
    result, process_error = _process_analysis(analysis_id)
    if process_error:
        return jsonify({"ok": False, "error": process_error}), 422
    return jsonify({"ok": True, "analysis_id": result, "status": "Completed", "images": len(files)})


@upload_bp.post("/api/analyze")
def analyze():
    files = request.files.getlist("images")
    error = _validate_files(files)
    if error:
        return jsonify({"ok": False, "error": error}), 400
    analysis_id = _create_uploaded_analysis(files)
    result, error = _process_analysis(analysis_id)
    if error:
        return jsonify({"ok": False, "error": error}), 422
    return jsonify({"ok": True, "analysis_id": result, "status": "Completed",
                    "images": len(list_images(result)), "regions": len(__import__("models.measurement_model", fromlist=["list_measurements"]).list_measurements(result))})
```

Figure 6.39: routes/upload_routes.py, lines 137 to 169

routes/analysis_routes.py — Results page and file serving

`analysis_page` computes the summary (totals, averages, severity counts and percentages, overall severity and the remark) and renders the template. `analysis_api` returns the same data as JSON. The two file routes serve original, annotated and processed images only if the stored path lies inside the analysis folder.

```python
from pathlib import Path
from collections import Counter

from flask import Blueprint, current_app, jsonify, render_template, request, send_file

from models.analysis_model import get_analysis
from models.image_model import list_images
from models.measurement_model import list_measurements
from utils.file_utils import is_within

analysis_bp = Blueprint("analysis", __name__)


def serialize(value):
    if hasattr(value, "isoformat"):
        return value.isoformat()
    if isinstance(value, dict):
        return {k: serialize(v) for k, v in value.items() if k != "_id"}
    if isinstance(value, list):
        return [serialize(v) for v in value]
    return value


@analysis_bp.get("/analysis/<analysis_id>")
def analysis_page(analysis_id):
    analysis = get_analysis(analysis_id)
    if not analysis:
        return render_template("analysis_not_found.html", analysis_id=analysis_id), 404
    images = list_images(analysis_id)
    measurements = list_measurements(analysis_id)
    counts = Counter(item.get("severity", "Unknown") for item in measurements)
    total = len(measurements)
    summary = {
        "total_cracks": total,
```

Figure 6.40: routes/analysis_routes.py, lines 1 to 34

```python
        "total_length_px": sum(item.get("skeleton_length_px", 0) for item in measurements),
        "total_length_mm": sum(item.get("skeleton_length_mm", 0) for item in measurements),
        "average_width_px": sum(item.get("mean_width_px", 0) for item in measurements) / total if total else 0,
        "average_width_mm": sum(item.get("mean_width_mm", 0) for item in measurements) / total if total else 0,
        "maximum_width_px": max((item.get("max_width_px", 0) for item in measurements), default=0),
        "maximum_width_mm": max((item.get("max_width_mm", 0) for item in measurements), default=0),
        "minimum_width_px": min((item.get("mean_width_px", 0) for item in measurements), default=0),
        "minimum_width_mm": min((item.get("mean_width_mm", 0) for item in measurements), default=0),
        "good_count": counts.get("Good", 0),
        "medium_count": counts.get("Medium", 0),
        "heavy_count": counts.get("Heavy", 0),
        "overall_severity": "Heavy" if counts.get("Heavy", 0) else "Medium" if counts.get("Medium", 0) else "Good" if total else "No data",
    }
    for key in ("good_count", "medium_count", "heavy_count"):
        summary[f"{key}_percentage"] = summary[key] / total * 100 if total else 0
    if not total:
        summary["remark"] = "No crack measurements were produced. Upload a clearer crack image for another analysis."
    elif summary["heavy_count"]:
        summary["remark"] = (
            f"Heavy-severity indications were detected in {summary['heavy_count']} of {total} crack region(s). "
            "Review the annotated image and arrange a qualified structural inspection."
        )
    elif summary["medium_count"]:
        summary["remark"] = (
            f"Medium-severity indications were detected in {summary['medium_count']} of {total} crack region(s). "
            "Monitor the affected area and consider a qualified inspection."
        )
    else:
        summary["remark"] = (
            f"All {total} detected crack region(s) are in the Good software category. "
            "Continue routine monitoring because image-based results are not a structural safety assessment."
        )
    return render_template("results.html", analysis=analysis, images=images, measurements=measurements, summary=summary)
```

Figure 6.41: routes/analysis_routes.py, lines 35 to 68

```python
@analysis_bp.get("/api/analysis/<analysis_id>")
def analysis_api(analysis_id):
    analysis = get_analysis(analysis_id)
    if not analysis:
        return jsonify({"ok": False, "error": "Analysis not found."}), 404
    return jsonify({"ok": True, "analysis": serialize(analysis), "images": serialize(list_images(analysis_id)), "measurements": serialize(list_measurements(analysis_id))})


@analysis_bp.get("/files/<analysis_id>/<image_id>/<file_type>")
def file_route(analysis_id, image_id, file_type):
    allowed = {"original": "original_path", "annotated": "annotated_path"}
    images = list_images(analysis_id)
    image = next((item for item in images if item["image_id"] == image_id), None)
    if not image or file_type not in allowed:
        return "File not found", 404
    path = image.get(allowed[file_type])
    if not path or not is_within(Path(current_app.config["STORAGE_DIR"]) / "analyses" / analysis_id, path):
        return "File not found", 404
    return send_file(path)


@analysis_bp.get("/files/<analysis_id>/<image_id>/processed/<stage>")
def processed_file_route(analysis_id, image_id, stage):
    image = next((item for item in list_images(analysis_id) if item["image_id"] == image_id), None)
    path = image.get("processed_paths", {}).get(stage) if image else None
    root = Path(current_app.config["STORAGE_DIR"]) / "analyses" / analysis_id
    if not path or not is_within(root, path):
        return "File not found", 404
    return send_file(path)
```

Figure 6.42: routes/analysis_routes.py, lines 69 to 98

routes/history_routes.py — History routes

The history page lists up to 100 analyses; the API supports a case-insensitive name search through a MongoDB regular expression; the DELETE route removes records and files.

```
from flask import Blueprint, jsonify, redirect, render_template, request, url_for

from models.analysis_model import delete_analysis_records, list_analyses
from models.database import get_db
from services.storage_service import remove_analysis
from flask import current_app

history_bp = Blueprint("history", __name__)


@history_bp.get("/history")
def history():
    return render_template("history.html", analyses=list_analyses(limit=100))


@history_bp.get("/api/history")
def history_api():
    query = {}
    search = request.args.get("q")
    if search:
        query = {"analysis_name": {"$regex": search, "$options": "i"}}
    return jsonify({"ok": True, "analyses": [{k: v for k, v in item.items() if k != "_id"} for item in list_analyses(query, limit=100)]})


@history_bp.delete("/api/analysis/<analysis_id>")
def delete(analysis_id):
    delete_analysis_records(analysis_id)
    remove_analysis(current_app.config["STORAGE_DIR"], analysis_id)
    return jsonify({"ok": True})
```

Figure 6.43: routes/history_routes.py, lines 1 to 29

routes/report_routes.py — Report routes

`build_report` generates CSV or PDF on demand; for ZIP it first builds both, then the manifest, then the archive. `download_report` rejects unknown report types with HTTP 400 and unknown analyses with 404.

```
import uuid
from pathlib import Path

from flask import Blueprint, current_app, jsonify, render_template, send_file

from models.analysis_model import get_analysis
from models.image_model import list_images
from models.measurement_model import list_measurements
from services.csv_service import create_csv
from services.pdf_service import create_pdf
from services.storage_service import analysis_dir
from services.zip_service import create_manifest, create_zip

report_bp = Blueprint("reports", __name__)


@report_bp.get("/reports")
def reports():
    return render_template("reports.html")


def build_report(analysis_id, report_type):
    analysis = get_analysis(analysis_id)
    if not analysis:
        return None
    images = list_images(analysis_id)
    measurements = list_measurements(analysis_id)
    root = analysis_dir(current_app.config["STORAGE_DIR"], analysis_id)
    report_dir = root / "reports"
    report_dir.mkdir(exist_ok=True)
    if report_type == "csv":
        path = report_dir / f"{analysis_id}.csv"
        return create_csv(analysis, images, measurements, path)
    if report_type == "pdf":
```

Figure 6.44: routes/report_routes.py, lines 1 to 34

```
        path = report_dir / f"{analysis_id}.pdf"
        return create_pdf(analysis, images, measurements, path)
    csv_path = build_report(analysis_id, "csv")
    pdf_path = build_report(analysis_id, "pdf")
    manifest_path = create_manifest(analysis, images, measurements, report_dir / f"{analysis_id}_manifest.txt")
    return create_zip(analysis_id, root, [csv_path, pdf_path, manifest_path], report_dir / f"{analysis_id}.zip")


@report_bp.get("/reports/<analysis_id>/<report_type>")
def download_report(analysis_id, report_type):
    if report_type not in {"csv", "pdf", "zip"}:
        return jsonify({"ok": False, "error": "Unknown report type."}), 400
    path = build_report(analysis_id, report_type)
    return send_file(path, as_attachment=True, download_name=Path(path).name) if path else ("Analysis not found", 404)
```

Figure 6.45: routes/report_routes.py, lines 35 to 48

utils/validators.py — Upload validation

The validator checks the extension against the allowed set, rejects empty files, decodes the image to prove that it is valid and enforces the minimum and maximum dimensions.

```
from pathlib import Path

import cv2


def validate_upload(file, config):
    if not file or not file.filename:
        return "A file was not selected."
    ext = Path(file.filename).suffix.lower().lstrip(".")
    if ext not in config["ALLOWED_EXTENSIONS"]:
        return "Only JPG, JPEG, JFIF, and PNG images are supported."
    data = file.read()
    file.seek(0)
    if not data:
        return "The uploaded file is empty."
    image = cv2.imdecode(__import__("numpy").frombuffer(data, dtype="uint8"), cv2.IMREAD_COLOR)
    if image is None:
        return "The file is not a valid decodable image."
    height, width = image.shape[:2]
    if width < 2 or height < 2 or max(width, height) > config["MAX_IMAGE_DIMENSION"]:
        return "Image dimensions are outside the supported range."
    return None
```

Figure 6.46: utils/validators.py, lines 1 to 22

utils/file_utils.py — File-name and path safety

`safe_filename` replaces unsafe characters and strips leading and trailing dots and underscores. `is_within` resolves both paths and checks containment, which blocks path traversal.

```
import re
from pathlib import Path


def safe_filename(name):
    stem = re.sub(r"[^A-Za-z0-9._-]+", "_", Path(name).name)
    return stem.strip("._") or "image"


def is_within(base, candidate):
    try:
        Path(candidate).resolve().relative_to(Path(base).resolve())
        return True
    except ValueError:
        return False
```

Figure 6.47: utils/file_utils.py, lines 1 to 15

static/app.js — Client-side behaviour

The script previews selected files, lets the user remove one, submits the form asynchronously and shows the result or error message, and implements the image lightbox with zoom controls, wheel zoom and keyboard shortcuts.

```
const input = document.querySelector("#images");
const preview = document.querySelector("#preview");

if (input) input.addEventListener("change", () => {
  preview.innerHTML = "";
  [...input.files].forEach(file => {
    const col = document.createElement("div");
    col.className = "col-6 col-md-3 upload-preview-item";
    const img = document.createElement("img");
    img.className = "img-fluid rounded upload-preview-image";
    img.src = URL.createObjectURL(file);
    img.alt = file.name;
    const name = Object.assign(document.createElement("small"), {textContent: file.name, className: "upload-preview-name"});
    const remove = Object.assign(document.createElement("button"), {
      type: "button",
      className: "upload-remove-button",
      textContent: "Remove image",
      title: `Remove ${file.name}`
    });
    remove.setAttribute("aria-label", `Remove ${file.name}`);
    remove.addEventListener("click", () => {
      const remaining = [...input.files].filter(candidate => candidate !== file);
      const transfer = new DataTransfer();
      remaining.forEach(candidate => transfer.items.add(candidate));
      input.files = transfer.files;
      input.dispatchEvent(new Event("change"));
    });
    col.append(img, name, remove);
    preview.append(col);
  });
});

const form = document.querySelector("#analysis-form");
if (form) form.addEventListener("submit", async event => {
```

Figure 6.48: static/app.js, lines 1 to 34

```
  event.preventDefault();
  const status = document.querySelector("#upload-status");
  const uploadButton = document.querySelector("#upload-button");
  const setStatus = (message, type) => {
    status.textContent = message;
    status.className = `alert alert-${type} mt-3 mb-0`;
  };
  if (!input.files.length) {
    setStatus("Please select at least one crack image before uploading.", "warning");
    return;
  }
  setStatus("Uploading and detecting cracks. Please wait...", "info");
  uploadButton.disabled = true;
  try {
    const response = await fetch("/api/analyze", {method: "POST", body: new FormData(form)});
    const data = await response.json();
    if (!response.ok || !data.ok) throw new Error(data.error || "Image processing failed.");
    window.location = `/analysis/${data.analysis_id}`;
  } catch (error) {
    uploadButton.disabled = false;
    setStatus(error.message, "danger");
  }
});

const lightbox = document.querySelector("#image-lightbox");
const lightboxImage = document.querySelector("#image-lightbox-image");
const lightboxCaption = document.querySelector("#image-lightbox-caption");
const lightboxZoom = document.querySelector("#image-lightbox-zoom");
let imageZoom = 1;
const setImageZoom = value => {
  imageZoom = Math.min(4, Math.max(0.5, value));
  if (lightboxImage) {
    lightboxImage.style.transform = "none";
    lightboxImage.style.maxWidth = imageZoom > 1 ? "none" : "100%";
```

Figure 6.49: static/app.js, lines 35 to 68

```
    lightboxImage.style.maxHeight = imageZoom > 1 ? "none" : "82vh";
    lightboxImage.style.width = imageZoom > 1 ? `${imageZoom * 100}%` : "auto";
  }
  if (lightboxZoom) lightboxZoom.textContent = `${Math.round(imageZoom * 100)}%`;
};
const closeLightbox = () => {
  if (!lightbox) return;
  lightbox.hidden = true;
  lightboxImage.removeAttribute("src");
  setImageZoom(1);
};

document.addEventListener("click", event => {
  const zoomButton = event.target.closest("[data-image-zoom]");
  if (zoomButton && lightbox) {
    const action = zoomButton.dataset.imageZoom;
    setImageZoom(action === "in" ? imageZoom + 0.25 : action === "out" ? imageZoom - 0.25 : 1);
    event.stopPropagation();
    return;
  }
  const trigger = event.target.closest("[data-image-preview]");
  if (trigger && lightbox) {
    lightboxImage.src = trigger.dataset.imagePreview;
    lightboxImage.alt = trigger.dataset.imageAlt || "Crack analysis image";
    lightboxCaption.textContent = trigger.dataset.imageCaption || "";
    setImageZoom(1);
    lightbox.hidden = false;
  } else if (event.target === lightbox || event.target.closest(".image-lightbox-close")) {
    closeLightbox();
  }
});

if (lightbox) lightbox.addEventListener("wheel", event => {
  if (lightbox.hidden) return;
```

Figure 6.50: static/app.js, lines 69 to 102

```
  event.preventDefault();
  setImageZoom(imageZoom + (event.deltaY < 0 ? 0.15 : -0.15));
}, {passive: false});

document.addEventListener("keydown", event => {
  if (event.key === "Escape") closeLightbox();
  if (!lightbox || lightbox.hidden) return;
  if (event.key === "+" || event.key === "=") setImageZoom(imageZoom + 0.25);
  if (event.key === "-" || event.key === "_") setImageZoom(imageZoom - 0.25);
  if (event.key === "0") setImageZoom(1);
});
```

Figure 6.51: static/app.js, lines 103 to 113

requirements.txt — Python dependencies

Version ranges for Flask, python-dotenv, PyMongo, OpenCV (headless), NumPy, scikit-image, pandas, ReportLab, pytest and mongomock.

```
Flask>=3.0,<4
python-dotenv>=1.0
pymongo>=4.6
opencv-python-headless>=4.9
numpy>=1.26
scikit-image>=0.22
pandas>=2.1
reportlab>=4.0
pytest>=8.0
mongomock>=4.1
```

Figure 6.52: requirements.txt, lines 1 to 10

.env.example — Environment template

Sample values for the MongoDB URI and database name, the Flask secret key, the upload limit and the storage folder. The real `.env` file is excluded by `.gitignore`.

```
MONGO_URI=mongodb://localhost:27017
MONGO_DB_NAME=crack_detection_db
FLASK_SECRET_KEY=replace-with-a-random-secret
MAX_CONTENT_LENGTH=16777216
STORAGE_DIR=storage
```

Figure 6.53: .env.example, lines 1 to 5

6.12 Additional user-interface screenshots

The upload page before any file is chosen shows the drop zone, and the history page lists earlier analyses. Further parts of the results page follow.

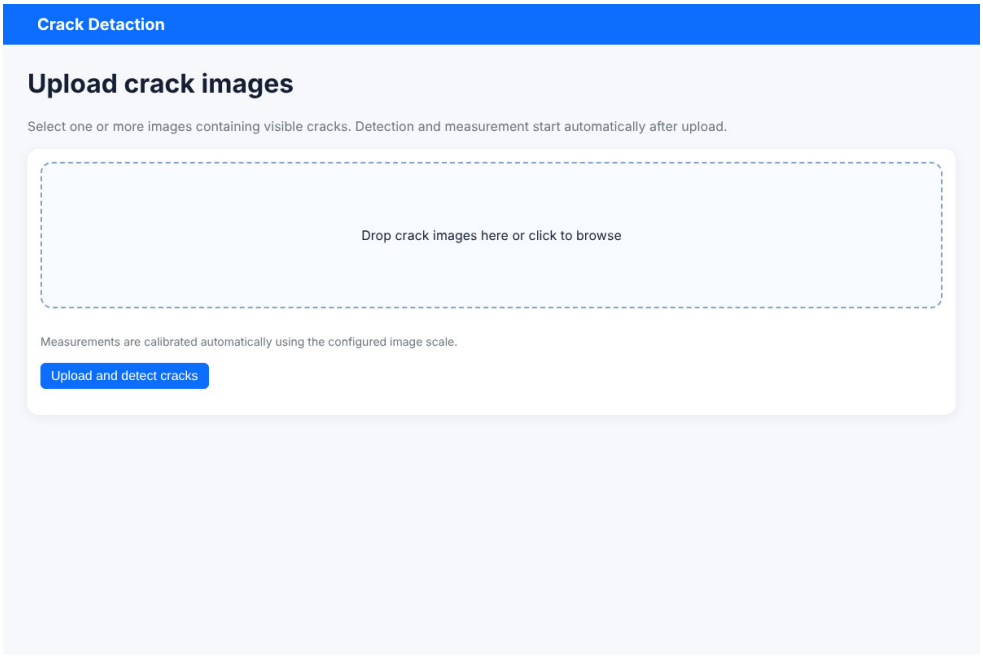


Figure 6.54: Upload page before selecting files

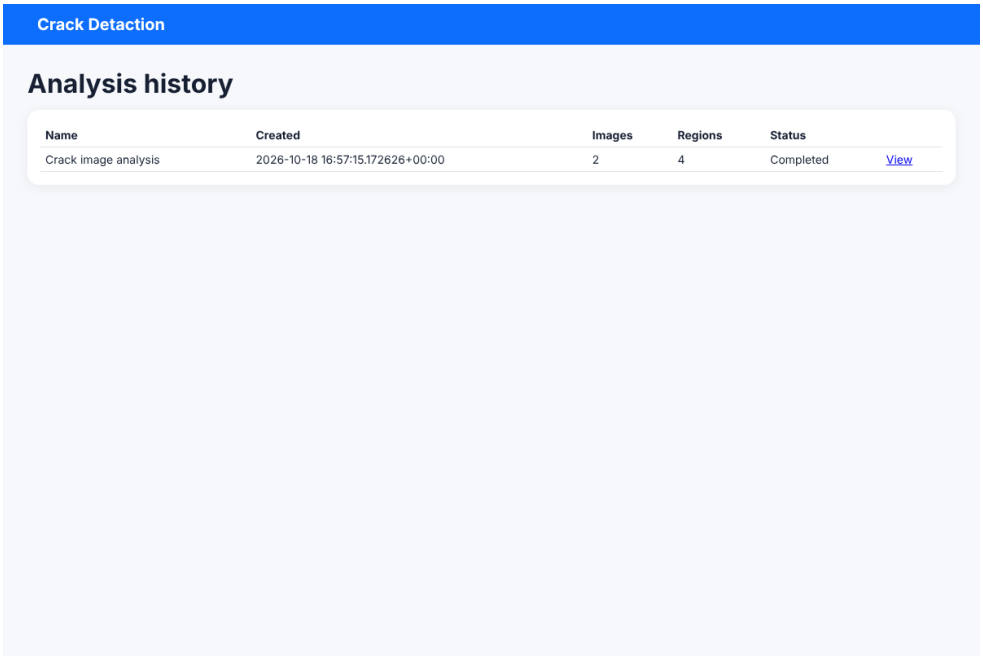


Figure 6.55: History page

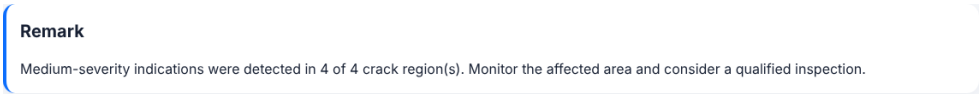


Figure 6.56: Download-report card

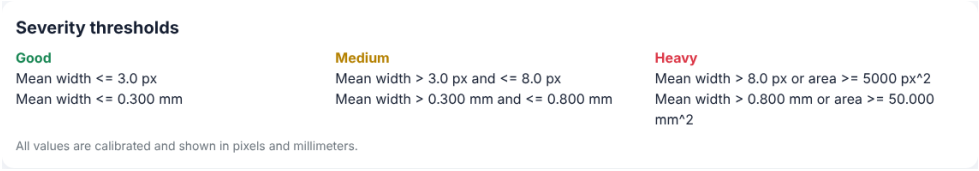


Figure 6.57: Severity thresholds card, shown in pixels and millimetres

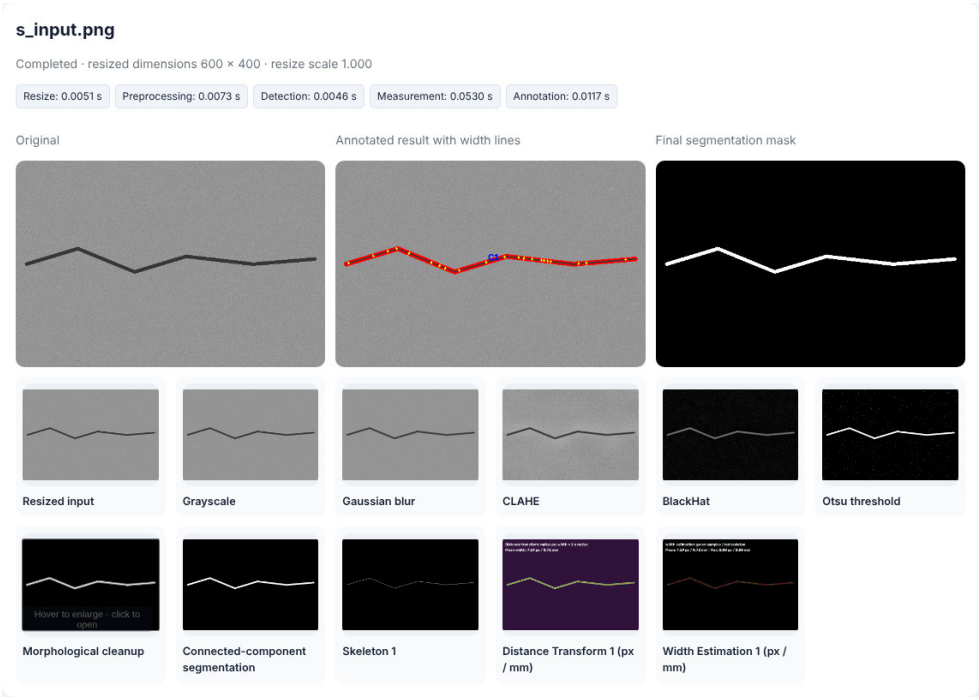
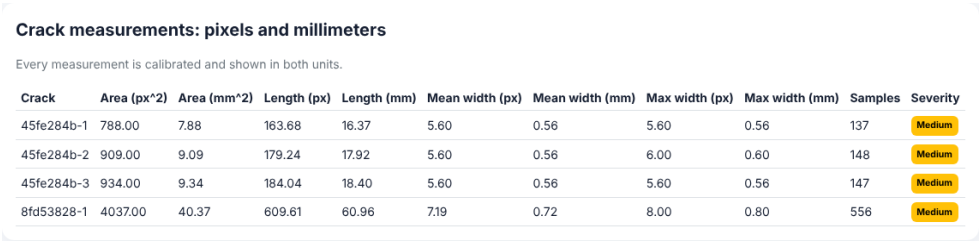


Figure 6.58: Per-image result card for the first synthetic image



Crack	Area (px^2)	Area (mm^2)	Length (px)	Length (mm)	Mean width (px)	Mean width (mm)	Max width (px)	Max width (mm)	Samples	Severity
45fe284b-1	788.00	7.88	163.68	16.37	5.60	0.56	5.60	0.56	137	Medium
45fe284b-2	909.00	9.09	179.24	17.92	5.60	0.56	6.00	0.60	148	Medium
45fe284b-3	934.00	9.34	184.04	18.40	5.60	0.56	5.60	0.56	147	Medium
8fd53828-1	4037.00	40.37	609.61	60.96	7.19	0.72	8.00	0.80	556	Medium

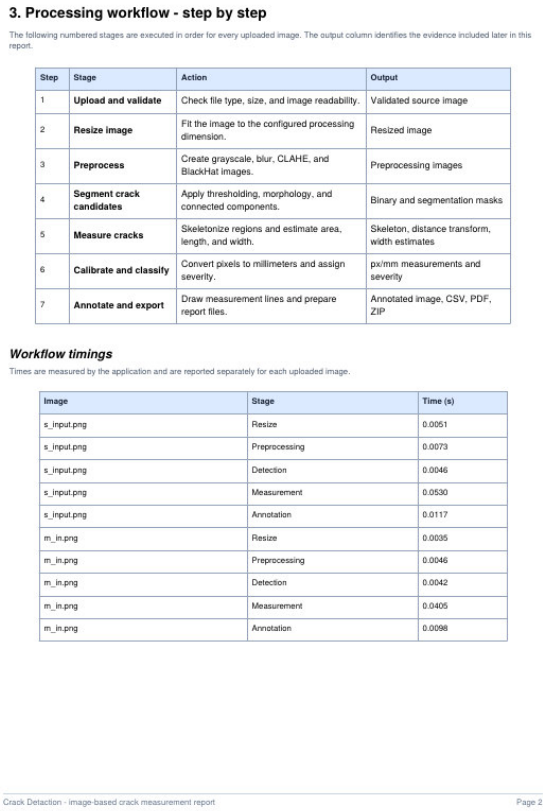
Figure 6.59: Crack measurement table in pixels and millimetres

6.13 Generated output files

The CSV export of the end-to-end run is shown in Figure 6.60 (selected columns; the file has 34 columns). The PDF report has 33 pages for two images because every stage image is included; pages 2 to 4 are shown next. The ZIP archive contained 35 files; its listing follows.

Crack ID	Image Name	Crack Area Pixels	Crack Length Pixels	Crack Length MM	Mean Width Pixels	Max Width Pixels	Mean Width MM	Calibration Status	Severity
8fd53828-1	s_input.png	4037.0	609.605	60.961	7.194	8.0	0.719	calibrated	Medium
45fe284b-1	m_in.png	788.0	163.681	16.368	5.6	5.6	0.56	calibrated	Medium
45fe284b-2	m_in.png	909.0	179.238	17.924	5.6	6.0	0.56	calibrated	Medium
45fe284b-3	m_in.png	934.0	184.037	18.404	5.6	5.6	0.56	calibrated	Medium

Figure 6.60: CSV report opened as a table (selected columns)



3. Processing workflow - step by step

The following numbered stages are executed in order for every uploaded image. The output column identifies the evidence included later in this report.

Step	Stage	Action	Output
1	**Upload and validate**	Check file type, size, and image readability.	Validated source image
2	**Resize image**	Fit the image to the configured processing dimension.	Resized image
3	**Preprocess**	Create grayscale, blur, CLAHE, and BlackHat images.	Preprocessing images
4	**Segment crack candidates**	Apply thresholding, morphology, and connected components.	Binary and segmentation masks
5	**Measure cracks**	Skeletonize regions and estimate area, length, and width.	Skeleton, distance transform, width estimates
6	**Calibrate and classify**	Convert pixels to millimeters and assign severity.	px/mm measurements and severity
7	**Annotate and export**	Draw measurement lines and prepare report files.	Annotated image, CSV, PDF, ZIP

Workflow timings

Times are measured by the application and are reported separately for each uploaded image.

Image	Stage	Time (s)
s_input.png	Resize	0.0051
s_input.png	Preprocessing	0.0073
s_input.png	Detection	0.0046
s_input.png	Measurement	0.0530
s_input.png	Annotation	0.0117
m_in.png	Resize	0.0035
m_in.png	Preprocessing	0.0046
m_in.png	Detection	0.0042
m_in.png	Measurement	0.0405
m_in.png	Annotation	0.0098

Crack Detection - Image-based crack measurement report Page 2

Figure 6.61: PDF report, page 2



4. Crack measurements

Every value is shown as pixels and millimeters where calibration is available.

Image / crack	Area px^2	Area mm^2	Length px	Length mm	Severity
45fe284b7eb6 / 45fe284b-1	788.00 px^2	7.88 mm^2	163.68 px	16.37 mm	Medium
45fe284b7eb6 / 45fe284b-2	909.00 px^2	9.09 mm^2	179.24 px	17.92 mm	Medium
45fe284b7eb6 / 45fe284b-3	934.00 px^2	9.34 mm^2	184.04 px	18.40 mm	Medium
8fd5382f89656 / 8fd53828-1	4037.00 px^2	40.37 mm^2	609.61 px	60.96 mm	Medium

Crack	Mean width px	Mean width mm	Max width px	Max width mm	Samples
45fe284b-1	5.60 px	0.56 mm	5.60 px	0.56 mm	137
45fe284b-2	5.60 px	0.56 mm	6.00 px	0.60 mm	148
45fe284b-3	5.60 px	0.56 mm	5.60 px	0.56 mm	147
8fd53828-1	7.19 px	0.72 mm	8.00 px	0.80 mm	556

Crack Detaction - image-based crack measurement report Page 3

Figure 6.62: PDF report, page 3

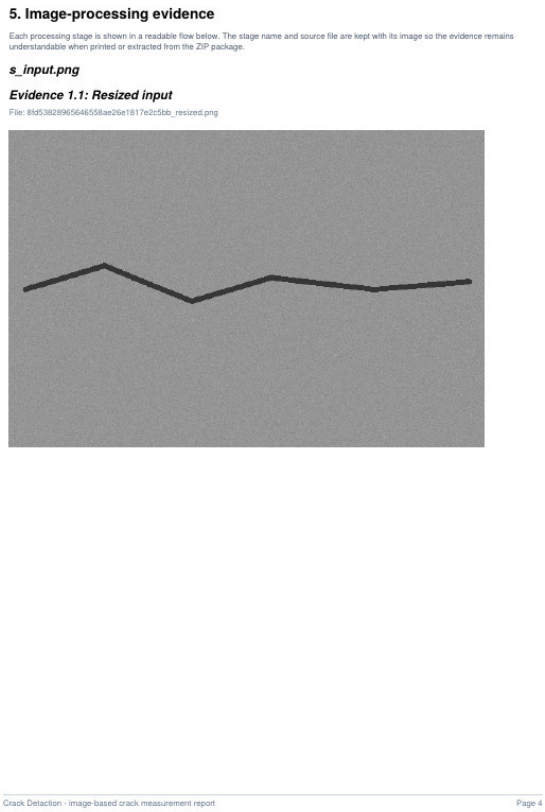


Figure 6.63: PDF report, page 4

Contents of the ZIP archive (size in bytes and path):

```
  329628  CAR/annotated_images/45fe284b7eb642c39bf9a958a00772d1_annotated.png
  451958  CAR/annotated_images/8fd53828965646558ae26e1817e2c5bb_annotated.png
   19521  CAR/processed_images/45fe284b7eb642c39bf9a958a00772d1_distance_transform_2.png
   21448  CAR/processed_images/45fe284b7eb642c39bf9a958a00772d1_width_estimation_2.png
   65787  CAR/processed_images/45fe284b7eb642c39bf9a958a00772d1_clahe.png
   21502  CAR/processed_images/45fe284b7eb642c39bf9a958a00772d1_width_estimation_3.png
    1005  CAR/processed_images/45fe284b7eb642c39bf9a958a00772d1_skeleton_1.png
   87755  CAR/processed_images/8fd53828965646558ae26e1817e2c5bb_clahe.png
    1403  CAR/processed_images/8fd53828965646558ae26e1817e2c5bb_skeleton_1.png
  327929  CAR/processed_images/45fe284b7eb642c39bf9a958a00772d1_resized.png
   24129  CAR/processed_images/8fd53828965646558ae26e1817e2c5bb_distance_transform_1.png
    1069  CAR/processed_images/45fe284b7eb642c39bf9a958a00772d1_skeleton_3.png
    1547  CAR/processed_images/8fd53828965646558ae26e1817e2c5bb_segmentation_mask.png
    3688  CAR/processed_images/8fd53828965646558ae26e1817e2c5bb_binary.png
   72954  CAR/processed_images/8fd53828965646558ae26e1817e2c5bb_blurred.png
  155067  CAR/processed_images/8fd53828965646558ae26e1817e2c5bb_gray.png
  112704  CAR/processed_images/45fe284b7eb642c39bf9a958a00772d1_gray.png
   68831  CAR/processed_images/45fe284b7eb642c39bf9a958a00772d1_blackhat.png
    1547  CAR/processed_images/8fd53828965646558ae26e1817e2c5bb_cleaned_mask.png
   19640  CAR/processed_images/45fe284b7eb642c39bf9a958a00772d1_distance_transform_3.png
   19169  CAR/processed_images/45fe284b7eb642c39bf9a958a00772d1_distance_transform_1.png
  451806  CAR/processed_images/8fd53828965646558ae26e1817e2c5bb_resized.png
   22705  CAR/processed_images/8fd53828965646558ae26e1817e2c5bb_width_estimation_1.png
    3221  CAR/processed_images/45fe284b7eb642c39bf9a958a00772d1_binary.png
    1036  CAR/processed_images/45fe284b7eb642c39bf9a958a00772d1_skeleton_2.png
   92628  CAR/processed_images/8fd53828965646558ae26e1817e2c5bb_blackhat.png
   55124  CAR/processed_images/45fe284b7eb642c39bf9a958a00772d1_blurred.png
    1763  CAR/processed_images/45fe284b7eb642c39bf9a958a00772d1_cleaned_mask.png
   21283  CAR/processed_images/45fe284b7eb642c39bf9a958a00772d1_width_estimation_1.png
    1763  CAR/processed_images/45fe284b7eb642c39bf9a958a00772d1_segmentation_mask.png
    2266  CAR/reports/9530840468384f5f917c3a677f49986b.csv
     441  CAR/reports/9530840468384f5f917c3a677f49986b_manifest.txt
 2017238  CAR/reports/9530840468384f5f917c3a677f49986b.pdf
  451806  CAR/original_images/8fd53828965646558ae26e1817e2c5bb_s_input.png
  327929  CAR/original_images/45fe284b7eb642c39bf9a958a00772d1_m_in.png
```

(`CAR/` abbreviates the top-level folder `Crack_Analysis_Results/`.)

6.14 JSON API response

The JSON returned by `GET /api/analysis/<id>` for a one-image analysis is shown below, shortened for readability (some lists and the processing configuration are abbreviated with "...").


```
{
 "analysis": {
  "analysis_id": "1ae414da1af34a6bb1089de919f3a33b",
  "analysis_name": "Crack image analysis",
  "calibration_enabled": true,
  "created_at": "2026-10-24T04:43:18.402417+00:00",
  "error_summary": [],
  "notes": "",
  "pixels_per_mm": 10.0,
  "processing_configuration": {
   "...": "..."
  },
  "severity_thresholds": {
   "good_max_width_px": 3.0,
   "heavy_min_area_px": 5000,
   "medium_max_width_px": 8.0
  },
  "status": "Completed",
  "total_detected_regions": 3,
  "total_images": 1,
  "updated_at": "2026-10-24T04:43:18.541741+00:00"
 },
 "images": [
  {
   "analysis_id": "1ae414da1af34a6bb1089de919f3a33b",
   "annotated_path":
"/tmp/tmprz6i0e1k/analyses/1ae414da1af34a6bb1089de919f3a33b/annotated_images/eeeaf5f943634f69a4
dd104588d55a05_annotated.png",
   "image_height": 360,
   "image_id": "eeeaf5f943634f69a4dd104588d55a05",
   "image_width": 480,
   "original_filename": "m_in.png",
   "original_path":
"/tmp/tmprz6i0e1k/analyses/1ae414da1af34a6bb1089de919f3a33b/original_images/eeeaf5f943634f69a4d
d104588d55a05_m_in.png",
   "processed_paths": {
    "binary":
"/tmp/tmprz6i0e1k/analyses/1ae414da1af34a6bb1089de919f3a33b/processed_images/eeeaf5f943634f69a4
dd104588d55a05_binary.png",
    "blackhat":
"/tmp/tmprz6i0e1k/analyses/1ae414da1af34a6bb1089de919f3a33b/processed_images/eeeaf5f943634f69a4
dd104588d55a05_blackhat.png",
    "blurred":
"/tmp/tmprz6i0e1k/analyses/1ae414da1af34a6bb1089de919f3a33b/processed_images/eeeaf5f943634f69a4
dd104588d55a05_blurred.png"
   },
   "processing_status": "Completed",
   "resize_scale": 1.0,
   "stored_filename": "eeeaf5f943634f69a4dd104588d55a05_m_in.png",
   "uploaded_at": "2026-10-24T04:43:18.403694+00:00",
   "workflow_times": {
    "annotation": 0.0116,
    "detection": 0.0096,
    "measurement": 0.0823,
    "preprocessing": 0.007,
    "resize": 0.0039
   }
  }
 ],
 "measurements": [
  {
   "analysis_id": "1ae414da1af34a6bb1089de919f3a33b",
   "calibration_status": "calibrated",
   "contour_area_mm2": 7.88,
   "contour_area_px": 788.0,
   "crack_id": "eeeaf5f9-1",
   "created_at": "2026-10-24T04:43:18.489541+00:00",
   "image_id": "eeeaf5f943634f69a4dd104588d55a05",
```

```
   "max_width_mm": 0.5599999904632569,
   "max_width_px": 5.6,
   "mean_width_mm": 0.5599999904632569,
   "mean_width_px": 5.6,
   "measurement_points": [
    {
     "angle": 4.2055331289795035,
     "width_mm": 0.43937997817993163,
     "width_px": 4.394,
     "x": 159,
     "y": 55
    },
    {
     "angle": 4.1685933256376915,
     "width_mm": 0.43937997817993163,
     "width_px": 4.394,
     "x": 152,
     "y": 59
    },
    {
     "...": "..."
    }
   ],
   "severity": "Medium",
   "skeleton_length_mm": 16.368124086713195,
   "skeleton_length_px": 163.681,
   "width_sample_count": 137
  }
 ],
 "ok": true
}
```


6.15 Step-by-step walkthrough of one analysis

This section follows the synthetic image of Figure 6.1 (600 x 400 px, crack raster width 7 px, Gaussian noise of standard deviation 6) through the pipeline, showing the image produced by each stage and what the next stage does with it.

Input image

The input is a mid-grey noisy surface crossed by a dark six-segment crack. It is smaller than 1600 px, so no resizing is applied (scale factor 1.0).

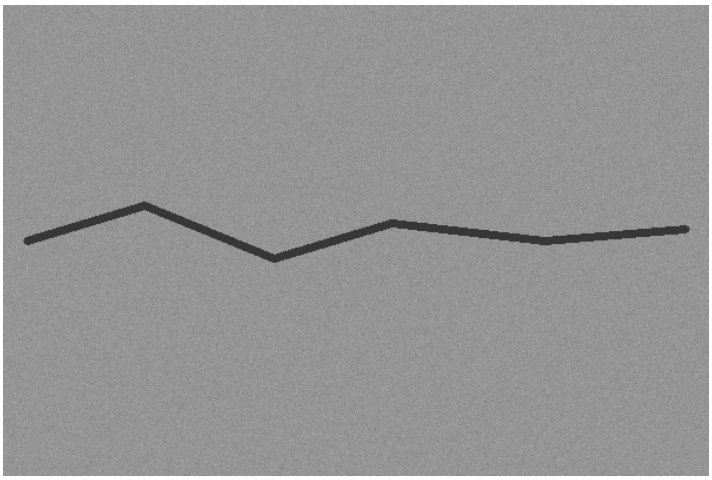


Figure 6.64: Input image

Grayscale

Colour is discarded because a crack is recognised by its darkness, not its hue. All later stages work on this single channel.

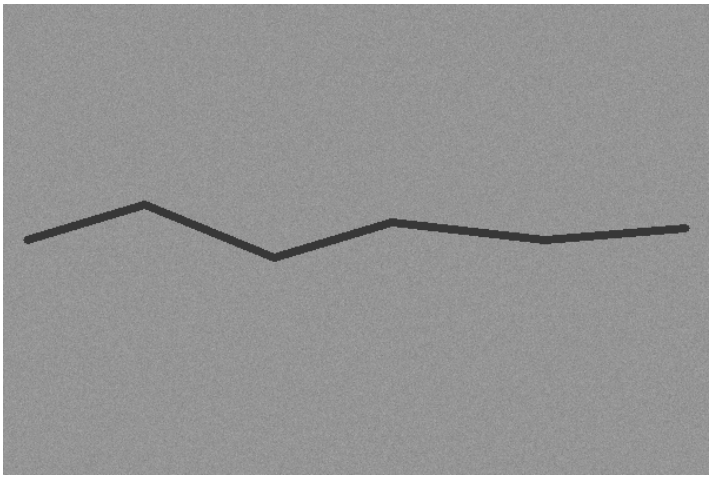


Figure 6.65: Grayscale

Gaussian blur (5 x 5)

The blur averages each pixel with its neighbours and suppresses the grain noise, at the cost of slightly softening the crack edges.

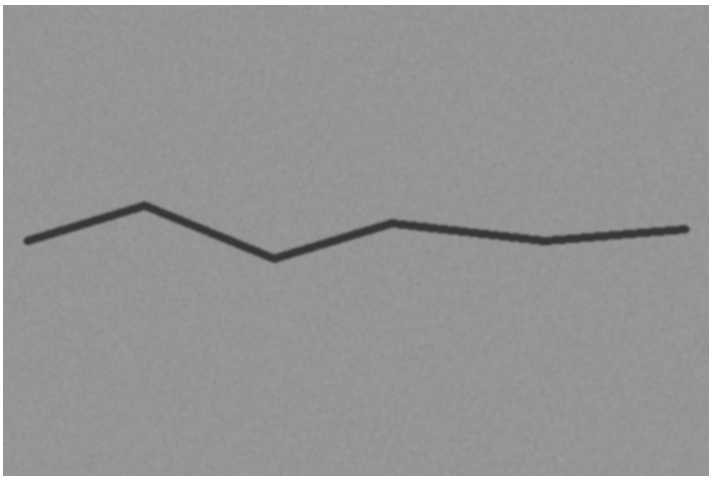


Figure 6.66: Gaussian blur (5 x 5)

CLAHE

Local contrast equalisation in 8 x 8 tiles makes the crack stand out even where lighting varies. It is applied to the blurred image and feeds the BlackHat stage.

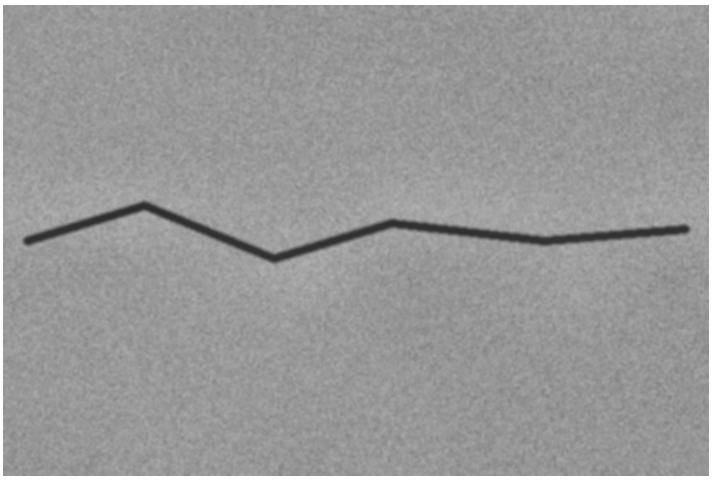


Figure 6.67: CLAHE

BlackHat transform

The difference between the morphological closing and the original image highlights thin dark structures. In this application it is a visual diagnostic; the mask is made from the grayscale image.

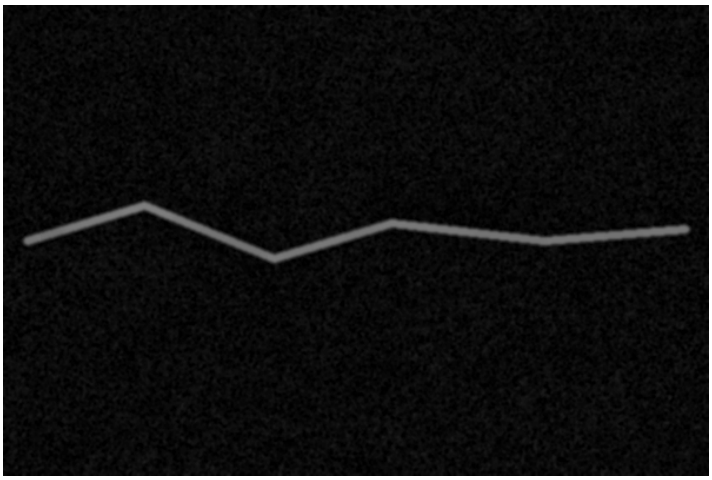


Figure 6.68: BlackHat transform

Adaptive threshold

Each pixel is compared with a Gaussian-weighted local mean (41 x 41 window) minus 16. Pixels darker than that become foreground. The result contains the crack and scattered noise specks.

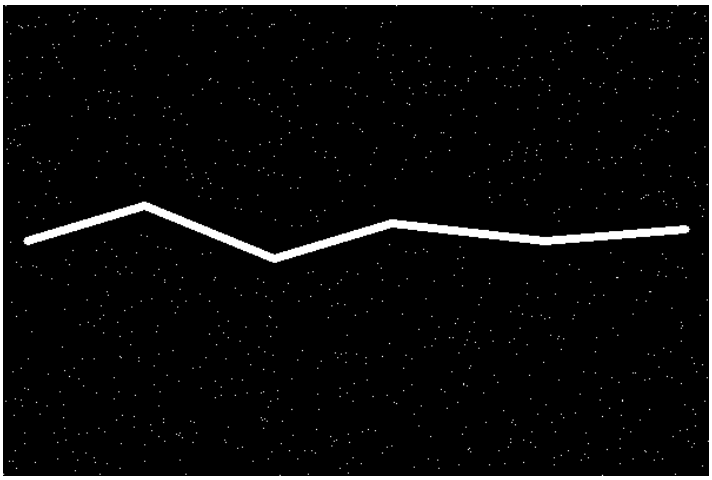


Figure 6.69: Adaptive threshold

Morphological opening and closing

Opening removes isolated specks, closing fills one-pixel gaps in the crack. The crack is now a continuous band.

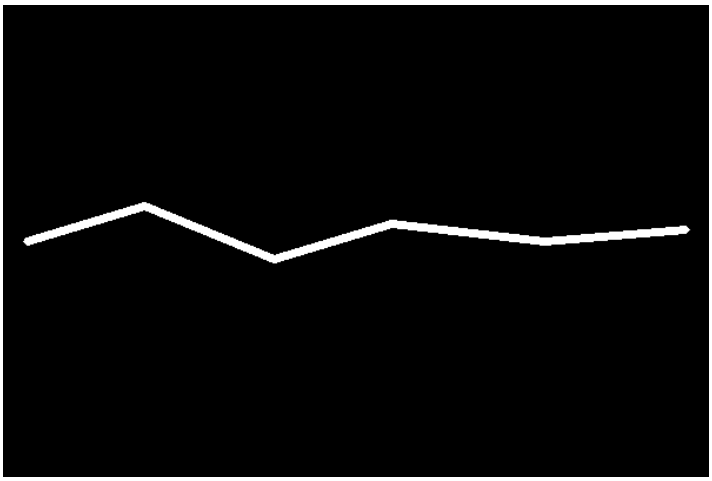


Figure 6.70: Morphological opening and closing

Component mask

Components smaller than 30 px are removed. Here a single elongated component remains, which passes the aspect-ratio test.

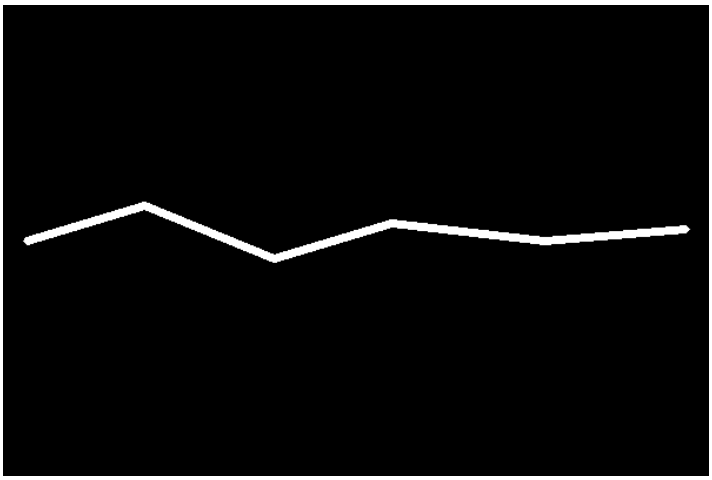


Figure 6.71: Component mask

Skeleton

The band is thinned to a one-pixel centre line. The length of this line is the crack length: 609.6 px, or 60.96 mm at 10 px/mm.

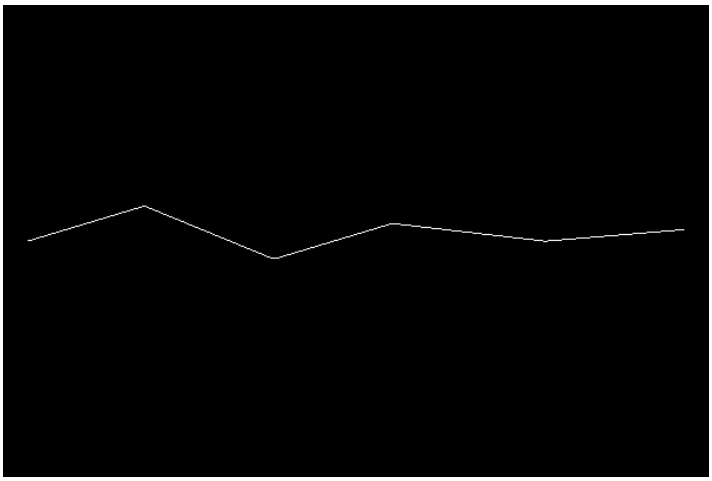


Figure 6.72: Skeleton

Distance transform

Each pixel of the mask stores its distance to the background; the colour map shows larger values (the middle of the band) in lighter colours. The text overlay states mean width 7.19 px / 0.72 mm.

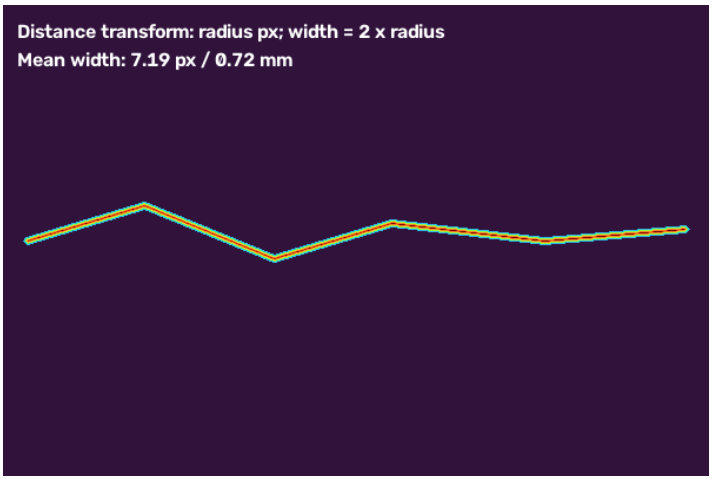


Figure 6.73: Distance transform

Width estimation

Green lines are width samples taken at evenly spaced skeleton points, and the red line is the skeleton. The overlay states mean and maximum widths of 7.19 px and 8.00 px.

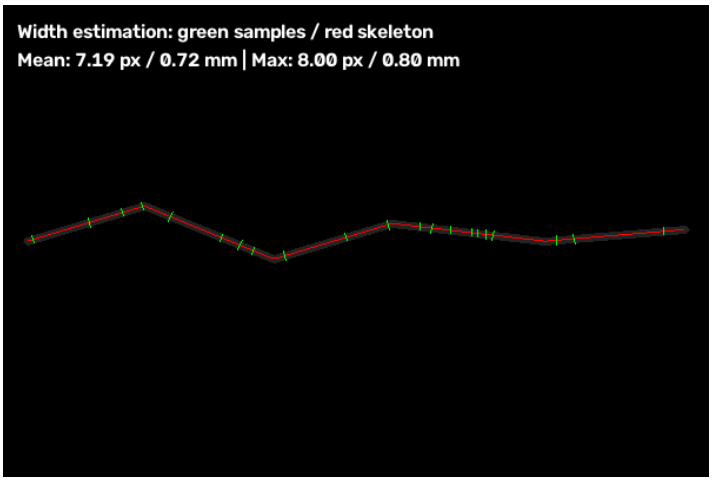


Figure 6.74: Width estimation

Annotated result

The final image draws the crack contour in red, labels it C1 and overlays the width samples. This is the image shown on the results page and stored in `annotated_images/`.

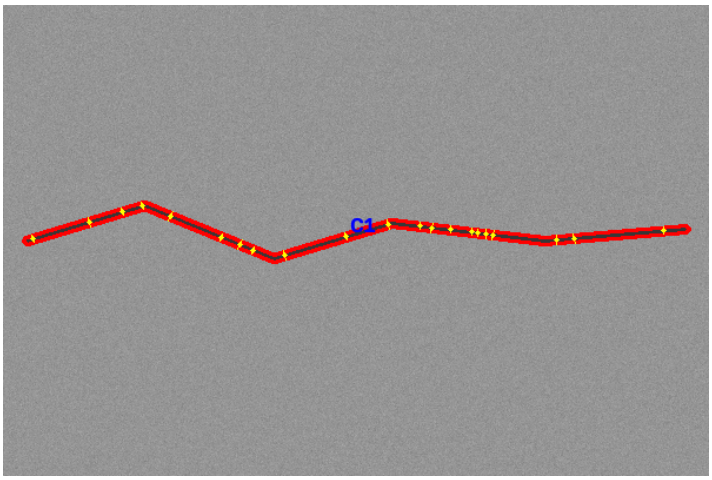


Figure 6.75: Annotated result

Result of the run. One crack was found: length 609.605 px (60.961 mm), mean width 7.194 px (0.719 mm), maximum width 8.000 px (0.800 mm), contour area 4037 px², severity Medium (mean width above 3 px and not above 8 px). Figure 6.76 shows a second run with three separate crack networks.

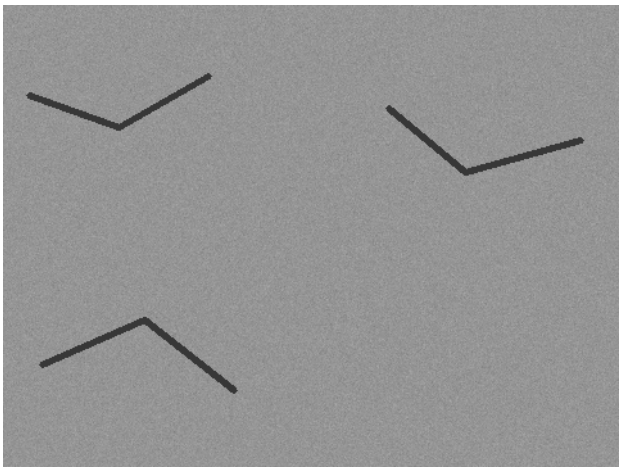


Figure 6.76: Second test image: three separate synthetic cracks

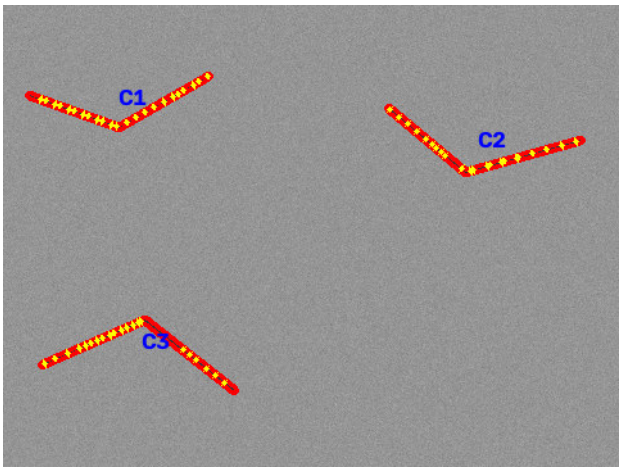


Figure 6.77: Annotated result: cracks C1, C2 and C3 measured independently

6.16 Implementation challenges and how they were handled

Challenge	Observation	Handling in the code
Pavement and wall texture	Global thresholds turn grain into foreground and flood the mask	Adaptive local threshold with offset; minimum component area; elongation filter; coverage limit
Fragmented cracks	A crack split into several short pieces looks like noise	Group components within 18 px; alternative dominant-crack acceptance rule
Several cracks in one image	Keeping only the largest group discards real cracks	Retain every group above an area threshold relative to the largest group
Unstable width values	Branches and blobs distort mean and maximum	Median and 95th percentile of junction-free centre-line samples
Text or objects uploaded by mistake	Letters produce many small elongated components	Component-count, coverage and alignment tests in `validate_crack_image`
Database not running	Application would fail at start	Short timeout and in-memory fallback store
Large files	Memory and time grow with pixels	Resize to 1600 px and enforce file-size and dimension limits

Table 6.5: Implementation challenges

Chapter 7 — Testing and Validation

7.1 Testing strategy and test environment

Testing combined the repository's automated tests (unit and light integration level, `pytest`), a scripted end-to-end run of the web application in a headless browser, export checks, and a controlled experiment on synthetic images. All results in this chapter were obtained by running the code during preparation of this report.

Environment and deviations. Linux container, Python 3.13.16, OpenCV 5.0.0.93, scikit-image 0.26.0, Flask 3.1.3. MongoDB was not available, so the application ran on its in-memory fallback store; the `pymongo` and `python-dotenv` packages could not be installed in the sandbox and were replaced by minimal stand-in modules (the PyMongo stand-in raises on connection, which triggers the fallback), and `pytest` could not be installed, so the repository's test functions were executed by a small script that supplies a temporary directory in place of the `tmp_path` fixture. The test code itself was unchanged. Running `pytest -q` with the real packages and MongoDB on the student's computer is recommended and should be added to this chapter as screenshot evidence.

7.2 Unit testing

The unit tests exercise the pipeline functions directly with synthetic images: detection of a dark line, rejection of a filled circle and of a text-heavy image, acceptance of fragmented and textured-background cracks, adaptive-threshold behaviour on a noisy road-like image, stable width statistics, independent measurement of three separate crack networks, rejection of dispersed regions, calibration and classification, and ZIP packaging. All 16 tests passed.

7.3 Integration testing

`tests/test_app.py` builds the Flask application with a test configuration and posts to `/api/analyze` with no files and with a disallowed file type; both are rejected with HTTP 400. A scripted browser session (Playwright with Chromium) additionally selected two images, submitted the form, was redirected to the results page and checked the page content; the CSV, PDF and ZIP downloads were then requested.

7.4 System testing

The end-to-end run (TC-17) analysed two images (a six-segment crack of drawn width 6 px on a 600 x 400 background, and a 480 x 360 image with three separate crack networks). Four crack regions were detected in total, the summary showed total length 1136.56 px (113.66 mm), average width 6.00 px (0.60 mm), maximum width 8.00 px (0.80 mm) and overall severity Medium, and every stage image was displayed (Figures 6.2 to 6.4).

7.5 Performance and validation tests

Pipeline time (resize excluded, measurement included, single process, CPU) was measured by timing three runs of detection plus measurement on synthetic images; results are in Section 8.3. Negative images (TC-21) and the width-accuracy sweep (TC-22) are reported in Chapter 8. No security penetration test was performed; security measures were reviewed in the code (Section 5.10).

7.6 Test results, defects found and fixes

Test ID	Req. / module	Input / steps	Expected result	Actual result	Status
TC-01	FR-12 / zip service	`create_zip` with image, CSV, PDF and an existing ZIP	ZIP omits itself; each report once	As expected	Pass
TC-02	FR-01	POST /api/analyze with no files	HTTP 400	HTTP 400	Pass
TC-03	FR-02	POST a file with a disallowed extension	HTTP 400	HTTP 400	Pass
TC-04	FR-03/04	Dark 5 px line on light grey; run pipeline, validate, measure	Detected; length > 0; mm length present	As expected	Pass
TC-05	FR-07	4 px horizontal line, measure	5 ≤ mean width ≤ 7 px; max < 8 px; max ≥ mean	As expected	Pass
TC-06	FR-08/09	`validate_pixels_per_mm`, `convert_pixels`, `classify`	"4" → 4; "0" → None; 20 px at 4 px/mm → 5 mm; Good / Medium / Heavy cases	As expected	Pass
TC-07	FR-05	Filled circle on light background	No crack-like component	None found	Pass
TC-08	FR-05	Image with rows of text	Not accepted	Rejected	Pass
TC-09	FR-04	Noisy road-like image, 2 px crack, adaptive threshold	Crack spans > 400 px; total length < 800 px	As expected	Pass
TC-10	FR-04	Noisy texture, percentile threshold 90	Mean binary < 80; crack kept and accepted	As expected	Pass

Test ID	Req. / module	Input / steps	Expected result	Actual result	Status
TC-11	FR-05	Three-segment crack plus five texture blobs	1 to 4 components; accepted	As expected	Pass
TC-12	FR-05	Fragmented dominant crack (three segments)	Accepted	Accepted	Pass
TC-13	FR-05	Dominant crack with texture fragments	Components found; accepted	As expected	Pass
TC-14	FR-05	Wide 8 px dominant crack	Accepted	Accepted	Pass
TC-15	FR-07	Three separate crack networks	At least 3 components, each length > 30 px; accepted	As expected	Pass
TC-16	FR-05	Six dispersed short marks with strict limits	Rejected	Rejected	Pass
TC-17	FR-01 to 13	Browser upload of two synthetic images	Redirect to results; 4 cracks; stages shown	4 cracks; 1136.56 px; Medium	Pass
TC-18	FR-12	GET /reports/<id>/csv	CSV with one row per crack	4 rows, 34 columns	Pass
TC-19	FR-12	GET /reports/<id>/pdf	PDF file	33-page PDF generated	Pass
TC-20	FR-12	GET /reports/<id>/zip	ZIP with images and reports	35 files in archive	Pass
TC-21	FR-05/06	Plain wall, round blob, text image, texture only	No crack accepted	All four rejected	Pass
TC-22	FR-07	Cracks of drawn width 2 to 12 px	Report measured widths	See Table 8.2	Informational

Table 7.1: Test cases and results

Defects and fixes. No test failed in the run reported here. The repository's commit history records earlier defects that were fixed during development (pavement texture producing false cracks, whole-image masks, unstable width values dominated by junctions, and only one crack network being kept); the corresponding regression tests (TC-09 to TC-11, TC-05 and TC-15) are part of the suite. The accuracy experiment (Chapter 8) showed that skeleton length is over-estimated on oblique cracks; this has not been corrected and is listed as a limitation.

7.7 Traceability matrix

Requirement	Test cases
FR-01, FR-02	TC-02, TC-03, TC-17
FR-03, FR-04	TC-04, TC-09, TC-10, TC-17
FR-05, FR-06	TC-07, TC-08, TC-11 to TC-14, TC-16, TC-21
FR-07	TC-04, TC-05, TC-15, TC-22
FR-08, FR-09	TC-06
FR-10, FR-11	TC-17
FR-12	TC-01, TC-18, TC-19, TC-20

Requirement	Test cases
FR-13, FR-14	TC-17 (fallback store used in all runs); delete and search covered by code review only
NFR-01	Section 8.3 timings
NFR-02 to NFR-04	TC-02, TC-03 and code review

Table 7.2: Requirements-to-tests traceability

7.8 Test code and results for each automated test

This section explains each of the 16 automated tests and shows the test files as screenshots, followed by the recorded result of the run. The tests use only synthetic images created in code with NumPy and OpenCV, so they need no external data and run in well under a second each.

TC-01: test_create_zip_excludes_itself_and_duplicate_reports

What it does. Creates a temporary analysis folder containing an image, a CSV, a PDF and an old ZIP, calls `create_zip`, and opens the archive. It asserts that the archive does not contain itself and that each report appears exactly once.

What it proves. Guards against the archive growing by including itself, and against duplicate report entries.

TC-02: test_upload_rejects_empty_batch

What it does. Builds the application with a test configuration (invalid MongoDB host, so the fallback store is used) and posts to `/api/analyze` with no files.

What it proves. The server must answer HTTP 400 and not create an analysis.

TC-03: test_upload_rejects_invalid_extension

What it does. Posts a file with a disallowed extension to the same endpoint.

What it proves. The validator must reject the extension with HTTP 400.

TC-04: test_pipeline_detects_synthetic_dark_crack

What it does. Draws a 5 px dark diagonal line on a light grey image, runs pre-processing, detection, filtering and validation, then measures the first component at 10 px/mm.

What it proves. A component must be found and accepted, with positive pixel and millimetre lengths.

TC-05: test_width_measurement_uses_stable_centerline_statistics

What it does. Draws a horizontal 4 px line (5 px raster width) and measures it.

What it proves. Mean width must lie between 5 and 7 px, maximum width must be at least the mean and below 8 px.

TC-06: test_calibration_and_classification

What it does. Calls `validate_pixels_per_mm`, `convert_pixels` and `classify` with fixed inputs.

What it proves. "4" gives 4; "0" gives None; 20 px at 4 px/mm gives 5 mm; (10, 1) is Good; (10, 4) is Medium; (6000, 1) is Heavy.

TC-07: test_validation_rejects_round_non_crack_region

What it does. Draws a filled circle of radius 35 px and runs the filter.

What it proves. The round blob must not pass the elongation test, so no component is returned.

TC-08: test_validation_rejects_text_heavy_image

What it does. Writes ten rows of text on a light image and runs the full detection and validation.

What it proves. The image must not be accepted as a crack image.

TC-09: test_adaptive_threshold_measures_full_normal_road_crack

What it does. Creates a noisy road-like image with a 2 px wide polyline spanning the width and runs the adaptive-threshold pipeline.

What it proves. The components must reach beyond x = 400 and the total skeleton length must stay below 800 px, showing that the whole crack was found without runaway length.

TC-10: test_percentile_threshold_limits_road_texture_noise

What it does. Same style of image with the percentile-threshold option (90th percentile of BlackHat).

What it proves. The mean binary value must be below 80 (limited foreground), the crack must be kept and the image accepted.

TC-11: test_filter_keeps_coherent_crack_and_discards_texture_blobs

What it does. A three-segment crack plus five short texture blobs scattered over the image.

What it proves. Between one and four components must remain and the image must be accepted, showing that isolated blobs are dropped.

TC-12: test_validation_allows_fragmented_dominant_crack

What it does. A crack drawn as three separate segments with very strict component and coverage limits.

What it proves. The dominant-crack rule must still accept the image.

TC-13: test_validation_allows_dominant_crack_with_texture_fragments

What it does. A long 4 px crack on a darker background with four short low-contrast fragments.

What it proves. Components must be found and the image accepted.

TC-14: test_validation_allows_wide_dominant_road_crack

What it does. A short wide (8 px) crack with relaxed dominant-crack parameters.

What it proves. The image must be accepted.

TC-15: test_multiple_separate_crack_networks_are_measured_independently

What it does. Three distant polylines on a noisy background, measured one by one.

What it proves. At least three components with skeleton length above 30 px each must be returned and the image accepted.

TC-16: test_validation_rejects_dispersed_portrait_like_regions

What it does. Six short unrelated marks spread over the image with strict limits.

What it proves. The image must be rejected because the marks are dispersed and not aligned.

Source of tests/test_app.py

```python
from io import BytesIO

import cv2
import numpy as np
from zipfile import ZipFile

from app import create_app
from services.zip_service import create_zip


def test_create_zip_excludes_itself_and_duplicate_reports(tmp_path):
    root = tmp_path / "analysis"
    reports = root / "reports"
    reports.mkdir(parents=True)
    image = root / "processed.png"
    csv_path = reports / "analysis.csv"
    pdf_path = reports / "analysis.pdf"
    zip_path = reports / "analysis.zip"
    image.write_bytes(b"image")
    csv_path.write_bytes(b"csv")
    pdf_path.write_bytes(b"pdf")

    create_zip("analysis", root, [csv_path, pdf_path], zip_path)

    with ZipFile(zip_path) as archive:
        names = archive.namelist()
    assert "Crack_Analysis_Results/reports/analysis.zip" not in names
    assert names.count("Crack_Analysis_Results/reports/analysis.csv") == 1
    assert names.count("Crack_Analysis_Results/reports/analysis.pdf") == 1


def test_upload_rejects_empty_batch(tmp_path):
    class TestConfig:
        TESTING = True
```

Figure 7.1: tests/test_app.py, lines 1 to 34

```python
        SECRET_KEY = "test"
        STORAGE_DIR = tmp_path
        MAX_CONTENT_LENGTH = 1024 * 1024
        ALLOWED_EXTENSIONS = {"jpg", "jpeg", "png"}
        MAX_FILES_PER_ANALYSIS = 20
        MAX_IMAGE_DIMENSION = 8000
        MONGO_URI = "mongodb://invalid-host:27017"
        MONGO_DB_NAME = "test"
        PROCESSING = {"resize_max_dimension": 1600, "gaussian_kernel": 5, "clahe_clip_limit": 2,
                      "clahe_tile_grid": 8, "blackhat_kernel": 21, "morphology_kernel": 3,
                      "morphology_iterations": 1, "minimum_region_area": 10}
    client = create_app(TestConfig).test_client()
    response = client.post("/api/analyze", data={})
    assert response.status_code == 400


def test_upload_rejects_invalid_extension(tmp_path):
    class TestConfig:
        TESTING = True
        SECRET_KEY = "test"
        STORAGE_DIR = tmp_path
        MAX_CONTENT_LENGTH = 1024 * 1024
        ALLOWED_EXTENSIONS = {"jpg", "jpeg", "png"}
        MAX_FILES_PER_ANALYSIS = 20
        MAX_IMAGE_DIMENSION = 8000
        MONGO_URI = "mongodb://invalid-host:27017"
        MONGO_DB_NAME = "test"
        PROCESSING = {"resize_max_dimension": 1600, "gaussian_kernel": 5, "clahe_clip_limit": 2,
                      "clahe_tile_grid": 8, "blackhat_kernel": 21, "morphology_kernel": 3,
                      "morphology_iterations": 1, "minimum_region_area": 10}
    client = create_app(TestConfig).test_client()
    response = client.post("/api/analyze", data={"images": (BytesIO(b"not image"), "x.txt")}, content_type="multipart/form-data")
    assert response.status_code == 400
```

Figure 7.2: tests/test_app.py, lines 35 to 67

Source of tests/test_processing.py

```python
import cv2
import numpy as np

from services.calibration_service import convert_pixels, validate_pixels_per_mm
from services.classification_service import classify
from services.detection_service import detect_components, filter_crack_components, validate_crack_image
from services.measurement_service import measure_component
from services.preprocessing_service import preprocess


CONFIG = {"gaussian_kernel": 5, "clahe_clip_limit": 2, "clahe_tile_grid": 8, "blackhat_kernel": 21,
          "morphology_kernel": 3, "morphology_iterations": 1, "minimum_region_area": 10}


def test_pipeline_detects_synthetic_dark_crack():
    image = np.full((220, 220, 3), 210, dtype=np.uint8)
    cv2.line(image, (25, 30), (190, 180), (20, 20, 20), 5)
    stages = preprocess(image, CONFIG)
    _, _, components = detect_components(stages["binary"], CONFIG)
    assert components
    assert filter_crack_components(components, CONFIG)
    assert validate_crack_image(filter_crack_components(components, CONFIG), image.shape, CONFIG)
    result = measure_component(components[0], 10)
    assert result["skeleton_length_px"] > 0
    assert result["skeleton_length_mm"] is not None


def test_calibration_and_classification():
    assert validate_pixels_per_mm("4") == 4
    assert validate_pixels_per_mm("0") is None
    assert convert_pixels(20, 4) == 5
    assert classify(10, 1) == "Good"
    assert classify(10, 4) == "Medium"
    assert classify(6000, 1) == "Heavy"
```

Figure 7.3: tests/test_processing.py, lines 1 to 34

```python
def test_validation_rejects_round_non_crack_region():
    image = np.full((220, 220, 3), 210, dtype=np.uint8)
    cv2.circle(image, (110, 110), 35, (20, 20, 20), -1)
    stages = preprocess(image, CONFIG)
    _, _, components = detect_components(stages["binary"], CONFIG)
    assert not filter_crack_components(components, CONFIG)


def test_validation_rejects_text_heavy_image():
    image = np.full((220, 220, 3), 210, dtype=np.uint8)
    for y in range(20, 210, 20):
        cv2.putText(image, "TEXT", (10, y), cv2.FONT_HERSHEY_SIMPLEX, 0.45, (20, 20, 20), 1)
    stages = preprocess(image, CONFIG)
    _, _, components = detect_components(stages["binary"], CONFIG)
    filtered = filter_crack_components(components, CONFIG)
    assert not validate_crack_image(filtered, image.shape, CONFIG)


def test_validation_allows_fragmented_dominant_crack():
    image = np.full((220, 220, 3), 210, dtype=np.uint8)
    for start, end in [((15, 110), (70, 100)), ((75, 98), (130, 80)), ((135, 78), (205, 45))]:
        cv2.line(image, start, end, (20, 20, 20), 4)
    stages = preprocess(image, {**CONFIG, "maximum_crack_components": 1, "maximum_crack_coverage": 0.001})
    _, _, components = detect_components(stages["binary"], {**CONFIG, "maximum_crack_components": 1})
    filtered = filter_crack_components(components, CONFIG)
    assert validate_crack_image(filtered, image.shape, {**CONFIG, "maximum_crack_components": 1, "maximum_crack_coverage": 0.001})


def test_validation_allows_dominant_crack_with_texture_fragments():
    image = np.full((220, 220, 3), 150, dtype=np.uint8)
    cv2.line(image, (20, 112), (200, 92), (80, 80, 80), 4)
    for x, y in ((30, 30), (75, 180), (130, 45), (180, 165)):
```

Figure 7.4: tests/test_processing.py, lines 35 to 68

```
        cv2.line(image, (x, y), (x + 8, y + 2), (110, 110, 110), 2)

    stages = preprocess(image, CONFIG)
    _, _, components = detect_components(stages["binary"], CONFIG)
    filtered = filter_crack_components(components, CONFIG)

    assert filtered
    assert validate_crack_image(filtered, image.shape, CONFIG)


def test_filter_keeps_coherent_crack_and_discards_texture_blobs():
    image = np.full((220, 220, 3), 150, dtype=np.uint8)
    for start, end in [((15, 112), (70, 105)), ((74, 104), (130, 94)), ((134, 93), (205, 78))]:
        cv2.line(image, start, end, (80, 80, 80), 5)
    for x, y in ((30, 30), (110, 40), (180, 170), (40, 180), (190, 45)):
        cv2.line(image, (x, y), (x + 7, y + 2), (80, 80, 80), 3)

    stages = preprocess(image, CONFIG)
    _, _, components = detect_components(stages["binary"], CONFIG)
    filtered = filter_crack_components(components, {**CONFIG, "crack_link_distance_px": 18})

    assert 1 <= len(filtered) <= 4
    assert validate_crack_image(filtered, image.shape, CONFIG)


def test_percentile_threshold_limits_road_texture_noise():
    image = np.full((320, 480, 3), 120, dtype=np.uint8)
    noise = np.random.default_rng(7).normal(0, 8, image.shape[:2]).clip(-30, 30).astype(np.int16)
    textured = np.clip(image[:, :, 0].astype(np.int16) + noise, 0, 255).astype(np.uint8)
    image[:] = cv2.cvtColor(textured, cv2.COLOR_GRAY2BGR)
    cv2.polylines(image, [np.array([[10, 190], [100, 170], [190, 205], [300, 185], [470, 195]])], False, (55, 55, 55), 3)

    stages = preprocess(image, {**CONFIG, "blackhat_threshold_percentile": 90})
    _, _, components = detect_components(stages["binary"], CONFIG)
```

Figure 7.5: tests/test_processing.py, lines 69 to 102

```
    filtered = filter_crack_components(components, CONFIG)

    assert stages["binary"].mean() < 80
    assert filtered
    assert validate_crack_image(filtered, image.shape, CONFIG)


def test_adaptive_threshold_measures_full_normal_road_crack():
    image = np.full((320, 480, 3), 120, dtype=np.uint8)
    noise = np.random.default_rng(12).normal(0, 6, image.shape[:2]).clip(-25, 25).astype(np.int16)
    textured = np.clip(image[:, :, 0].astype(np.int16) + noise, 0, 255).astype(np.uint8)
    image[:] = cv2.cvtColor(textured, cv2.COLOR_GRAY2BGR)
    cv2.polylines(
        image,
        [np.array([[5, 170], [100, 155], [200, 190], [310, 175], [475, 185]])],
        False,
        (55, 55, 55),
        2,
    )

    config = {**CONFIG, "adaptive_threshold_block_size": 41, "adaptive_threshold_offset": 8}
    stages = preprocess(image, config)
    _, _, components = detect_components(stages["binary"], config)
    filtered = filter_crack_components(components, config)
    measurements = [measure_component(component, 10) for component in filtered]

    assert filtered
    assert max(cv2.boundingRect(component["contour"])[0] + cv2.boundingRect(component["contour"])[2] for component in filtered) > 400
    assert sum(item["skeleton_length_px"] for item in measurements) < 800


def test_width_measurement_uses_stable_centerline_statistics():
    image = np.full((220, 220, 3), 210, dtype=np.uint8)
    cv2.line(image, (25, 110), (195, 110), (20, 20, 20), 4)
```

Figure 7.6: tests/test_processing.py, lines 103 to 136

```
    stages = preprocess(image, CONFIG)
    _, _, components = detect_components(stages["binary"], CONFIG)

    measurement = measure_component(components[0], 10)

    assert 5 <= measurement["mean_width_px"] <= 7
    assert measurement["max_width_px"] >= measurement["mean_width_px"]
    assert measurement["max_width_px"] < 8


def test_multiple_separate_crack_networks_are_measured_independently():
    image = np.full((360, 480, 3), 150, dtype=np.uint8)
    cv2.polylines(image, [np.array([[20, 70], [90, 95], [160, 55]])], False, (55, 55, 55), 4)
    cv2.polylines(image, [np.array([[300, 80], [360, 130], [450, 105]])], False, (55, 55, 55), 4)
    cv2.polylines(image, [np.array([[30, 280], [110, 245], [180, 300]])], False, (55, 55, 55), 4)

    config = {**CONFIG, "adaptive_threshold_block_size": 41, "adaptive_threshold_offset": 16}
    stages = preprocess(image, config)
    _, _, components = detect_components(stages["binary"], config)
    filtered = filter_crack_components(components, config)
    measurements = [measure_component(component, 10) for component in filtered]

    assert len(filtered) >= 3
    assert all(item["skeleton_length_px"] > 30 for item in measurements)
    assert validate_crack_image(filtered, image.shape, config)


def test_validation_allows_wide_dominant_road_crack():
    image = np.full((384, 544, 3), 150, dtype=np.uint8)
    cv2.line(image, (80, 310), (160, 280), (80, 80, 80), 8)
    stages = preprocess(image, CONFIG)
    _, _, components = detect_components(stages["binary"], CONFIG)
    filtered = filter_crack_components(components, CONFIG)
```

Figure 7.7: tests/test_processing.py, lines 137 to 170

```
    assert validate_crack_image(
        filtered,
        image.shape,
        {
            **CONFIG,
            "minimum_dominant_crack_area_ratio": 0.003,
            "minimum_dominant_crack_aspect_ratio": 2.4,
        },
    )


def test_validation_rejects_dispersed_portrait_like_regions():
    image = np.full((220, 220, 3), 210, dtype=np.uint8)
    for x, y in ((30, 30), (110, 35), (185, 45), (40, 150), (120, 140), (195, 165)):
        cv2.line(image, (x, y), (x + 15, y + 4), (20, 20, 20), 4)
    stages = preprocess(image, CONFIG)
    _, _, components = detect_components(stages["binary"], CONFIG)
    filtered = filter_crack_components(components, CONFIG)
    assert not validate_crack_image(
        filtered,
        image.shape,
        {**CONFIG, "maximum_crack_components": 1, "maximum_crack_coverage": 0.001},
    )
```

Figure 7.8: tests/test_processing.py, lines 171 to 193

Recorded result of the test run

The output below was produced by the script that executes the repository's test functions (Section 7.1). Each line shows the test file, the test name, the result and the time in seconds.

```
test_app test_create_zip_excludes_itself_and_duplicate_reports PASS 0.002
test_app test_upload_rejects_empty_batch PASS 0.022
test_app test_upload_rejects_invalid_extension PASS 0.013
test_processing test_adaptive_threshold_measures_full_normal_road_crack PASS 0.077
test_processing test_calibration_and_classification PASS 0.0
test_processing test_filter_keeps_coherent_crack_and_discards_texture_blobs PASS 0.007
test_processing test_multiple_separate_crack_networks_are_measured_independently PASS 0.057
test_processing test_percentile_threshold_limits_road_texture_noise PASS 0.384
test_processing test_pipeline_detects_synthetic_dark_crack PASS 0.012
test_processing test_validation_allows_dominant_crack_with_texture_fragments PASS 0.003
test_processing test_validation_allows_fragmented_dominant_crack PASS 0.003
test_processing test_validation_allows_wide_dominant_road_crack PASS 0.009
test_processing test_validation_rejects_dispersed_portrait_like_regions PASS 0.006
test_processing test_validation_rejects_round_non_crack_region PASS 0.002
test_processing test_validation_rejects_text_heavy_image PASS 0.007
test_processing test_width_measurement_uses_stable_centerline_statistics PASS 0.011
16 / 16
```

7.9 Test coverage assessment

Area	Covered by	Not covered
Upload validation	Empty batch, bad extension	Corrupt image data, oversized dimensions, more than 20 files, unreadable file
Pre-processing and detection	Synthetic lines, texture, circle, text, dispersed marks	Real photographs, extreme lighting, very large images
Measurement	Width statistics, multiple cracks, calibration	Length accuracy at different angles (added in Section 8.8), sub-pixel accuracy
Classification	Three severity outcomes	Boundary values exactly at the thresholds
Persistence	Fallback store used by all tests	Live MongoDB, indexes, history search and delete
Reports	ZIP assembly (unit test); CSV, PDF, ZIP generation (manual run)	Automated tests of CSV and PDF content
Web pages	Manual browser run	Automated UI tests, accessibility checks

Table 7.3: Test coverage assessment

A coverage tool was not available in the sandbox, so no percentage is given. The table is a manual assessment from reading the tests and the code.

Chapter 8 — Results, Discussion and Limitations

8.1 Implemented features and evidence of completion

All features in FR-01 to FR-14 are implemented in the repository. Evidence is the passing tests (Table 7.1), the end-to-end run (Figures 6.2 to 6.4) and the generated reports (Figure 6.5). Figure 8.1 shows the pipeline outputs for the synthetic image: the annotated result with width-sample lines, the skeleton, and the width-estimation image with the red skeleton and green width samples.

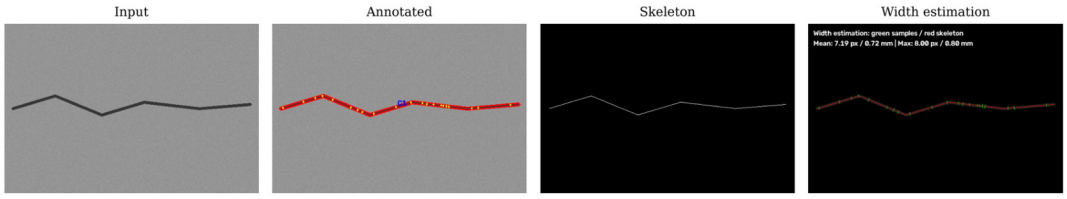


Figure 8.1: Input, annotated result, skeleton and width-estimation image

8.2 Results against each project objective

Objective	Result	Status
O1 Upload and validate	Valid files accepted; empty and invalid-type uploads rejected with 400	Met
O2 Segment cracks	Synthetic cracks of 2 to 12 px drawn on noisy backgrounds were detected in all 8 sweep images; stages visible	Met on synthetic data
O3 Reject non-crack images	4 of 4 negative synthetic images rejected; 3 dedicated unit tests pass	Met on synthetic data
O4 Measure length and width	Mean width within -0.2 to +0.6 px of the true raster width; skeleton length 5 to 7 % too long on oblique cracks	Met for width; length bias noted (Section 8.5)
O5 Calibrate and classify	px to mm conversion and 3-level labels work as specified	Met
O6 Present and export	Results page, CSV, PDF, ZIP produced	Met
O7 History	Implemented with MongoDB and fallback; not exercised with a live MongoDB in this report	Implemented; partly verified

Table 8.1: Objective-wise results

8.3 Quantitative metrics and how they were measured

Width accuracy. Eight synthetic 600 x 400 images were created with a Gaussian-noise background (standard deviation 6 grey levels) and a six-point polyline of fixed drawn width (thickness argument 2, 3, 4, 5, 6, 8, 10 and 12, dark grey 55 on a mid-grey background) using OpenCV's `polylines`. The application's own pipeline and parameters from `config.py` were applied and the mean-width and maximum-width outputs

compared with the width of the line as actually rasterised. This distinction matters: OpenCV draws a line with thickness argument t that is 3, 5, 5, 7, 7, 9, 11 and 13 pixels wide for t = 2, 3, 4, 5, 6, 8, 10 and 12 (counted on a horizontal test line), so the thickness argument is not the true width. An earlier comparison against the thickness argument gave a misleading apparent over-estimate and was discarded; all comparisons here use the counted raster width.

Thickness argument	Raster width (px)	Mean width (px)	Error (px)	Error (%)	Max width P95 (px)	Length (px)	Severity
2	3	2.8	-0.20	-6.7 %	4.0	613.8	Good
3	5	5.6	+0.60	+12.0 %	6.0	610.2	Medium
4	5	5.6	+0.60	+12.0 %	6.0	610.2	Medium
5	7	7.19	+0.19	+2.7 %	8.0	609.6	Medium
6	7	7.19	+0.19	+2.7 %	8.0	609.6	Medium
8	9	8.79	-0.21	-2.3 %	10.0	612.2	Heavy
10	11	10.79	-0.21	-1.9 %	12.0	612.2	Heavy
12	13	12.79	-0.21	-1.6 %	14.0	608.5	Heavy

Table 8.2: Measured width against true raster width on synthetic images (geometric polyline length 578.9 px)

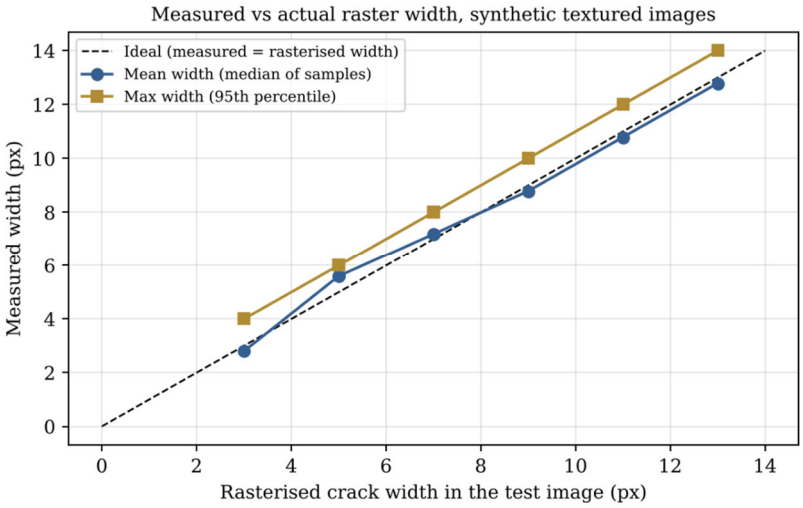


Figure 8.2: Measured versus true raster crack width

Length. The skeleton length of the polyline was 608.5 to 613.8 px against a geometric length of 578.9 px, about 5 to 6 % too long. A separate test with a straight 400 px line at seven angles (Section 8.8) showed errors of about -1 % at 0, 45 and 90 degrees but +6 to +7 % at 15, 30, 60 and 75 degrees.

Speed. Detection plus measurement (median of three runs) took 0.031 s for 640 x 480, 0.079 s for 1024 x 768 and 0.157 s for 1600 x 1200 images with one crack. In the browser

run, the per-image timings reported by the application for a 480 x 360 image with three cracks were 0.0035 s (resize), 0.0046 s (pre-processing), 0.0042 s (detection), 0.0405 s (measurement) and 0.0098 s (annotation). Times depend on the machine and on the number and length of cracks (measurement time is dominated by skeleton-point neighbour searches).

Rejection. Plain wall, a filled circle on noisy background, a text-filled image and texture only (standard deviation 10) were all rejected (no crack components retained).

Multiple cracks. Three separate polylines drawn with thickness argument 4 (raster width 5 px) produced three measured cracks of 163.7, 179.2 and 184.0 px skeleton length, each with mean width 5.6 px.

8.4 Comparison with baseline / existing method

No comparison with manual gauge measurements, with a deep-learning model or with another open-source tool was carried out, so no claim of superiority is made. The experiment above measures the system against known synthetic geometry only. A comparison with gauge readings and with an annotated public dataset is listed as future work.

8.5 Interpretation of results and unexpected outcomes

- Against the true raster width, the mean-width estimate was between -0.21 and +0.60 px (-6.7 % to +12 %) for bars 3 to 13 px wide. The largest relative error is for the thinnest cracks, as expected for pixel-based measurement.
- Several widths give identical measurements (thickness arguments 3 and 4 both give 5.6 px; 5 and 6 both give 7.19 px, because they rasterise to the same width), and the other values differ in steps of about 0.4 to 1 px, which shows that width values are quantised by the discrete distance transform and the threshold. Sub-pixel accuracy should not be expected.
- For an ideal rectangular bar the method returns the true width plus one pixel, because the distance transform measures to the nearest background pixel (a 5 px bar gives 2 x 3.0 = 6.0 px; Section 5.12). In the full pipeline the threshold slightly narrows or widens the mask and the net error stayed within the range above; the two effects were not separated.
- Because the severity thresholds are in pixels (3 and 8 px), the same physical crack could receive different labels at different image resolutions; for example, the crack with raster width 5 px was labelled Medium because its measured width was 5.6 px.
- Skeleton length is over-estimated on oblique cracks (Section 8.8), probably because the neighbour-link counting adds both a diagonal and redundant axis-aligned links along a staircase skeleton; this was not isolated by a separate experiment.

- The maximum-width statistic (95th percentile) is about 0 to 1.4 px above the mean width on straight synthetic cracks.

8.6 Known limitations and unresolved issues

- **Synthetic evidence only.** Accuracy and robustness have been measured on synthetic images; no quantitative evaluation on annotated real concrete or pavement photographs has been done.
- **Scale handling.** Calibration is a single default (10 px/mm) for all images. Images larger than 1600 px are resized, but the scale is not adjusted for the resize factor (the factor is stored but not applied to the px/mm value), so millimetre values for resized images are only correct if the supplied scale refers to the resized image. A function to recalibrate stored measurements exists in `models/measurement_model.py`, but no route calls it.
- **Length bias** on oblique cracks (Section 8.8) and one-pixel quantisation of width (Section 8.5); millimetre values inherit both.
- **Appearance assumptions.** Cracks are assumed darker than their surroundings; shadows, joints, stains, vegetation and markings can be mistaken for cracks, and very faint cracks can be missed.
- **Batch rejection.** A single image without a detectable crack causes the whole upload to be discarded.
- **Labels.** The results page labels the binary stage "Otsu threshold" although the default pipeline uses an adaptive Gaussian threshold; the PROCESSING configuration, not the label, is authoritative.
- **No authentication or multi-user control**; the delete endpoint is unauthenticated, so the application must not be exposed publicly.
- **Persistence.** Without MongoDB the history is lost on restart; `report_history` is indexed but never written.
- **Severity categories** are software classes, not engineering judgement, and ignore crack depth, location and cause.
- The repository name and application title use the spelling "Crack Detaction" in the user interface and README.

8.7 User feedback or evaluation

No user study was carried out and no user data were collected.

8.8 Additional experiments: robustness and sensitivity

Seven further experiments were run to see how the pipeline behaves when conditions change. In every case the application's own services and the default parameters from `config.py` were used, on synthetic 600 x 400 images unless stated otherwise (crack: six-point polyline, thickness argument 6, which rasterises to 7 px; background

grey 150 plus Gaussian noise). The scripts are the same style as the accuracy experiment and the numbers below are the recorded outputs.

Experiment A — noise level

Noise SD (grey levels)	Runs detected (of 5)	Mean width (px)	Mean components
2	5	7.19	1.0
4	5	7.19	1.0
6	5	7.19	1.0
8	5	7.19	1.0
10	5	7.19	1.0
12	5	7.19	1.0
16	5	7.19	1.0
20	5	7.19	1.0
25	5	7.19	1.0

Table 8.3: Effect of background noise on detection and width

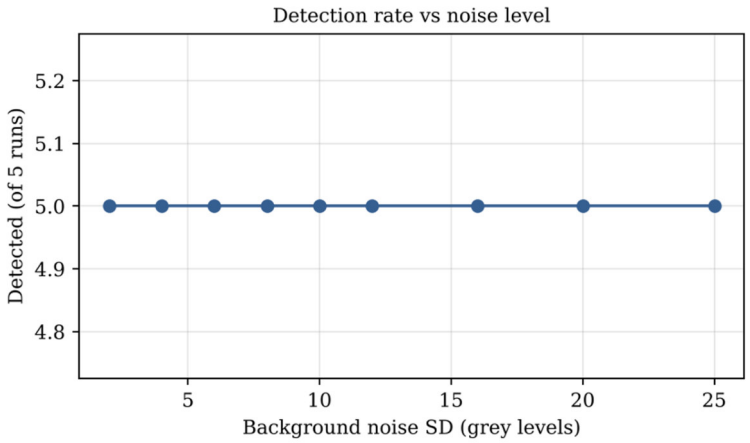


Figure 8.3: Detection rate versus noise level

The crack was detected in all five runs at every noise level from 2 to 25 grey levels, with an identical mean width of 7.19 px and one component. The crack is dark (contrast about 95 grey levels), so noise up to this level does not hide it. This says nothing about weak cracks, which Experiment B addresses.

Experiment B — crack contrast

Crack grey level	Contrast (150 - level)	Components	Accepted	Mean width (px)	Length (px)
20	130	1	yes	7.19	609.6
55	95	1	yes	7.19	609.6
80	70	1	yes	7.19	609.6
100	50	1	yes	7.19	609.6
115	35	1	yes	7.19	609.6
125	25	0	no	-	0.0

Crack grey level	Contrast (150 - level)	Components	Accepted	Mean width (px)	Length (px)
135	15	0	no	-	0.0

Table 8.4: Effect of crack contrast on detection

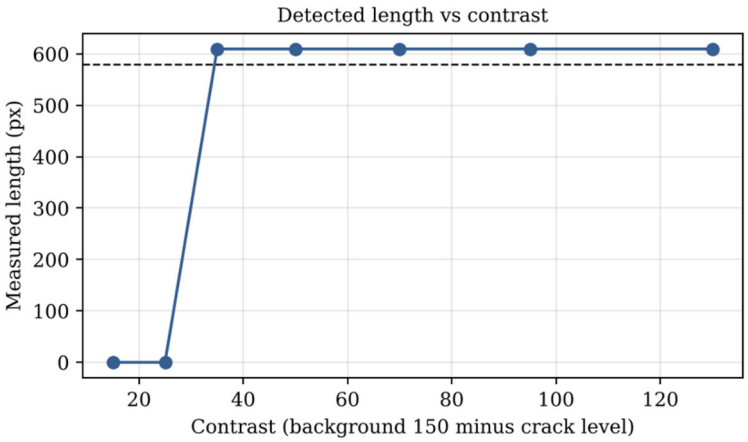


Figure 8.4: Measured length versus contrast (dashed line: geometric length 578.9 px)

Cracks with contrast of 35 grey levels or more were detected with identical measurements. At 25 and 15 grey levels nothing was detected. The practical limit of the default parameters on this background is therefore a contrast of roughly 25 to 35 grey levels; faint cracks on real surfaces may fall below it. Lowering the threshold offset would increase sensitivity but also increase false positives on texture (Experiment D).

Experiment C — crack orientation

Angle (deg)	Geometric length (px)	Measured length (px)	Error (%)	Mean width (px)
0	400.0	396.0	-1.0	8.0
15	400.5	427.7	+6.8	7.19
30	400.5	428.8	+7.1	7.19
45	400.2	398.8	-0.4	7.19
60	401.0	426.8	+6.4	7.19
75	400.5	425.2	+6.2	7.19
90	400.0	395.0	-1.2	8.0

Table 8.5: Length and width of a straight crack at different angles

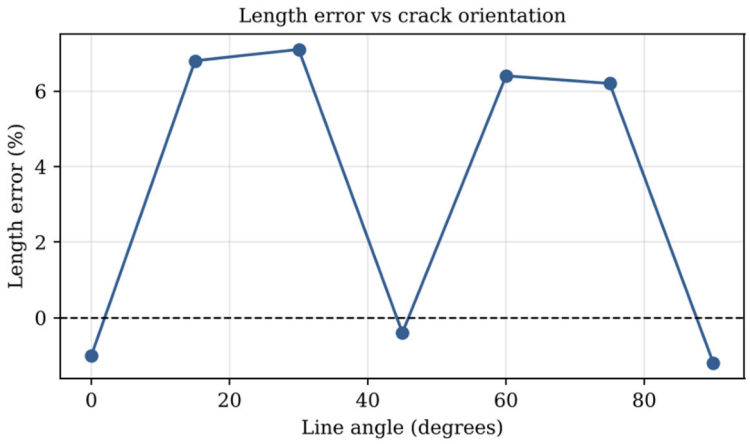


Figure 8.5: Length error versus line angle

The width is stable (7.19 or 8.0 px) at all angles. The length, however, is over-estimated by 6 to 7 % at 15, 30, 60 and 75 degrees and is accurate within about 1 % at 0, 45 and 90 degrees. This pattern points to the way skeleton links are counted: along an oblique staircase-shaped skeleton the loop adds a diagonal link and additional axis-aligned links for the same stretch of crack. The effect is the cause of the 5 to 6 % excess seen on the polyline in Section 8.3. For reporting, lengths of oblique cracks should be read as up to about 7 % high until the counting is corrected.

Experiment D — adaptive-threshold offset C

Offset C	Crack components	Crack accepted	Mean width (px)	Texture-only image accepted	Foreground coverage (%)
4	1	yes	7.19	no	1.68
8	1	yes	7.19	no	1.68
12	1	yes	7.19	no	1.68
16	1	yes	7.19	no	1.68
20	1	yes	7.19	no	1.68
24	1	yes	7.19	no	1.68

Table 8.6: Sensitivity to the threshold offset (texture SD 10)

Within the range 4 to 24, the offset made no difference to the crack (one component, 7.19 px) and the texture-only image (noise SD 10) was always rejected. This is expected for a strong crack on a plain background. The experiment does not show how the offset behaves on weak cracks or on structured texture, where it will matter more, and the default of 16 was inherited from the repository's tuning on its own test images.

Experiment E — adaptive-threshold block size

Block size (px)	Crack components	Crack accepted	Mean width (px)	Texture-only image accepted	Coverage (%)
21	1	yes	7.19	no	1.68
31	1	yes	7.19	no	1.68
41	1	yes	7.19	no	1.68
61	1	yes	7.19	no	1.68
81	1	yes	7.19	no	1.68

Table 8.7: Sensitivity to the threshold neighbourhood size

Block sizes from 21 to 81 px gave the same result. A block that is much narrower than the crack would break it, while one much larger than the crack's surroundings would behave like a global threshold; neither extreme was tested because the default is well inside the stable range.

Experiment F — image scale

Scale	Image size	Thickness argument	Mean width (px)	Measured length (px)	Geometric length (px)	Time (s)
0.5	300x200	3	5.6	303.3	289.5	0.016
0.75	450x300	4	5.6	457.8	434.2	0.025
1.0	600x400	6	7.19	609.6	578.9	0.04
1.5	900x600	9	10.79	918.3	868.4	0.077
2.0	1200x800	12	12.79	1222.4	1157.9	0.122

Table 8.8: Same scene at five image scales

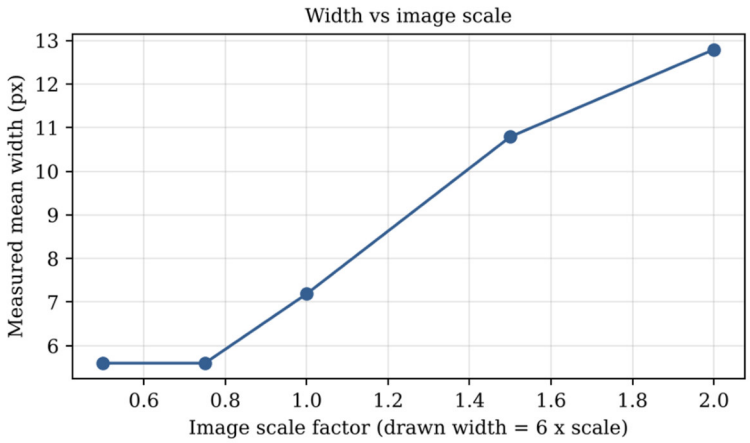


Figure 8.6: Measured width versus image scale

The measured width grows with the image scale, as it should for a quantity measured in pixels, from 5.6 px in the 300 x 200 image to 12.79 px in the 1200 x 800

image. The length scales proportionally and its error stays between 4.8 % and 5.7 %. This confirms that millimetre results are only meaningful when the pixels-per-millimetre value matches the image actually analysed, and it illustrates why the resize step must be accompanied by a scale correction (Section 8.6).

Experiment G — time per processing stage

Stage	Mean time (s)	Standard deviation (s)
Pre-processing	0.0208	0.0012
Detection, filtering and validation	0.0168	0.0015
Measurement (skeleton, width, diagnostic images)	0.057	0.004
Total	0.0946	

Table 8.9: Mean stage time over 20 images of 1024 x 768 px with one crack

Measurement takes the largest share (about 60 %), consistent with the quadratic neighbour search discussed in Section 5.14. The total of about 0.095 s per image means that a batch of 20 images would be analysed in a couple of seconds on this machine, excluding file writing and report generation.

8.9 Discussion

What worked. On synthetic data the pipeline detected strong cracks reliably at all tested noise levels, rejected all four kinds of non-crack image, measured width to within about 0.6 px of the true raster width, handled several crack networks in one image and ran in a fraction of a second. The stage-by-stage output made every result easy to inspect, which was one of the project's design aims.

What did not. Weak cracks (contrast below about 30 grey levels) were not detected; lengths of oblique cracks were about 6 to 7 % too long; width values are quantised to roughly one pixel; and the scale is fixed. These are properties of the current implementation and parameters, not of classical methods in general.

Comparison of the method with its alternatives. A learned segmentation model would probably cope better with real surface texture, but it would need labelled images and a GPU for training, and it would not by itself provide a width in millimetres. The present approach trades robustness on unseen textures for transparency, speed and the absence of training data. A hybrid in which a learned model proposes the crack mask and the present measurement stage computes length and width is a natural next step.

8.10 Threats to validity

- **Construct validity:** the crack width of a synthetic line is exactly known, but a real crack has irregular, soft edges, so the definition of width differs. Real validation needs gauge readings at marked points.
- **Internal validity:** an early version of the accuracy comparison used the wrong ground truth (the drawn thickness argument instead of the rasterised width); this was found and corrected, and all values in this report use the counted raster width.
- **External validity:** the experiments use a single synthetic background type and a single crack colour; results may differ on concrete, brick, asphalt or painted walls.
- **Reproducibility:** the images are generated with fixed random seeds, and the parameters are those of `config.py` at the time of writing. The scripts are not part of the repository; adding them as a test or benchmark folder would make the experiments repeatable.

Chapter 9 — Conclusion and Future Scope

9.1 Summary of the problem and solution

The project addressed the need for a repeatable, documented way to detect cracks in images and to measure their length and width. It delivered a Flask web application that applies a classical OpenCV and scikit-image pipeline, estimates length and width from skeletons and distance transforms, converts to millimetres, labels severity, and exports CSV, PDF and ZIP reports, with MongoDB persistence and an in-memory fallback.

9.2 Objective-wise completion status

Objectives O1, O2, O3, O5 and O6 are met as verified on automated tests and synthetic images; O4 is implemented, with width accurate to within about 0.6 px on synthetic data but length over-estimated by about 6 to 7 % on oblique cracks; O7 is implemented but was exercised only through the fallback store. See Table 8.1.

9.3 Main contributions and conclusions

- An end-to-end, explainable crack-measurement tool that runs on a CPU without training data.
- A texture-suppressing detection and validation stage (shape, coherence, coverage and alignment rules) that rejected all negative images in the experiment.
- A robust width statistic (median and 95th percentile of centreline distance-transform widths) and per-crack length, area and severity output.
- A reporting workflow with all intermediate images, suitable for inspection records.

9.4 Limitations

The principal limitations are the absence of real-photograph validation, the one-pixel width quantisation, the length over-estimate on oblique cracks, the fixed and resize-unaware calibration, and the lack of authentication (Section 8.6).

9.5 Future enhancements

- Evaluate on public annotated crack datasets and on gauge measurements; report precision, recall, IoU and width error.
- Correct the skeleton-length counting for oblique cracks (for example, by counting each skeleton step once along an ordered path) and add regression tests with known lengths and widths.
- Per-image calibration (reference object or user-entered scale) and automatic adjustment of the scale for resized images.
- Define severity thresholds in millimetres, with standards-based categories agreed with a structural engineer.

- Optional learned segmentation model combined with the existing measurement stage.
- Perspective correction, video or drone input, crack-growth tracking across visits, user accounts and deployment hardening.

9.6 Learning outcomes and skills demonstrated

The project demonstrates applied image processing (contrast enhancement, thresholding, morphology, skeletonisation, distance transforms), Python web development with Flask, NoSQL data modelling with MongoDB, automated testing with synthetic data, report generation, version control with Git and GitHub, and the practice of reporting measured results and limitations honestly.

References

- Borgefors, G. (1986). Distance transformations in digital images. *Computer Vision, Graphics, and Image Processing, 34*(3), 344–371. https://doi.org/10.1016/S0734-189X(86)80047-0
- Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.
- Cha, Y.-J., Choi, W., & Büyüköztürk, O. (2017). Deep learning-based crack damage detection using convolutional neural networks. *Computer-Aided Civil and Infrastructure Engineering, 32*(5), 361–378. https://doi.org/10.1111/mice.12263
- Dorafshan, S., Thomas, R. J., & Maguire, M. (2018). Comparison of deep convolutional neural networks and edge detectors for image-based crack detection in concrete. *Construction and Building Materials, 186*, 1031–1045. https://doi.org/10.1016/j.conbuildmat.2018.08.011
- Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature, 585*, 357–362. https://doi.org/10.1038/s41586-020-2649-2
- Koch, C., Georgieva, K., Kasireddy, V., Akinci, B., & Fieguth, P. (2015). A review on computer vision based defect detection and condition assessment of concrete and asphalt civil infrastructure. *Advanced Engineering Informatics, 29*(2), 196–210. https://doi.org/10.1016/j.aei.2015.01.008
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61.
- Mohan, A., & Poobal, S. (2018). Crack detection using image processing: A critical review and analysis. *Alexandria Engineering Journal, 57*(2), 787–798. https://doi.org/10.1016/j.aej.2017.01.020
- MongoDB, Inc. (n.d.). *MongoDB documentation*. https://www.mongodb.com/docs/
- OpenCV team. (n.d.). *OpenCV documentation*. https://docs.opencv.org/
- Otsu, N. (1979). A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics, 9*(1), 62–66. https://doi.org/10.1109/TSMC.1979.4310076
- Özgenel, Ç. F. (2019). *Concrete crack images for classification* (Version 2) [Data set]. Mendeley Data. https://doi.org/10.17632/5y9wdsg2zt.2
- Özgenel, Ç. F., & Gönenç Sorguç, A. (2018). Performance comparison of pretrained convolutional neural networks on crack detection in buildings. *Proceedings of the 35th International Symposium on Automation and Robotics in Construction (ISARC)*.
- Pallets. (n.d.). *Flask documentation*. https://flask.palletsprojects.com/
- ReportLab. (n.d.). *ReportLab documentation*. https://docs.reportlab.com/
- van der Walt, S., Schönberger, J. L., Nunez-Iglesias, J., et al. (2014). scikit-image: Image processing in Python. *PeerJ, 2*, e453. https://doi.org/10.7717/peerj.453
- Yamaguchi, T., & Hashimoto, S. (2010). Fast crack detection method for large-size concrete surface images using percolation-based image processing. *Machine Vision and Applications, 21*(5), 797–809. https://doi.org/10.1007/s00138-009-0189-8

- Zhang, L., Yang, F., Zhang, Y. D., & Zhu, Y. J. (2016). Road crack detection using deep convolutional neural network. *2016 IEEE International Conference on Image Processing (ICIP)*, 3708–3712. https://doi.org/10.1109/ICIP.2016.7533052
- Zhang, T. Y., & Suen, C. Y. (1984). A fast parallel algorithm for thinning digital patterns. *Communications of the ACM, 27*(3), 236–239. https://doi.org/10.1145/357994.358023
- Zuiderveld, K. (1994). Contrast limited adaptive histogram equalization. In P. S. Heckbert (Ed.), *Graphics Gems IV* (pp. 474–485). Academic Press.
- Patel, M. (2026). *OpenCV-Based System for Crack Identification and Width Estimation* [Source code]. GitHub. https://github.com/ManPatel448/OpenCV-Based-System-for-Crack-Identification-and-Width-Estimation

Appendices / Annexures

Appendices A to J follow the list in the institutional format. Items that depend on university records are marked for the student to attach.

Appendix A — Approved synopsis / proposal and approval record

[Attach the approved synopsis and approval record, if required by the institution.]

Appendix B — Detailed Software Requirements Specification

Purpose and product perspective. A self-contained web application used locally to analyse crack photographs. **User classes:** inspector or student (single class). **Operating environment:** Python 3.10+, any OS, optional MongoDB. **Design constraints:** OpenCV-based processing; images stored on disk; calibration by a single positive number.

Interface	Specification
User interface	Web pages /upload, /analysis/<id>, /history, /reports; files .jpg, .jpeg, .jfif, .png
Software interface	OpenCV (image operations), scikit-image (skeletonize), NumPy, pandas (CSV), ReportLab (PDF), PyMongo (database)
Communication	HTTP on 127.0.0.1:5000; JSON for APIs; multipart form data for upload
Data limits	At most 20 files per analysis; request size limit 16 MB; longest side 8000 px; resize to 1600 px
Environment variables	FLASK_SECRET_KEY, MONGO_URI, MONGO_DB_NAME, MAX_CONTENT_LENGTH, STORAGE_DIR, FLASK_AUTO_RELOAD

Table 9.1: External interface specification

Appendix C — Database dictionary and sample report

Collection	Fields
analyses	analysis_id (string, unique), analysis_name, notes, created_at, updated_at, status (Uploaded / Processing / Completed), total_images, total_detected_regions, calibration_enabled, pixels_per_mm, processing_configuration, severity_thresholds, error_summary
images	image_id, analysis_id, original_filename, stored_filename, original_path, uploaded_at, processing_status (Uploaded / Completed / Failed), workflow_times, image_width, image_height, resize_scale, processed_paths, annotated_path, error_message
crack_measurements	crack_id, analysis_id, image_id, created_at, contour_area_px, contour_area_mm2, skeleton_length_px, skeleton_length_mm, mean_width_px, max_width_px, mean_width_mm, max_width_mm, calibration_status (calibrated / pixel-only), severity, measurement_points (x, y, width_px, width_mm, angle), width_sample_count
report_history	Indexed (analysis_id, generated_at) but not written by the current code

Table 9.2: Logical database dictionary

Sample CSV rows (from the end-to-end run, selected columns; the file has 34 columns):

Crack_ID	Length (px)	Length (mm)	Mean width (px)	Max width (px)	Mean width (mm)	Severity
8fd53828-1	609.605	60.961	7.194	8.000	0.719	Medium
45fe284b-1	163.681	16.368	5.600	5.600	0.560	Medium
45fe284b-2	179.238	17.924	5.600	5.600	0.560	Medium
45fe284b-3	184.037	18.404	5.600	5.600	0.560	Medium

Table 9.3: Sample CSV measurement rows

Appendix D — Detailed test cases and test evidence

The test cases are listed in Table 7.1. The procedures for the experiments are as follows.

- **Unit tests (TC-01 to TC-16):** functions in `tests/test_app.py` and `tests/test_processing.py`, executed with Python 3.13.16 using a small runner; 16 of 16 passed, each in under 0.25 s.
- **TC-17 to TC-20:** Flask server started with `python app.py` (auto-reload off); Playwright/Chromium opened /upload, attached two images, clicked "Upload and detect cracks", waited for the results URL, captured screenshots; reports requested with HTTP GET.
- **TC-21 negatives:** 400 x 300 images: constant grey wall (170); noisy background with a filled circle of radius 40; light background with rows of text; noisy background (SD 10) only.
- **TC-22 sweep:** 600 x 400 images; background 150 plus Gaussian noise (SD 6, clipped at ±25); polyline points (20,200), (120,170), (230,215), (330,185), (460,200), (580,190); colour (55,55,55); widths 2, 3, 4, 5, 6, 8, 10, 12 px; pixels per millimetre 10.

Appendix E — User manual / installation and execution guide

1. Follow the steps in Section 6.9 to install and start the application.
2. Open http://127.0.0.1:5000 (redirects to /upload). Click the drop area or drag images onto it. Use "Remove image" to deselect a file.
3. Click "Upload and detect cracks". If an image contains no detectable crack, an error message is shown and nothing is stored; choose clearer images and retry.
4. On the results page read the summary, remark and thresholds; click any image to enlarge it; scroll to the measurement table for per-crack values.
5. Use Download CSV, Download PDF or Download ZIP. Use /history to find or delete earlier analyses.
6. Interpret severity as a screening label only; consult a qualified engineer for structural decisions.

Appendix F — Repository details and selected source

Repository:https://github.com/ManPatel448/OpenCV-Based-System-for-Crack-Identification-and-Width-Estimation.git Commit history (from `git log`):

Date	Commit message
2026-10-05	Initial commit: crack identification application
2026-10-05	Delete PROJECT_DOCUMENTATION.md; Delete run_crackscope.bat; Update README to remove Windows launch instructions
2026-10-12	Improve crack validation for textured images; Merge remote repository updates; Improve road crack validation and auto reload; Restore Windows application launcher
2026-10-12	Keep only coherent crack paths in annotations; Reduce pavement texture in crack segmentation; Improve normal crack measurement geometry; Stabilize crack width measurements; Detect multiple crack networks per image
2026-10-18	Update project files (three commits); Remove batch file from repository

Table 9.4: Commit history summary

File	Lines	File	Lines
services/pdf_service.py	291	services/measurement_service.py	164
tests/test_processing.py	193	routes/upload_routes.py	169
services/detection_service.py	130	static/app.js	113
routes/analysis_routes.py	98	tests/test_app.py	67
services/csv_service.py	57	models/analysis_model.py	56
app.py	54	services/zip_service.py	50

Table 9.5: Larger source files (line counts); the repository has about 1,990 lines in total

F.1 Full source listings

The files below are reproduced from the repository so that the report is self-contained. Core algorithm files are shown as screenshots in Section 6.11; the test files, report generator, templates, style sheet and configuration files are listed here as text. Long single-line template files are wrapped.

F.1 — services/pdf_service.py

```
  1  from collections import Counter
  2  from datetime import datetime, timezone
  3  from pathlib import Path
  4  from xml.sax.saxutils import escape
  5
  6  from reportlab.lib import colors
  7  from reportlab.lib.enums import TA_CENTER
  8  from reportlab.lib.pagesizes import A4
  9  from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
 10  from reportlab.lib.units import inch
 11  from reportlab.platypus import (
 12      Image,
 13      PageBreak,
 14      Paragraph,
 15      SimpleDocTemplate,
 16      Spacer,
 17      Table,
 18      TableStyle,
 19      KeepTogether,
 20  )
 21  from reportlab.lib.utils import ImageReader
 22
 23
 24  WORKFLOW_STEPS = [
```

```
    ("1", "Upload and validate", "Check file type, size, and image readability.", "Validated source
        image"),
    ("2", "Resize image", "Fit the image to the configured processing dimension.", "Resized image"),
    ("3", "Preprocess", "Create grayscale, blur, CLAHE, and BlackHat images.", "Preprocessing images"),
    ("4", "Segment crack candidates", "Apply thresholding, morphology, and connected components.",
        "Binary and segmentation masks"),
    ("5", "Measure cracks", "Skeletonize regions and estimate area, length, and width.", "Skeleton,
        distance transform, width estimates"),
    ("6", "Calibrate and classify", "Convert pixels to millimeters and assign severity.", "px/mm
        measurements and severity"),
    ("7", "Annotate and export", "Draw measurement lines and prepare report files.", "Annotated image,
        CSV, PDF, ZIP"),
]


def _text(value):
    return escape(str(value if value is not None else ""))


def _number(value, unit=""):
    if value is None:
        return "Not calibrated" if unit == "mm" else "-"
    return f"{float(value):.2f} {unit}".strip()


def _date_time(value):
    if hasattr(value, "astimezone"):
        local_value = value.astimezone()
        return local_value.strftime("%Y-%m-%d %H:%M:%S %z")
    if value:
        return str(value)
    return "Not available"


def _image(path, max_width=6.4 * inch, max_height=3.8 * inch):
    """Create an aspect-preserving ReportLab image."""
    reader = ImageReader(str(path))
    width, height = reader.getSize()
    scale = min(max_width / width, max_height / height)
    image = Image(str(path), width=width * scale, height=height * scale)
    image.hAlign = "LEFT"
    return image


def _table(rows, widths, header=True, font_size=8):
    style = [
        ("GRID", (0, 0), (-1, -1), 0.35, colors.HexColor("#94a3b8")),
        ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
        ("FONTSIZE", (0, 0), (-1, -1), font_size),
        ("LEFTPADDING", (0, 0), (-1, -1), 5),
        ("RIGHTPADDING", (0, 0), (-1, -1), 5),
        ("TOPPADDING", (0, 0), (-1, -1), 5),
        ("BOTTOMPADDING", (0, 0), (-1, -1), 5),
    ]
    if header:
        style.extend([
            ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#dbeafe")),
            ("TEXTCOLOR", (0, 0), (-1, 0), colors.HexColor("#172033")),
            ("FONTNAME", (0, 0), (-1, 0), "Helvetica-Bold"),
        ])
    return Table(rows, colWidths=widths, repeatRows=1 if header else 0,
        style=TableStyle(style))


def _footer(canvas, document):
    canvas.saveState()
    canvas.setStrokeColor(colors.HexColor("#cbd5e1"))
    canvas.line(36, 28, A4[0] - 36, 28)
    canvas.setFont("Helvetica", 8)
    canvas.setFillColor(colors.HexColor("#64748b"))
    canvas.drawString(36, 16, "Crack Detaction - image-based crack measurement report")
    canvas.drawRightString(A4[0] - 36, 16, f"Page {document.page}")
    canvas.restoreState()


```

```
 99  def create_pdf(analysis, images, measurements, output_path):
100      Path(output_path).parent.mkdir(parents=True, exist_ok=True)
101      doc = SimpleDocTemplate(
102          str(output_path),
103          pagesize=A4,
104          rightMargin=36,
105          leftMargin=36,
106          topMargin=42,
107          bottomMargin=42,
108          title=f"Crack Detaction report - {analysis.get('analysis_name', '')}",
109      )
110      styles = getSampleStyleSheet()
111      styles.add(ParagraphStyle(name="ReportTitle", parent=styles["Title"],
alignment=TA_CENTER,
112      fontSize=20, leading=24, spaceAfter=10, textColor=colors.HexColor("#123b66")))
113      styles.add(ParagraphStyle(name="Small", parent=styles["BodyText"], fontSize=8,
leading=10,
114      spaceAfter=6, textColor=colors.HexColor("#475569")))
115      styles.add(ParagraphStyle(name="Step", parent=styles["BodyText"], fontSize=9,
leading=12,
116      spaceAfter=3))
117      styles["Heading2"].spaceBefore = 12
118      styles["Heading2"].spaceAfter = 8
119      styles["Heading3"].spaceBefore = 10
120      styles["Heading3"].spaceAfter = 6
121      generated_at = datetime.now(timezone.utc)
122      story = [
123          Paragraph("Crack Detaction Measurement Report", styles["ReportTitle"]),
124          Spacer(1, 6),
125          Paragraph(f"<b>Analysis:</b> {_text(analysis.get('analysis_name', 'Untitled
analysis'))}",
126      styles["Heading2"]),
127          Paragraph(f"<b>Analysis ID:</b> {_text(analysis.get('analysis_id', ''))}
  
128      <b>Status:</b> {_text(analysis.get('status', ''))}", styles["Small"]),
129          Paragraph(f"<b>Analysis date and time:</b>
{_text(_date_time(analysis.get('created_at')))}",
130      styles["Small"]),
131          Paragraph(f"<b>Report generated date and time:</b>
{_text(_date_time(generated_at))}",
132      styles["Small"]),
133          Paragraph("This report contains automated image-processing results. It is not a
certified
134      structural safety assessment.", styles["Small"]),
135          Spacer(1, 14),
136      ]
137
138      counts = Counter(item.get("severity", "Unknown") for item in measurements)
139      total = len(measurements)
140      calibrated = bool(analysis.get("calibration_enabled") and
analysis.get("pixels_per_mm"))
141      pixels_per_mm = analysis.get("pixels_per_mm")
142      total_length_px = sum(item.get("skeleton_length_px", 0) for item in measurements)
143      total_length_mm = sum(item.get("skeleton_length_mm") or 0 for item in measurements)
if calibrated
144      else None
145      average_width_px = sum(item.get("mean_width_px", 0) for item in measurements) / total
if total else 0
146      average_width_mm = sum(item.get("mean_width_mm") or 0 for item in measurements) /
total if total and
147      calibrated else None
148      maximum_width_px = max((item.get("max_width_px", 0) for item in measurements),
default=0)
149      maximum_width_mm = max((item.get("max_width_mm") or 0 for item in measurements),
default=0) if
150      calibrated else None
151      overall = "Heavy" if counts["Heavy"] else "Medium" if counts["Medium"] else "Good" if
total else "No
152      data"
153
154      summary_rows = [
155          ["Report item", "Result"],
156          ["Images processed", str(len(images))],
157          ["Detected crack regions", str(total)],
158          ["Total crack length", f"{_number(total_length_px, 'px')} /
{_number(total_length_mm, 'mm')}"],
159          ["Average crack width", f"{_number(average_width_px, 'px')} /
{_number(average_width_mm,
160      'mm')}"],
161          ["Maximum crack width", f"{_number(maximum_width_px, 'px')} /
{_number(maximum_width_mm,
162      'mm')}"],
```

```
 163          ["Overall severity", f"{overall} (Good {counts['Good']}, Medium
{counts['Medium']}, Heavy
 164      {counts['Heavy']})"],
 165          ["Calibration", f"{_number(pixels_per_mm, 'pixels/mm')}" if calibrated else
"Pixel-only;
 166      millimeters not calibrated"],
 167      ]
 168      story.extend([
 169          KeepTogether([
 170              Paragraph("1. Crack analysis summary", styles["Heading2"]),
 171              Paragraph("This section provides the key results before the detailed workflow
and evidence.",
 172      styles["Small"]),
 173              Spacer(1, 10),
 174              _table(summary_rows, [2.3 * inch, 4.0 * inch]),
 175          ]),
 176          Spacer(1, 14),
 177      ])
 178
 179      thresholds = analysis.get("severity_thresholds", {"good_max_width_px": 3,
"medium_max_width_px": 8,
 180      "heavy_min_area_px": 5000})
 181      threshold_rows = [["Category", "Pixel rule", "Millimeter rule"]]
 182      if calibrated:
 183          threshold_rows.extend([
 184              ["Good", f"Mean width <= {thresholds['good_max_width_px']} px", f"Mean width
<=
 185      {thresholds['good_max_width_px'] / pixels_per_mm:.3f} mm"],
 186              ["Medium", f"> {thresholds['good_max_width_px']} and <=
{thresholds['medium_max_width_px']}
 187      px", f"> {thresholds['good_max_width_px'] / pixels_per_mm:.3f} and <=
 188      {thresholds['medium_max_width_px'] / pixels_per_mm:.3f} mm"],
 189              ["Heavy", f"> {thresholds['medium_max_width_px']} px or area >=
 190      {thresholds['heavy_min_area_px']} px^2", f"> {thresholds['medium_max_width_px'] /
pixels_per_mm:.3f}
 191      mm or area >= {thresholds['heavy_min_area_px'] / pixels_per_mm**2:.3f} mm^2"],
 192          ])
 193      else:
 194          threshold_rows.extend([
 195              ["Good", f"Mean width <= {thresholds['good_max_width_px']} px", "Not
calibrated"],
 196              ["Medium", f"> {thresholds['good_max_width_px']} and <=
{thresholds['medium_max_width_px']}
 197      px", "Not calibrated"],
 198              ["Heavy", f"> {thresholds['medium_max_width_px']} px or area >=
 199      {thresholds['heavy_min_area_px']} px^2", "Not calibrated"],
 200          ])
 201      story.extend([
 202          KeepTogether([
 203              Paragraph("2. Calibration and severity thresholds", styles["Heading2"]),
 204              Spacer(1, 10),
 205              _table(threshold_rows, [1.0 * inch, 2.8 * inch, 2.5 * inch]),
 206          ]),
 207          Spacer(1, 14),
 208          PageBreak(),
 209      ])
 210
 211      workflow_rows = [["Step", "Stage", "Action", "Output"]]
 212      workflow_rows.extend([
 213          [number, Paragraph(f"<b>{_text(title)}</b>", styles["Step"]),
Paragraph(_text(description),
 214      styles["Step"]), Paragraph(_text(output), styles["Step"])]
 215          for number, title, description, output in WORKFLOW_STEPS
 216      ])
 217      story.extend([
 218          KeepTogether([
 219              Paragraph("3. Processing workflow - step by step", styles["Heading2"]),
 220              Paragraph("The following numbered stages are executed in order for every
uploaded image. The
 221      output column identifies the evidence included later in this report.",
styles["Small"]),
 222              Spacer(1, 10),
 223          ]),
 224          _table(workflow_rows, [.45 * inch, 1.45 * inch, 2.55 * inch, 1.95 * inch],
font_size=8),
 225          Spacer(1, 12),
 226      ])
 227      timing_rows = [["Image", "Stage", "Time (s)"]]
 228      for image in images:
 229          for stage, seconds in image.get("workflow_times", {}).items():
 230              timing_rows.append([_text(image.get("original_filename", "")),
stage.replace("_", "
```

```
231       ").title(), f"{float(seconds):.4f}"])
232       story.append(Paragraph("Workflow timings", styles["Heading3"]))
233       story.append(Paragraph("Times are measured by the application and are reported
separately for each
234       uploaded image.", styles["Small"]))
235       if len(timing_rows) > 1:
236           story.extend([Spacer(1, 8), _table(timing_rows, [2.8 * inch, 2.3 * inch, 1.2 *
inch])])
237       story.append(PageBreak())
238       story.append(KeepTogether([
239           Paragraph("4. Crack measurements", styles["Heading2"]),
240           Paragraph("Every value is shown as pixels and millimeters where calibration is
available.",
241       styles["Small"]),
242           Spacer(1, 10),
243       ]))
244       measurement_rows = [["Image / crack", "Area px^2", "Area mm^2", "Length px", "Length
mm",
245       "Severity"]]
246       for item in measurements:
247           measurement_rows.append([
248               f"{_text(item.get('image_id', ''))[:12]} / {_text(item.get('crack_id',
'')[)}",
249               _number(item.get("contour_area_px"), "px^2"),
250               _number(item.get("contour_area_mm2"), "mm^2"),
251               _number(item.get("skeleton_length_px"), "px"),
252               _number(item.get("skeleton_length_mm"), "mm"),
253               _text(item.get("severity", "")),
254           ])
255       if len(measurement_rows) == 1:
256           measurement_rows.append(["No candidate crack regions", "-", "-", "-", "-", "-"])
257       story.append(_table(measurement_rows, [1.55 * inch, .85 * inch, .85 * inch, .85 *
inch, .85 * inch,
258       .95 * inch], font_size=7))
259       story.append(Spacer(1, 10))
260       width_rows = [["Crack", "Mean width px", "Mean width mm", "Max width px", "Max width
mm", "Samples"]]
261       for item in measurements:
262           width_rows.append([
263               _text(item.get("crack_id", "")),
264               _number(item.get("mean_width_px"), "px"),
265               _number(item.get("mean_width_mm"), "mm"),
266               _number(item.get("max_width_px"), "px"),
267               _number(item.get("max_width_mm"), "mm"),
268               str(item.get("width_sample_count", 0)),
269           ])
270       if len(width_rows) == 1:
271           width_rows.append(["No measurements", "-", "-", "-", "-", "-"])
272       story.extend([Spacer(1, 10), _table(width_rows, [1.55 * inch, 1.05 * inch, 1.05 *
inch, 1.05 * inch,
273       1.05 * inch, .65 * inch], font_size=7)])
274       story.append(PageBreak())
275       story.append(KeepTogether([
276           Paragraph("5. Image-processing evidence", styles["Heading2"]),
277           Paragraph("Each processing stage is shown in a readable flow below. The stage
name and source
278       file are kept with its image so the evidence remains understandable when printed or
extracted from
279       the ZIP package.", styles["Small"]),
280       ]))
281       for image_index, image in enumerate(images, 1):
282           story.append(Paragraph(_text(image.get("original_filename", "Image")),
styles["Heading3"]))
283           stage_order = [
284               ("resized", "Resized input"),
285               ("gray", "Grayscale"),
286               ("blurred", "Gaussian blur"),
287               ("clahe", "CLAHE"),
288               ("blackhat", "BlackHat"),
289               ("binary", "Otsu threshold"),
290               ("cleaned_mask", "Morphological cleanup"),
291               ("segmentation_mask", "Segmentation mask"),
292           ]
293           stage_order.extend(
294               (stage, stage.replace("_", " ").title())
295               for stage in image.get("processed_paths", {})
296               if stage.startswith(("distance_transform_", "width_estimation_",
"skeleton_"))
297           )
298           evidence_rows = []
299           for stage_index, (stage, title) in enumerate(stage_order, 1):
300               path = image.get("processed_paths", {}).get(stage)
```

```
            if path and Path(path).exists():
                if stage_index > 1 or image_index > 1:
                    story.append(PageBreak())
                story.extend([
                    Paragraph(f"Evidence {image_index}.{stage_index}: {_text(title)}",
    styles["Heading3"]),
                    Paragraph(f"<font color='#64748b'>File:
{_text(Path(path).name)}</font>",
    styles["Small"]),
                    Spacer(1, 6),
                    _image(path, 6.4 * inch, 5.0 * inch),
                    Spacer(1, 14),
                ])
        annotated = image.get("annotated_path")
        if annotated and Path(annotated).exists():
            story.extend([Spacer(1, 10), Paragraph("Annotated result with measurement
lines",
    styles["Heading3"]), _image(annotated)])
        story.append(PageBreak())

    story.append(Paragraph("6. Assessment notes", styles["Heading2"]))
    if counts["Heavy"]:
        remark = f"Heavy-severity indications were detected in {counts['Heavy']} of
{total} crack
    region(s). Review the annotated evidence and arrange a qualified structural
inspection."
    elif counts["Medium"]:
        remark = f"Medium-severity indications were detected in {counts['Medium']} of
{total} crack
    region(s). Monitor the affected area and consider a qualified inspection."
    elif total:
        remark = f"All {total} detected crack region(s) are in the Good software
category. Continue
    routine monitoring."
    else:
        remark = "No crack measurements were produced. Upload a clearer crack image for
another
    analysis."
    story.extend([
        Paragraph(remark, styles["BodyText"]),
        Spacer(1, 8),
        Paragraph("The results depend on lighting, image resolution, texture, resizing,
segmentation
    thresholds, and calibration accuracy. Use this report as an inspection aid, not as a
standalone
    engineering conclusion.", styles["BodyText"]),
    ])
    doc.build(story, onFirstPage=_footer, onLaterPages=_footer)
    return output_path
```

F.1 — routes/history_routes.py

```
from flask import Blueprint, jsonify, redirect, render_template, request, url_for

from models.analysis_model import delete_analysis_records, list_analyses
from models.database import get_db
from services.storage_service import remove_analysis
from flask import current_app

history_bp = Blueprint("history", __name__)


@history_bp.get("/history")
def history():
    return render_template("history.html", analyses=list_analyses(limit=100))


@history_bp.get("/api/history")
def history_api():
    query = {}
    search = request.args.get("q")
    if search:
        query = {"analysis_name": {"$regex": search, "$options": "i"}}
    return jsonify({"ok": True, "analyses": [{k: v for k, v in item.items() if k !=
"_id"} for item in
    list_analyses(query, limit=100)]})


@history_bp.delete("/api/analysis/<analysis_id>")
```

```
def delete(analysis_id):
    delete_analysis_records(analysis_id)
    remove_analysis(current_app.config["STORAGE_DIR"], analysis_id)
    return jsonify({"ok": True})
```

F.1 — routes/report_routes.py

```
import uuid
from pathlib import Path

from flask import Blueprint, current_app, jsonify, render_template, send_file

from models.analysis_model import get_analysis
from models.image_model import list_images
from models.measurement_model import list_measurements
from services.csv_service import create_csv
from services.pdf_service import create_pdf
from services.storage_service import analysis_dir
from services.zip_service import create_manifest, create_zip

report_bp = Blueprint("reports", __name__)


@report_bp.get("/reports")
def reports():
    return render_template("reports.html")


def build_report(analysis_id, report_type):
    analysis = get_analysis(analysis_id)
    if not analysis:
        return None
    images = list_images(analysis_id)
    measurements = list_measurements(analysis_id)
    root = analysis_dir(current_app.config["STORAGE_DIR"], analysis_id)
    report_dir = root / "reports"
    report_dir.mkdir(exist_ok=True)
    if report_type == "csv":
        path = report_dir / f"{analysis_id}.csv"
        return create_csv(analysis, images, measurements, path)
    if report_type == "pdf":
        path = report_dir / f"{analysis_id}.pdf"
        return create_pdf(analysis, images, measurements, path)
    csv_path = build_report(analysis_id, "csv")
    pdf_path = build_report(analysis_id, "pdf")
    manifest_path = create_manifest(analysis, images, measurements, report_dir /
    f"{analysis_id}_manifest.txt")
    return create_zip(analysis_id, root, [csv_path, pdf_path, manifest_path], report_dir
/
    f"{analysis_id}.zip")


@report_bp.get("/reports/<analysis_id>/<report_type>")
def download_report(analysis_id, report_type):
    if report_type not in {"csv", "pdf", "zip"}:
        return jsonify({"ok": False, "error": "Unknown report type."}), 400
    path = build_report(analysis_id, report_type)
    return send_file(path, as_attachment=True, download_name=Path(path).name) if path
else ("Analysis not
    found", 404)
```

F.1 — services/storage_service.py

```
import shutil
from pathlib import Path


def analysis_dir(storage_dir, analysis_id):
    root = Path(storage_dir)
    path = root / "analyses" / analysis_id
    path.mkdir(parents=True, exist_ok=True)
    return path


def remove_analysis(storage_dir, analysis_id):
    path = Path(storage_dir) / "analyses" / analysis_id
```

```
    if path.exists():
        shutil.rmtree(path)
```

F.1 — tests/test_app.py

```
from io import BytesIO

import cv2
import numpy as np
from zipfile import ZipFile

from app import create_app
from services.zip_service import create_zip


def test_create_zip_excludes_itself_and_duplicate_reports(tmp_path):
    root = tmp_path / "analysis"
    reports = root / "reports"
    reports.mkdir(parents=True)
    image = root / "processed.png"
    csv_path = reports / "analysis.csv"
    pdf_path = reports / "analysis.pdf"
    zip_path = reports / "analysis.zip"
    image.write_bytes(b"image")
    csv_path.write_bytes(b"csv")
    pdf_path.write_bytes(b"pdf")

    create_zip("analysis", root, [csv_path, pdf_path], zip_path)

    with ZipFile(zip_path) as archive:
        names = archive.namelist()
    assert "Crack_Analysis_Results/reports/analysis.zip" not in names
    assert names.count("Crack_Analysis_Results/reports/analysis.csv") == 1
    assert names.count("Crack_Analysis_Results/reports/analysis.pdf") == 1


def test_upload_rejects_empty_batch(tmp_path):
    class TestConfig:
        TESTING = True
        SECRET_KEY = "test"
        STORAGE_DIR = tmp_path
        MAX_CONTENT_LENGTH = 1024 * 1024
        ALLOWED_EXTENSIONS = {"jpg", "jpeg", "png"}
        MAX_FILES_PER_ANALYSIS = 20
        MAX_IMAGE_DIMENSION = 8000
        MONGO_URI = "mongodb://invalid-host:27017"
        MONGO_DB_NAME = "test"
        PROCESSING = {"resize_max_dimension": 1600, "gaussian_kernel": 5,
"clahe_clip_limit": 2,
                      "clahe_tile_grid": 8, "blackhat_kernel": 21, "morphology_kernel":
3,
                      "morphology_iterations": 1, "minimum_region_area": 10}
    client = create_app(TestConfig).test_client()
    response = client.post("/api/analyze", data={})
    assert response.status_code == 400


def test_upload_rejects_invalid_extension(tmp_path):
    class TestConfig:
        TESTING = True
        SECRET_KEY = "test"
        STORAGE_DIR = tmp_path
        MAX_CONTENT_LENGTH = 1024 * 1024
        ALLOWED_EXTENSIONS = {"jpg", "jpeg", "png"}
        MAX_FILES_PER_ANALYSIS = 20
        MAX_IMAGE_DIMENSION = 8000
        MONGO_URI = "mongodb://invalid-host:27017"
        MONGO_DB_NAME = "test"
        PROCESSING = {"resize_max_dimension": 1600, "gaussian_kernel": 5,
"clahe_clip_limit": 2,
                      "clahe_tile_grid": 8, "blackhat_kernel": 21, "morphology_kernel":
3,
                      "morphology_iterations": 1, "minimum_region_area": 10}
    client = create_app(TestConfig).test_client()
    response = client.post("/api/analyze", data={"images": (BytesIO(b"not image"),
"x.txt")},
    content_type="multipart/form-data")
    assert response.status_code == 400
```

F.1 — tests/test_processing.py

```
import cv2
import numpy as np

from services.calibration_service import convert_pixels, validate_pixels_per_mm
from services.classification_service import classify
from services.detection_service import detect_components, filter_crack_components,
validate_crack_image
from services.measurement_service import measure_component
from services.preprocessing_service import preprocess


CONFIG = {"gaussian_kernel": 5, "clahe_clip_limit": 2, "clahe_tile_grid": 8,
"blackhat_kernel": 21,
          "morphology_kernel": 3, "morphology_iterations": 1, "minimum_region_area": 10}


def test_pipeline_detects_synthetic_dark_crack():
    image = np.full((220, 220, 3), 210, dtype=np.uint8)
    cv2.line(image, (25, 30), (190, 180), (20, 20, 20), 5)
    stages = preprocess(image, CONFIG)
    _, _, components = detect_components(stages["binary"], CONFIG)
    assert components
    assert filter_crack_components(components, CONFIG)
    assert validate_crack_image(filter_crack_components(components, CONFIG), image.shape,
CONFIG)
    result = measure_component(components[0], 10)
    assert result["skeleton_length_px"] > 0
    assert result["skeleton_length_mm"] is not None


def test_calibration_and_classification():
    assert validate_pixels_per_mm("4") == 4
    assert validate_pixels_per_mm("0") is None
    assert convert_pixels(20, 4) == 5
    assert classify(10, 1) == "Good"
    assert classify(10, 4) == "Medium"
    assert classify(6000, 1) == "Heavy"


def test_validation_rejects_round_non_crack_region():
    image = np.full((220, 220, 3), 210, dtype=np.uint8)
    cv2.circle(image, (110, 110), 35, (20, 20, 20), -1)
    stages = preprocess(image, CONFIG)
    _, _, components = detect_components(stages["binary"], CONFIG)
    assert not filter_crack_components(components, CONFIG)


def test_validation_rejects_text_heavy_image():
    image = np.full((220, 220, 3), 210, dtype=np.uint8)
    for y in range(20, 210, 20):
        cv2.putText(image, "TEXT", (10, y), cv2.FONT_HERSHEY_SIMPLEX, 0.45, (20, 20, 20),
1)
    stages = preprocess(image, CONFIG)
    _, _, components = detect_components(stages["binary"], CONFIG)
    filtered = filter_crack_components(components, CONFIG)
    assert not validate_crack_image(filtered, image.shape, CONFIG)


def test_validation_allows_fragmented_dominant_crack():
    image = np.full((220, 220, 3), 210, dtype=np.uint8)
    for start, end in [((15, 110), (70, 100)), ((75, 98), (130, 80)), ((135, 78), (205,
45))]:
        cv2.line(image, start, end, (20, 20, 20), 4)
    stages = preprocess(image, {**CONFIG, "maximum_crack_components": 1,
"maximum_crack_coverage":
    0.001})
    _, _, components = detect_components(stages["binary"], {**CONFIG,
"maximum_crack_components": 1})
    filtered = filter_crack_components(components, CONFIG)
    assert validate_crack_image(filtered, image.shape, {**CONFIG,
"maximum_crack_components": 1,
    "maximum_crack_coverage": 0.001})


def test_validation_allows_dominant_crack_with_texture_fragments():
    image = np.full((220, 220, 3), 150, dtype=np.uint8)
```

```
    cv2.line(image, (20, 112), (200, 92), (80, 80, 80), 4)
    for x, y in ((30, 30), (75, 180), (130, 45), (180, 165)):
        cv2.line(image, (x, y), (x + 8, y + 2), (110, 110, 110), 2)

    stages = preprocess(image, CONFIG)
    _, _, components = detect_components(stages["binary"], CONFIG)
    filtered = filter_crack_components(components, CONFIG)

    assert filtered
    assert validate_crack_image(filtered, image.shape, CONFIG)


def test_filter_keeps_coherent_crack_and_discards_texture_blobs():
    image = np.full((220, 220, 3), 150, dtype=np.uint8)
    for start, end in [((15, 112), (70, 105)), ((74, 104), (130, 94)), ((134, 93), (205,
78))]:
        cv2.line(image, start, end, (80, 80, 80), 5)
    for x, y in ((30, 30), (110, 40), (180, 170), (40, 180), (190, 45)):
        cv2.line(image, (x, y), (x + 7, y + 2), (80, 80, 80), 3)

    stages = preprocess(image, CONFIG)
    _, _, components = detect_components(stages["binary"], CONFIG)
    filtered = filter_crack_components(components, {**CONFIG, "crack_link_distance_px":
18})

    assert 1 <= len(filtered) <= 4
    assert validate_crack_image(filtered, image.shape, CONFIG)


def test_percentile_threshold_limits_road_texture_noise():
    image = np.full((320, 480, 3), 120, dtype=np.uint8)
    noise = np.random.default_rng(7).normal(0, 8, image.shape[:2]).clip(-30,
30).astype(np.int16)
    textured = np.clip(image[:, :, 0].astype(np.int16) + noise, 0, 255).astype(np.uint8)
    image[:] = cv2.cvtColor(textured, cv2.COLOR_GRAY2BGR)
    cv2.polylines(image, [np.array([[10, 190], [100, 170], [190, 205], [300, 185], [470,
195]])], False,
    (55, 55, 55), 3)

    stages = preprocess(image, {**CONFIG, "blackhat_threshold_percentile": 90})
    _, _, components = detect_components(stages["binary"], CONFIG)
    filtered = filter_crack_components(components, CONFIG)

    assert stages["binary"].mean() < 80
    assert filtered
    assert validate_crack_image(filtered, image.shape, CONFIG)


def test_adaptive_threshold_measures_full_normal_road_crack():
    image = np.full((320, 480, 3), 120, dtype=np.uint8)
    noise = np.random.default_rng(12).normal(0, 6, image.shape[:2]).clip(-25,
25).astype(np.int16)
    textured = np.clip(image[:, :, 0].astype(np.int16) + noise, 0, 255).astype(np.uint8)
    image[:] = cv2.cvtColor(textured, cv2.COLOR_GRAY2BGR)
    cv2.polylines(
        image,
        [np.array([[5, 170], [100, 155], [200, 190], [310, 175], [475, 185]])],
        False,
        (55, 55, 55),
        2,
    )

    config = {**CONFIG, "adaptive_threshold_block_size": 41, "adaptive_threshold_offset":
8}
    stages = preprocess(image, config)
    _, _, components = detect_components(stages["binary"], config)
    filtered = filter_crack_components(components, config)
    measurements = [measure_component(component, 10) for component in filtered]

    assert filtered
    assert max(cv2.boundingRect(component["contour"])[0] +
cv2.boundingRect(component["contour"])[2] for
    component in filtered) > 400
    assert sum(item["skeleton_length_px"] for item in measurements) < 800


def test_width_measurement_uses_stable_centerline_statistics():
    image = np.full((220, 220, 3), 210, dtype=np.uint8)
    cv2.line(image, (25, 110), (195, 110), (20, 20, 20), 4)
    stages = preprocess(image, CONFIG)
    _, _, components = detect_components(stages["binary"], CONFIG)

```

```
144      measurement = measure_component(components[0], 10)
145
146      assert 5 <= measurement["mean_width_px"] <= 7
147      assert measurement["max_width_px"] >= measurement["mean_width_px"]
148      assert measurement["max_width_px"] < 8
149
150
151  def test_multiple_separate_crack_networks_are_measured_independently():
152      image = np.full((360, 480, 3), 150, dtype=np.uint8)
153      cv2.polylines(image, [np.array([[20, 70], [90, 95], [160, 55]])], False, (55, 55,
55), 4)
154      cv2.polylines(image, [np.array([[300, 80], [360, 130], [450, 105]])], False, (55, 55,
55), 4)
155      cv2.polylines(image, [np.array([[30, 280], [110, 245], [180, 300]])], False, (55, 55,
55), 4)
156
157      config = {**CONFIG, "adaptive_threshold_block_size": 41, "adaptive_threshold_offset":
16}
158      stages = preprocess(image, config)
159      _, _, components = detect_components(stages["binary"], config)
160      filtered = filter_crack_components(components, config)
161      measurements = [measure_component(component, 10) for component in filtered]
162
163      assert len(filtered) >= 3
164      assert all(item["skeleton_length_px"] > 30 for item in measurements)
165      assert validate_crack_image(filtered, image.shape, config)
166
167
168  def test_validation_allows_wide_dominant_road_crack():
169      image = np.full((384, 544, 3), 150, dtype=np.uint8)
170      cv2.line(image, (80, 310), (160, 280), (80, 80, 80), 8)
171      stages = preprocess(image, CONFIG)
172      _, _, components = detect_components(stages["binary"], CONFIG)
173      filtered = filter_crack_components(components, CONFIG)
174
175      assert validate_crack_image(
176          filtered,
177          image.shape,
178          {
179              **CONFIG,
180              "minimum_dominant_crack_area_ratio": 0.003,
181              "minimum_dominant_crack_aspect_ratio": 2.4,
182          },
183      )
184
185
186  def test_validation_rejects_dispersed_portrait_like_regions():
187      image = np.full((220, 220, 3), 210, dtype=np.uint8)
188      for x, y in ((30, 30), (110, 35), (185, 45), (40, 150), (120, 140), (195, 165)):
189          cv2.line(image, (x, y), (x + 15, y + 4), (20, 20, 20), 4)
190      stages = preprocess(image, CONFIG)
191      _, _, components = detect_components(stages["binary"], CONFIG)
192      filtered = filter_crack_components(components, CONFIG)
193      assert not validate_crack_image(
194          filtered,
195          image.shape,
196          {**CONFIG, "maximum_crack_components": 1, "maximum_crack_coverage": 0.001},
197      )
```

F.1 — templates/base.html

```
  1  <!doctype html>
  2  <html lang="en">
  3  <head>
  4    <meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-
scale=1">
  5    <title>{% block title %}Crack Detaction{% endblock %}</title>
  6    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
  7    <link rel="stylesheet" href="{{ url_for('static', filename='app.css') }}">
  8  </head>
  9  <body>
 10  <nav class="navbar navbar-dark bg-primary"><div class="container"><a class="navbar-brand
fw-bold"
 11      href="/upload">Crack Detaction</a></div></nav>
 12  <main class="container py-4">{% block content %}{% endblock %}</main>
 13  <div class="image-lightbox" id="image-lightbox" hidden role="dialog" aria-modal="true"
aria-label="Image
 14      preview">
```

```
    <button class="image-lightbox-close" type="button" aria-label="Close image preview">&times;</button>
    <figure class="image-lightbox-content">
      <div class="image-lightbox-toolbar" role="toolbar" aria-label="Image zoom controls">
        <button type="button" data-image-zoom="out" aria-label="Zoom out">-</button>
        <button type="button" data-image-zoom="reset">Reset</button>
        <button type="button" data-image-zoom="in" aria-label="Zoom in">+</button>
        <span id="image-lightbox-zoom">100%</span>
      </div>
      <img id="image-lightbox-image" alt="">
      <figcaption id="image-lightbox-caption"></figcaption>
    </figure>
  </div>
  <script src="{{ url_for('static', filename='app.js') }}"></script>
  </body></html>
```

F.1 — templates/upload.html

```
  {% extends "base.html" %}{% block title %}Upload - Crack Detaction{% endblock %}{% block content %}
  <h1>Upload crack images</h1>
  <p class="text-muted">Select one or more images containing visible cracks. Detection and measurement
      start automatically after upload.</p>
  <form id="analysis-form" class="card card-body" enctype="multipart/form-data">
    <label class="dropzone mb-3" for="images">
      <input id="images" name="images" type="file" accept=".jpg,.jpeg,.jfif,.png" multiple>
      <span>Drop crack images here or click to browse</span>
    </label>
    <div id="preview" class="row g-2 mb-3"></div>
    <p class="small text-muted mb-3">Measurements are calibrated automatically using the configured image
      scale.</p>
    <button id="upload-button" class="btn btn-primary" type="submit">Upload and detect cracks</button>
    <div id="upload-status" class="mt-3" role="alert" aria-live="polite"></div>
  </form>
  {% endblock %}
```

F.1 — templates/history.html

```
  {% extends "base.html" %}{% block title %}History - Crack Detaction{% endblock %}{% block content
      %}<h1>Analysis history</h1><div class="card card-body"><div class="table-responsive"><table class="ta
  3
ble"><tr><th>Name</th><th>Created</th><th>Images</th><th>Regions</th><th>Status</th><th></th></tr>{%
      for item in analyses %}<tr><td>{{ item.analysis_name }}</td><td>{{ item.created_at }}</td><td>{{
      item.total_images }}</td><td>{{ item.total_detected_regions }}</td><td>{{ item.status }}</td><td><a
      href="/analysis/{{ item.analysis_id }}">View</a></td></tr>{% else %}<tr><td colspan="6">No saved
      analyses.</td></tr>{% endfor %}</table></div></div>{% endblock %}
```

F.1 — templates/reports.html

```
  {% extends "base.html" %}{% block title %}Reports - Crack Detaction{% endblock %}{% block content
      %}<h1>Reports</h1><p class="text-muted">Open an analysis from history to generate CSV, PDF, or ZIP
      outputs.</p><a href="/history" class="btn btn-primary">Choose analysis</a>{% endblock %}
```

F.1 — templates/analysis_not_found.html

```
  {% extends "base.html" %}{% block title %}Analysis unavailable - Crack Detaction{% endblock %}{% block
      content %}
  <div class="card card-body text-center py-5">
```

```
    <h1 class="h3">Analysis output is unavailable</h1>
    <p class="text-muted">This analysis record is no longer available in the current
application session.
      Upload the images again, or configure MongoDB so results persist after restarting
Flask.</p>
    <div><a class="btn btn-primary" href="/upload">Upload images again</a> <a class="btn
btn-outline-
      secondary" href="/history">Open history</a></div>
  </div>
{% endblock %}
```

F.1 — templates/results.html

```
{% extends "base.html" %}{% block title %}Results - Crack Detaction{% endblock %}{% block
content %}
  <div class="d-flex justify-content-between align-items-start mb-4"><div><h1>{{
analysis.analysis_name
      }}</h1><p class="text-muted mb-1">ID: {{ analysis.analysis_id }} · {{ analysis.status
}}</p><p
      class="small">Automatic calibration: {{ "%.4f"|format(analysis.pixels_per_mm) }}
      pixels/mm</p></div><a class="btn btn-primary" href="/upload">Retry image
upload</a></div>
  <div class="card card-body mb-4"><h2 class="h5">Crack analysis summary</h2><div
class="row g-3"><div
      class="col-md-3"><small class="text-muted">Total cracks</small><div class="h4 mb-
0">{{
      summary.total_cracks }}</div></div><div class="col-md-3"><small class="text-
muted">Total crack
      length</small><div class="h4 mb-0">{{ "%.2f"|format(summary.total_length_px) }}
px</div><div>{{
      "%.2f"|format(summary.total_length_mm) }} mm</div></div><div class="col-md-3"><small
class="text-
      muted">Average width</small><div class="h4 mb-0">{{
"%.2f"|format(summary.average_width_px) }}
      px</div><div>{{ "%.2f"|format(summary.average_width_mm) }} mm</div></div><div
class="col-md-3"><small
      class="text-muted">Maximum width</small><div class="h4 mb-0">{{
      "%.2f"|format(summary.maximum_width_px) }} px</div><div>{{
"%.2f"|format(summary.maximum_width_mm) }}
      mm</div></div></div><div class="row g-3 mt-1"><div class="col-md-4"><small
class="text-muted">Minimum
      width</small><div>{{ "%.2f"|format(summary.minimum_width_px) }} px / {{
      "%.2f"|format(summary.minimum_width_mm) }} mm</div></div><div class="col-md-4"><small
class="text-
      muted">Overall severity</small><div class="h5 mb-0">{{ summary.overall_severity
      }}</div></div></div><hr><div class="row g-3"><div class="col-md-4"><span class="badge
text-bg-
      success">GOOD</span> {{ summary.good_count }} ({{
"%.1f"|format(summary.good_count_percentage)
      }}%)</div><div class="col-md-4"><span class="badge text-bg-warning">MEDIUM</span> {{
      summary.medium_count }} ({{ "%.1f"|format(summary.medium_count_percentage)
}}%)</div><div class="col-
      md-4"><span class="badge text-bg-danger">HEAVY</span> {{ summary.heavy_count }} ({{
      "%.1f"|format(summary.heavy_count_percentage) }}%)</div></div><p class="small text-
muted mt-3
      mb-0">Severity is an automated software classification based on measured crack width
and configured
      thresholds; it is not a structural safety assessment.</p></div>
  <div class="card card-body mb-4 border-start border-4 border-primary"><h2
class="h5">Remark</h2><p
      class="mb-0">{{ summary.remark }}</p></div>
  <div class="card card-body mb-4"><div class="d-flex flex-wrap justify-content-between
align-items-center
      gap-2"><div><h2 class="h5 mb-1">Download reports</h2><p class="small text-muted mb-
0">Export this
      analysis and its processed images.</p></div><div class="d-flex flex-wrap gap-2"><a
class="btn btn-
      outline-primary" href="/reports/{{ analysis.analysis_id }}/csv" download>Download
CSV</a><a
      class="btn btn-outline-primary" href="/reports/{{ analysis.analysis_id }}/pdf"
download>Download
      PDF</a><a class="btn btn-primary" href="/reports/{{ analysis.analysis_id }}/zip"
download>Download
      ZIP</a></div></div></div>
  <div class="card card-body mb-4"><h2 class="h5">Severity thresholds</h2><div
class="row"><div class="col-
      md-4"><b class="text-success">Good</b><br>Mean width &lt;= {{
      analysis.severity_thresholds.good_max_width_px }} px<br>Mean width &lt;= {{
```

```
  39      "%.3f"|format(analysis.severity_thresholds.good_max_width_px /
analysis.pixels_per_mm) }}
  40      mm</div><div class="col-md-4"><b class="text-warning">Medium</b><br>Mean width &gt;
{{
  41      analysis.severity_thresholds.good_max_width_px }} px and &lt;= {{
  42      analysis.severity_thresholds.medium_max_width_px }} px<br>Mean width &gt; {{
  43      "%.3f"|format(analysis.severity_thresholds.good_max_width_px /
analysis.pixels_per_mm) }} mm and
  44      &lt;= {{ "%.3f"|format(analysis.severity_thresholds.medium_max_width_px /
analysis.pixels_per_mm) }}
  45      mm</div><div class="col-md-4"><b class="text-danger">Heavy</b><br>Mean width &gt; {{
  46      analysis.severity_thresholds.medium_max_width_px }} px or area &gt;= {{
  47      analysis.severity_thresholds.heavy_min_area_px }} px^2<br>Mean width &gt; {{
  48      "%.3f"|format(analysis.severity_thresholds.medium_max_width_px /
analysis.pixels_per_mm) }} mm or
  49      area &gt;= {{ "%.3f"|format(analysis.severity_thresholds.heavy_min_area_px /
(analysis.pixels_per_mm
  50      * analysis.pixels_per_mm)) }} mm^2</div></div><p class="small text-muted mt-2 mb-
0">All values are
  51      calibrated and shown in pixels and millimeters.</p></div>
  52  {% for image in images %}<div class="card mb-4"><div class="card-body"><h2 class="h5">{{
  53      image.original_filename }}</h2><p class="small text-muted">{{ image.processing_status
}} · resized
  54      dimensions {{ image.image_width }} × {{ image.image_height }} · resize scale {{
  55      "%.3f"|format(image.resize_scale or 1) }}</p><div class="workflow-times">{% for key,
value in
  56      image.workflow_times.items() %}<span>{{ key|replace("_", " ")|title }}: {{
"%.4f"|format(value) }}
  57      s</span>{% endfor %}</div><div class="row g-3"><div class="col-md-4"><p class="small
text-
  58      muted">Original</p><button class="image-preview border-0 p-0 w-100" type="button"
data-image-
  59      preview="/files/{{ analysis.analysis_id }}/{{ image.image_id }}/original" data-image-
alt="{{
  60      image.original_filename }}" data-image-caption="Original: {{ image.original_filename
}}"><img
  61      class="result-image" src="/files/{{ analysis.analysis_id }}/{{ image.image_id
}}/original"
  62      alt="Original {{ image.original_filename }}"></button></div><div class="col-md-4"><p
class="small
  63      text-muted">Annotated result with width lines</p><button class="image-preview border-
0 p-0 w-100"
  64      type="button" data-image-preview="/files/{{ analysis.analysis_id }}/{{ image.image_id
}}/annotated"
  65      data-image-alt="Annotated crack result" data-image-caption="Annotated result with
width sample
  66      lines"><img class="result-image" src="/files/{{ analysis.analysis_id }}/{{
image.image_id
  67      }}/annotated" alt="Annotated crack result"></button></div><div class="col-md-4"><p
class="small text-
  68      muted">Final segmentation mask</p><button class="image-preview border-0 p-0 w-100"
type="button"
  69      data-image-preview="/files/{{ analysis.analysis_id }}/{{ image.image_id
  70      }}/processed/segmentation_mask" data-image-alt="Final segmentation mask" data-image-
caption="Final
  71      segmentation mask"><img class="result-image" src="/files/{{ analysis.analysis_id
}}/{{ image.image_id
  72      }}/processed/segmentation_mask" alt="Final segmentation
mask"></button></div></div><div class="stage-
  73      grid">{% for stage, title in [("resized","Resized
input"),("gray","Grayscale"),("blurred","Gaussian
  74      blur"),("clahe","CLAHE"),("blackhat","BlackHat"),("binary","Otsu
  75      threshold"),("cleaned_mask","Morphological cleanup"),("segmentation_mask","Connected-
component
  76      segmentation")] %}{% if image.processed_paths.get(stage) %}<figure><button
class="image-preview
  77      border-0 p-0 w-100" type="button" data-image-preview="/files/{{ analysis.analysis_id
}}/{{
  78      image.image_id }}/processed/{{ stage }}" data-image-alt="{{ title }}" data-image-
caption="{{ title
  79      }}"><img class="result-image" src="/files/{{ analysis.analysis_id }}/{{
image.image_id
  80      }}/processed/{{ stage }}" alt="{{ title }}"></button><figcaption>{{ title
}}</figcaption></figure>{%
  81      endif %}{% endfor %}{% for key, path in image.processed_paths.items() if
key.startswith("skeleton_")
  82      %}<figure><button class="image-preview border-0 p-0 w-100" type="button" data-image-
  83      preview="/files/{{ analysis.analysis_id }}/{{ image.image_id }}/processed/{{ key }}"
data-image-
  84      alt="{{ key|replace("_", " ")|title }}" data-image-caption="{{ key|replace("_", "
")|title }}"><img
```

```
  85      class="result-image" src="/files/{{ analysis.analysis_id }}/{{ image.image_id
}}/processed/{{ key }}"
  86      alt="{{ key|replace("_", " ")|title }}"></button><figcaption>{{ key|replace("_", "
")|title
  87      }}</figcaption></figure>{% endfor %}{% for key, path in image.processed_paths.items()
if
  88      key.startswith("distance_transform_") or key.startswith("width_estimation_")
%}<figure><button
  89      class="image-preview border-0 p-0 w-100" type="button" data-image-preview="/files/{{
  90      analysis.analysis_id }}/{{ image.image_id }}/processed/{{ key }}" data-image-alt="{{
key|replace("_",
  91      " ")|title }}" data-image-caption="{{ key|replace("_", " ")|title }} (px and mm
labels
  92      included)"><img class="result-image" src="/files/{{ analysis.analysis_id }}/{{
image.image_id
  93      }}/processed/{{ key }}" alt="{{ key|replace("_", " ")|title
}}"></button><figcaption>{{
  94      key|replace("_", " ")|title }} (px / mm)</figcaption></figure>{% endfor
%}</div></div></div>{% endfor
  95      %}
  96  <div class="card card-body mb-4"><h2 class="h5">Assessment notes</h2><ul class="mb-
0"><li>Measurements
  97      are based on the uploaded image and automatic {{
"%.2f"|format(analysis.pixels_per_mm) }} pixels/mm
  98      calibration.</li><li>{{ summary.total_cracks }} crack(s) detected with a combined
length of {{
  99      "%.2f"|format(summary.total_length_mm) }} mm.</li><li>Severity distribution: Good {{
 100      "%.1f"|format(summary.good_count_percentage) }}%, Medium {{
 101      "%.1f"|format(summary.medium_count_percentage) }}%, Heavy {{
 102      "%.1f"|format(summary.heavy_count_percentage) }}%.</li><li>Review the original and
annotated images
 103      together; lighting, texture, image resolution and resizing can affect
measurements.</li></ul></div>
 104  <div class="card card-body"><h2 class="h5">Crack measurements: pixels and
millimeters</h2><p class="small
 105      text-muted">Every measurement is calibrated and shown in both units.</p><div
class="table-
 106      responsive"><table class="table table-sm align-middle"><tr><th>Crack</th><th>Area
(px^2)</th><th>Area
 107      (mm^2)</th><th>Length (px)</th><th>Length (mm)</th><th>Mean width (px)</th><th>Mean
width
 108      (mm)</th><th>Max width (px)</th><th>Max width
(mm)</th><th>Samples</th><th>Severity</th></tr>{% for
 109      item in measurements %}<tr><td>{{ item.crack_id }}</td><td>{{
"%.2f"|format(item.contour_area_px)
 110      }}</td><td>{{ "%.2f"|format(item.contour_area_mm2) }}</td><td>{{
 111      "%.2f"|format(item.skeleton_length_px) }}</td><td>{{
"%.2f"|format(item.skeleton_length_mm)
 112      }}</td><td>{{ "%.2f"|format(item.mean_width_px) }}</td><td>{{
"%.2f"|format(item.mean_width_mm)
 113      }}</td><td>{{ "%.2f"|format(item.max_width_px) }}</td><td>{{
"%.2f"|format(item.max_width_mm)
 114      }}</td><td>{{ item.width_sample_count }}</td><td><span class="badge text-bg-
warning">{{ item.severity
 115      }}</span></td></tr>{% else %}<tr><td colspan="11">No candidate regions detected. Try
a better-lit
 116      image or adjust processing thresholds.</td></tr>{% endfor %}</table></div></div>
 117  {% endblock %}
```

F.1 — static/app.css

```
   1  body { background: #f6f8fb; color: #172033; }
   2  .metric, .card { border: 0; border-radius: 14px; box-shadow: 0 3px 15px
rgba(23,32,51,.06); }
   3  .dropzone { display:flex; align-items:center; justify-content:center; min-height:180px;
border:2px dashed
   4      #8aa4c4; border-radius:14px; background:#f8fbff; cursor:pointer; }
   5  .dropzone input { display:none; }
   6  .upload-preview-item { position:relative; padding:8px; border:1px solid #dbe3ee; border-
radius:10px;
   7      background:#f8fafc; }
   8  .upload-preview-image { display:block; width:100%; height:150px; object-fit:cover; }
   9  .upload-preview-name { display:block; overflow:hidden; margin-top:6px; padding-right:4px;
text-
  10      overflow:ellipsis; white-space:nowrap; }
  11  .upload-remove-button { position:absolute; top:12px; right:12px; padding:4px 8px;
border:1px solid
  12      #dc3545; border-radius:6px; background:#fff; color:#dc3545; font-size:.75rem; font-
weight:600; line-
```

```
  13      height:1.2; cursor:pointer; box-shadow:0 1px 3px rgba(23,32,51,.2); }
  14  .upload-remove-button:hover, .upload-remove-button:focus { background:#dc3545;
color:#fff; }
  15  .result-image { width:100%; max-height:420px; object-fit:contain; background:#eef2f7;
border-radius:10px;
  16      }
  17  .stage-grid { display:grid; grid-template-columns:repeat(auto-fit,minmax(150px,1fr));
gap:12px; }
  18  .stage-grid figure { margin:0; padding:8px; background:#f8fafc; border-radius:10px; }
  19  .stage-grid .result-image { height:120px; }
  20  .stage-grid figcaption { font-size:.8rem; margin-top:5px; font-weight:600; }
  21  .image-preview { position:relative; display:block; cursor:zoom-in; }
  22  .image-preview::after { content:"Hover to enlarge · click to open"; position:absolute;
right:8px;
  23      bottom:8px; padding:3px 6px; color:#fff; background:rgba(23,32,51,.72); border-
radius:4px; font-
  24      size:.68rem; opacity:0; transition:opacity .15s ease; }
  25  .image-preview:hover::after, .image-preview:focus-within::after { opacity:1; }
  26  .image-preview img { transition:transform .15s ease, box-shadow .15s ease; }
  27  .image-preview:hover img, .image-preview:focus-within img { transform:scale(1.025); box-
shadow:0 8px 22px
  28      rgba(23,32,51,.18); }
  29  .image-lightbox[hidden] { display:none; }
  30  .image-lightbox { position:fixed; inset:0; z-index:1050; display:flex; align-
items:center; justify-
  31      content:center; padding:32px; background:rgba(7,15,30,.86); }
  32  .image-lightbox-content { width:min(1200px, 94vw); max-height:92vh; margin:0;
overflow:auto; text-
  33      align:center; }
  34  .image-lightbox-toolbar { display:flex; align-items:center; justify-content:center;
gap:8px; margin-
  35      bottom:10px; }
  36  .image-lightbox-toolbar button { min-width:38px; padding:5px 10px; border:1px solid
  37      rgba(255,255,255,.65); border-radius:5px; background:#fff; color:#172033;
cursor:pointer; }
  38  .image-lightbox-toolbar span { min-width:55px; color:#fff; font-size:.85rem; }
  39  .image-lightbox-content img { display:block; max-width:100%; max-height:82vh;
margin:auto; object-
  40      fit:contain; border-radius:10px; background:#fff; transform-origin:center center;
  41      transition:transform .12s ease; }
  42  .image-lightbox-content figcaption { margin-top:8px; color:#fff; font-size:.9rem; }
  43  .image-lightbox-close { position:absolute; top:14px; right:24px; border:0;
background:transparent;
  44      color:#fff; font-size:2.2rem; line-height:1; cursor:pointer; }
  45  .workflow { display:flex; flex-direction:column; gap:6px; max-width:760px; }
  46  .workflow span { padding:9px 12px; border-left:4px solid #0d6efd; background:#eef5ff;
border-radius:6px;
  47      font-size:.86rem; }
  48  .workflow-times { display:flex; flex-wrap:wrap; gap:6px; margin-bottom:12px; }
  49  .workflow-times span { background:#f1f5f9; border:1px solid #dbe3ee; border-radius:5px;
padding:4px 8px;
  50      font-size:.75rem; }
```

F.1 — pytest.ini

```
  1  [pytest]
  2  pythonpath = .
  3  testpaths = tests
```

F.1 — .gitignore

```
  1  .env
  2  .venv/
  3  __pycache__/
  4  *.py[cod]
  5  .pytest_cache/
  6  *.bat
  7
  8  # Runtime-generated uploads, reports, and analysis artifacts.
  9  storage/*
 10  !storage/.gitkeep
 11
 12  # Local editor and operating-system files.
 13  .idea/
 14  .vscode/
 15  .DS_Store
```

```
Thumbs.db
run_crackscope.bat.bat
*.bat
```

F.1 — README.md

```
# Crack Detaction

Crack Detaction is a Flask/OpenCV research application for identifying candidate crack-like regions in
    concrete, road, wall, and pavement images. It reports measurements in pixels and optionally converts
    them to millimeters when a valid pixels-per-millimeter calibration is provided. Its severity labels
    are configurable software categories, not professional structural safety judgments.

## Prerequisites

- Python 3.10+
- MongoDB Community Server, or a MongoDB Atlas connection string

## Setup

```powershell
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
Copy-Item .env.example .env
```

Edit `.env` with your MongoDB URI and a secret key. The default database is `crack_detection_db`. The app
    stores image stages and reports under `storage/`; those files are intentionally not stored in
    MongoDB.

Run

Automatic Windows launch

Double-click [`run_crackscope.bat`](./run_crackscope.bat). It will:

1. Create `.venv` if it does not exist.
2. Create `.env` from `.env.example` if needed.
3. Install or update the Python dependencies.
4. Start Flask on `http://127.0.0.1:5000`.
5. Wait until Flask responds and open the upload page in your default browser.

The launcher enables automatic source reload, so Python changes restart the server without manually
    closing and starting it again. Keep the **Crack Detaction Flask Server** console open while using the
    application. Close that console to stop Flask.

Manual launch

```powershell
python app.py
```

Open `http://127.0.0.1:5000`. Use **Upload** to select one or more JPG, JPEG, JFIF, or PNG files. Pixel-
    to-millimeter calibration is applied automatically using the configured default of 10 pixels/mm.
    Detection starts automatically; uploads that contain no detected crack are rejected with an error.
    Results include original, preprocessing stages, segmentation, skeletons, and annotated images. The
    analysis page provides CSV, PDF, and ZIP downloads.

Processing workflow

The pipeline resizes while preserving aspect ratio, converts to grayscale, applies Gaussian blur and
    CLAHE, computes a BlackHat transform, uses adaptive local thresholding to preserve normal crack lines
    without turning pavement texture into foreground, cleans morphology, filters connected components,
```

```
  59      preserves multiple sufficiently strong crack networks, skeletonizes each candidate,
estimates
  60      skeleton length and distance-transform width, and draws annotated results. Width is
reported from
  61      stable centerline samples: the median is used for mean width and the 95th percentile
is used for
  62      maximum width, preventing a branch or segmentation blob from dominating the result.
Synthetic tests
  63      exercise this pipeline; real photographs still require domain validation and suitable
calibration.
  64
  65  ## Tests
  66
  67  ```powershell
  68  pytest -q
  69  ```
  70
  71  If MongoDB is unavailable, the Flask process still starts and keeps newly created results
available for
  72      the current running session. Configure MongoDB for permanent history. When running
`python app.py`,
  73      source changes automatically restart Flask; set `FLASK_AUTO_RELOAD=0` to disable this
behavior.
  74
  75  ## Troubleshooting
  76
  77  - Check that MongoDB is running and `MONGO_URI` is correct.
  78  - Ensure the account can create indexes and collections.
  79  - Increase `MAX_CONTENT_LENGTH` for larger images.
  80  - Candidate detection is sensitive to lighting, texture, and the configured BlackHat and
minimum-area
  81      parameters.
```

Appendix G — Progress reports / mentor review records

The weekly progress-report schedule is shown below.

Reporting	Date
Progress Report 1	16 July 2026
Progress Report 2	23 July 2026
Progress Report 3	06 August 2026
Progress Report 4	13 August 2026
Progress Report 5	20 August 2026
Progress Report 6	27 August 2026
Progress Report 7	03 September 2026
Progress Report 8	10 September 2026
Progress Report 9	17 September 2026
Progress Report 10	24 September 2026

Figure 9.1: Progress report schedule

Appendix H — Questionnaire, consent form or data-collection instrument

Not applicable: no human participants or personal data were involved.

Appendix I — Team-member contribution statement

Not applicable: this is an individual project carried out by PATEL MAN VISHNUBHAI.